

Hybrid TK/LBIST Flow User's Manual

Software Version 2019.3

Document Revision 14

© 2012-2019 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

End-User License Agreement: You can print a copy of the End-User License Agreement from: mentor.com/eula.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: mentor.com
Support Center: support.mentor.com

Send Feedback on Documentation: support.mentor.com/doc_feedback_form

Revision History ISO-26262

Revision	Changes	Status/ Date
14	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Sep 2019
13	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released May 2019
12	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Mar 2019
11	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Dec 2018

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Mentor Graphics Technical Publication's source. For specific topic authors, contact Mentor Graphics Technical Publication department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.

Table of Contents

Revision History ISO-26262

Chapter 1

Introduction to the Hybrid TK/LBIST Flow	13
Bottom-Up Flow	14
Tessent Core Description (TCD) Flow	16
Test Point Analysis and Insertion	16
Scan Insertion and X-Bounding	17
EDT and Tessent LogicBIST IP Generation	17
LogicBIST Fault Simulation and Pattern Creation	18
Pattern Generation	18
Top-Level ICL Network Integration	19
ICL Extraction and Pattern Retargeting	19
Top-Down Flow	20
The Dofile Flow	20

Chapter 2

Test Point Analysis and Insertion	23
Test Point Analysis and Insertion Step Overview	23

Chapter 3

Scan Insertion and X-Bounding	27
Scan Insertion and X-Bounding Overview	27
X-Bounding	30
X-Bounding Control Signals (Existing or New Scan Cells)	30
Clock Selection	31
Multiple Clock Domain Handling	31
False and Multicycle Paths Handling	32
X-Sources Reaching Primary Outputs	33
X-Bounding and no_observe_point and no_control_point Attributes	33
EDT IP Handling	34
X-Bounding and the Tessent Memory BIST Controller	34
Performing Scan Insertion and X-Bounding	35
Scan Insertion and X-Bounding Command Summary	37

Chapter 4

EDT and LogicBIST IP Generation	39
EDT and LogicBIST IP Generation Overview	40
TAP Controller and LogicBIST	41
Clock Controller Connections to the EDT/LogicBIST IP	43
EDT and LogicBIST IP	44
Low Pin Count Test Controller	47

Clock Control Logic and Named Capture Procedures	52
Programmable Registers	54
Programmable Capture Procedures	54
Programmable Shift and Capture Pause Cycles	55
Low-Power Shift	57
Warm-Up Patterns	57
Single Chain Mode Logic	64
Controller Chain Mode	64
IJTAG Network in EDT/LogicBist IP	70
Burn-In Test Mode	71
Generating the EDT and LogicBIST IP	74
Generating the EDT and LogicBIST IP (TSDB Flow)	74
Generating the EDT and LogicBIST IP (Dofile Flow)	76
Dual Compression Configurations for the Hybrid IP	84
Timing Constraints for EDT and LogicBIST IP	86
Timing Constraint Generation for the Dofile Flow	86
EDT and EDT-Bypass Timing Constraints	87
LogicBIST Timing Constraints	87
EDT and LogicBIST IP Generation Command Summary	90
ECO Implementation in the Hybrid TK/LBIST Flow	91
 Chapter 5	
LogicBIST Fault Simulation and Pattern Creation	93
LogicBIST Fault Simulation and Pattern Creation Overview	93
Initial Static DFT Signal Values	95
Performing LogicBIST Fault Simulation and Pattern Creation	96
Fault Simulation When There Are Inversions	98
Example Dofiles for Core-Level Simulation	99
Fault Coverage Report for the Hybrid IP	100
Fault Simulation and Pattern Creation Command Summary	102
 Chapter 6	
Pattern Generation	105
Pattern Generation Overview	105
Pattern Generation for the TSDB Flow	107
Performing Pattern Generation for the TSDB Flow	107
Performing Pattern Generation for CCM in the TSDB Flow	109
Pattern Generation for the Dofile Flow	113
Performing Pattern Generation for the Dofile Flow	115
Performing Pattern Generation for CCM in the Dofile Flow	117
Pattern Generation Command Summary	120
Single Chain Mode Diagnosis	121
Pattern Mismatch Debugging	123
Debug Based on MISR Signature Divergence	123
Debug Based On Scan Cell Monitoring	129
Usage Examples	131

Chapter 7

Top-Level ICL Network Integration. 141

Top-Level ICL Network Integration Overview.	141
Performing Top-Level ICL Network Integration	142
Top-Level ICL Network Integration Command Summary	146

Chapter 8

ICL Extraction and Pattern Retargeting 149

ICL Extraction and Pattern Retargeting Overview	149
Performing ICL Extraction and Pattern Retargeting	150
Usage Examples for ICL Extraction and Pattern Retargeting	151
ICL Extraction and Pattern Retargeting Command Summary	154

Chapter 9

Hybrid TK/LBIST Embedded Structures 155

Shared Logic	155
Inserted Hybrid TK/LBIST IP	156
Scan Chain Masking	160
New LogicBIST Control Signals.	161
Clocking.	161
Programmable Registers Inside Hybrid IP	163
Low-Power Shift Controller	164

Chapter 10

Tessent OCC for Hybrid TK/LBIST 167

Tessent OCC TK/LBIST Flow	168
Tessent OCC for TK/LBIST Flow Configuration	169
NCP Index Decoder	169
OCC Generation and Insertion	171
Scan Insertion	173
OCC EDT/LBIST IP Creation	173
NCP Index Decoder Synthesis	175
Fault Simulation with a Tessent OCC	175
Pattern Generation with a Tessent OCC.	176
Example Tessent OCC TK/LBIST Flow.	177
Generating and Inserting the Tessent OCC	177
Inserting the Scan Chains	178
Generating the Hybrid TK/LBIST IP	179
Synthesizing and Inserting the LBIST NCP Index Decoder	181
Generating the EDT Patterns	182
Performing the LBIST Fault Simulation	183
Tessent OCC Examples	185

Chapter 11

Third-Party OCC for Hybrid TK/LBIST 191

Overview of the Third-Party OCC Flow	191
ThirdPartyOCC TCD File Syntax	193

Usage Example for the ThirdPartyOCC TCD	194
Appendix A	
EDT Pattern Generation for the Hybrid IP	199
EDT Mode Initialization with IJTAG	199
The EDT Setup iProc	199
Usage Examples	200
Appendix B	
Interface Pins	205
LogicBIST Controller Pins	205
Clock Controller Pins	206
EDT/LogicBIST Wrapper Pins	207
Segment Insertion Bit Signals	208
Appendix C	
File Examples for the Dofile Flow	209
Synthesis Script Example	209
Timing Script Example	211
ICL Example	212
PDL Example	214
Appendix D	
Getting Help	221
The Tessent Documentation System	221
Mentor Support Services	222
Third-Party Information	
End-User License Agreement	
with EDA Software Supplemental Terms	

List of Figures

Figure 1-1. Per-Core Steps (Bottom-Up Method) or Top-Down Method Steps	15
Figure 1-2. Complete Flow (Bottom-Up Method).	19
Figure 1-3. The Dofile Flow	21
Figure 2-1. Test Point Analysis and Insertion	24
Figure 3-1. Scan Insertion and X-Bounding	28
Figure 3-2. Inverted Feedback Muxes	32
Figure 4-1. EDT and LogicBIST IP Generation	40
Figure 4-2. TAP Controller	42
Figure 4-3. Clock Controller Before IP Generation	43
Figure 4-4. Clock Controller After EDT/LogicBIST IP Generation	44
Figure 4-5. Block-Level View of the EDT/LogicBIST IP	45
Figure 4-6. Final Top-Level Netlist	46
Figure 4-7. Clocking During EDT Shift Mode of Operation	47
Figure 4-8. Default Dead Cycle Pause Width	56
Figure 4-9. Dead Cycle Pause Width of 2	57
Figure 4-10. EDT/LogicBIST-Inserted Design for CCM, Segmented Scan Chains	68
Figure 4-11. EDT/LogicBIST-Inserted Design for CCM, Non-Segmented Scan Chains	69
Figure 4-12. SIBs Insertion and Integration of Cores	71
Figure 4-13. LogicBIST Controller with Burn-In Bit	73
Figure 6-1. Pattern Generation.	106
Figure 7-1. Top-Level SIB Netlist Insertion	142
Figure 7-2. Top-Level ICL Network Integration Dofile Example	144
Figure 7-3. DftSpecification Example	146
Figure 8-1. ICL Extraction and Pattern Retargeting	150
Figure 9-1. Timing Diagram for the FSM	160
Figure 9-2. Timing Diagram for LogicBIST	162
Figure 9-3. Hybrid TK/LBIST Clocking	163
Figure 9-4. Low-Power Controller	164
Figure 10-1. Modified TK/LBIST Flow for Tessent OCC	168
Figure 10-2. NCP Index Decoder Connections	170
Figure 10-3. Clock Gating With DFT Signals and OCC in the First Pass	190
Figure 10-4. Clock Gating With EDT and LogicBIST in the Second Pass.	190
Figure 11-1. Hybrid TK/LBIST Flow With Tessent Shell	191
Figure 11-2. Usage Example for Third-Party OCC	195
Figure 11-3. Usage Example for Third-Party OCC	196
Figure 11-4. Usage Example for Third-Party OCC	196
Figure A-1. Example of Tool-Generated Setup iProc	201

List of Tables

Table 3-1. Scan Insertion and X-Bounding Commands	37
Table 4-1. Output Files, EDT and LogicBIST IP Generation, TSDB Flow	78
Table 4-2. Output Files, EDT and LogicBIST IP Generation, Dofile Flow	79
Table 4-3. EDT and LogicBIST IP Generation Commands	90
Table 5-1. Initial Static DFT Signals for Fault Simulation	95
Table 5-2. Fault Simulation and Pattern Creation Commands	103
Table 6-1. Pattern Generation Commands	121
Table 7-1. Top-Level ICL Network Integration Commands	147
Table 8-1. ICL Extraction and Pattern Retargeting Commands	154
Table B-1. LogicBIST Controller Pins	205
Table B-2. Clock Controller Pins	206
Table B-3. EDT/LogicBIST Wrapper Pins	207
Table B-4. SIB Pins	208

Chapter 1

Introduction to the Hybrid TK/LBIST Flow

Using the hybrid TK/LBIST flow is based on Tessent Shell and uses the LogicTest controller you insert with hybrid logic to share EDT and Tessent LogicBIST IP to reduce hardware overhead. You can implement hybrid TK/LBIST with either the Bottom-Up Flow or the Top-Down Flow.

This manual describes the LogicBIST portion of the hybrid flow. For EDT-specific information, consult the [Tessent TestKompress User's Manual](#).

Note



In this manual, the terms “LogicBIST” and “LBIST” are used interchangeably.

Bottom-Up Flow	14
Tessent Core Description (TCD) Flow	16
Test Point Analysis and Insertion	16
Scan Insertion and X-Bounding	17
EDT and Tessent LogicBIST IP Generation	17
LogicBIST Fault Simulation and Pattern Creation	18
Pattern Generation	18
Top-Level ICL Network Integration	19
ICL Extraction and Pattern Retargeting	19
Top-Down Flow	20
The Dofile Flow	20

Bottom-Up Flow

When implementing the hybrid flow with the *bottom-up method*, you analyze each core in isolation. As soon as a core design is available, you can move on to the embedded insertion and simulation processes for this core without having to wait for the other cores, including the top-level core.

[Figure 1-1](#) shows the 5-step flow you perform on each of your cores. Note that the same steps (1-5) are used for the top-down method.

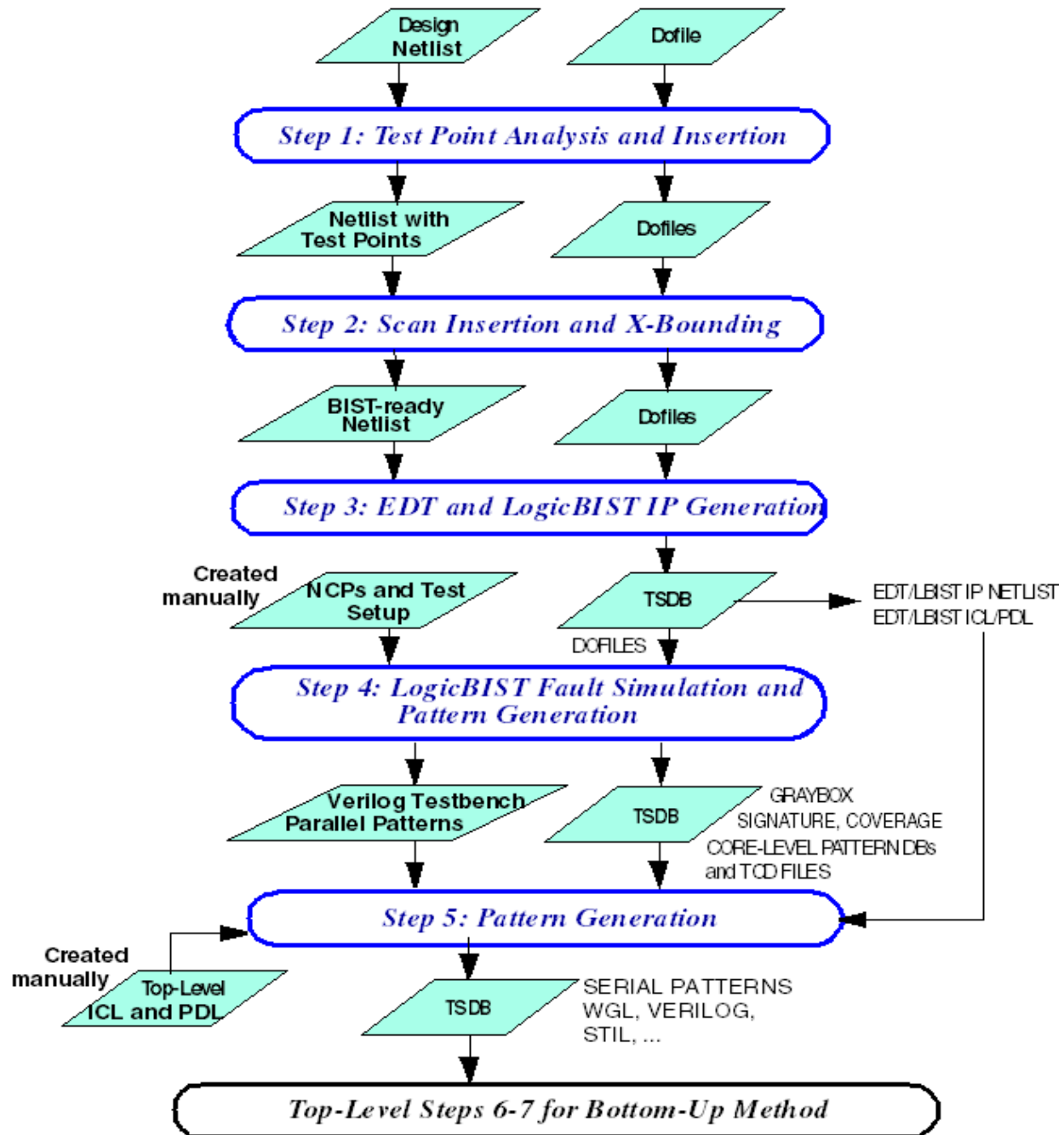
Starting at step 3 in the process, the tool stores the output files for each step into the Tessent Shell Database (TSDB). For information about the TSDB, refer to “[Tessent Shell Database](#).”

Note



For the Tessent On-Chip Clock Controller (OCC) flow, the tool automatically generates the NCPs and test setups as inputs into step 4. For more information, refer to “[Tessent OCC for Hybrid TK/LBIST](#).”

Figure 1-1. Per-Core Steps (Bottom-Up Method) or Top-Down Method Steps



Tessent Core Description (TCD) Flow.....	16
Test Point Analysis and Insertion.....	16
Scan Insertion and X-Bounding.....	17
EDT and Tessent LogicBIST IP Generation.....	17
LogicBIST Fault Simulation and Pattern Creation.....	18
Pattern Generation.....	18

Top-Level ICL Network Integration	19
ICL Extraction and Pattern Retargeting.	19

Tessent Core Description (TCD) Flow

The Tessent Core Description (TCD) is a file that contains the hybrid IP core description and eliminates the use of multiple dofiles and test procedure files for pattern generation.

TCD files are created by the tool to represent specific instruments and apply to the following steps in the TK/LBIST hybrid flow:

- **EDT and LogicBIST IP Generation** — See “[Generating the EDT and LogicBIST IP \(Dofile Flow\)](#)” on page 76.
- **LogicBIST Fault Simulation and Pattern Creation** — See “[Performing LogicBIST Fault Simulation and Pattern Creation](#)” on page 96.

The TCD files are used instead of the dofile commands that describe the hardware between tool invocations. To use a TCD file, you read the file into Tessent Shell and bind it to a specific instance along with configuring the instance with parameters.

The TCD file is generated with the `write_edt_files` command along with the other hybrid logic files during hybrid IP generation. The TCD file contains the description of the generated EDT IP and Tessent LogicBIST IP. With a TCD file, Tessent Shell can automatically extract the connectivity between the hybrid IP and the chip, apply any needed adjustment to test procedures, and allow pattern generation.

If your hybrid IP can operate in multiple configurations (for example, low power, bypass, and so on), then a single TCD file contains all the configurations in contrast to the multiple EDT IP dofile usage. During pattern generation, you will be able to specify how you want those parameters of the EDT IP configured for that ATPG mode.

Note that instrument TCD files are not used during pattern retargeting; rather, you use the core-level TCD.

Test Point Analysis and Insertion

In the first step of the flow, you use Tessent Shell to read in a gate-level non-scan Verilog netlist of your design, and generate and insert test points.

Upon completion, you use Tessent Shell to write out both the modified design and the scan setup dofile, both of which you use for scan insertion and X-bounding—see “[Test Point Analysis and Insertion](#).”

Scan Insertion and X-Bounding

In the second step of the flow, you insert scan chains into the modified design containing the test points, and then you perform X-bounding.

Note



The recommended flow is to always insert the X-bounding logic after you perform scan insertion.

Subsequently, the tool writes out the test procedure file and a dofile, which contains the information you use for the EDT and Tessent LogicBIST IP generation—see “[Scan Insertion and X-Bounding](#).”

EDT and Tessent LogicBIST IP Generation

During the third step of the flow, you generate the shared EDT/Tessent LogicBIST logic RTL. One LogicBIST controller is created for all EDT/Tessent LogicBIST blocks in a core.

You can also configure the low-power scheme to control the switching activity during “shift” to reduce power consumption.

There is no TAP controller at the core level. New core-level pins corresponding to the SIB control signals, tck, and LBIST scan I/O are created in this step. The core-level Verilog patterns operate these pins directly. These pins are subsequently connected to the TAP controller at the top level of the design—see “[Top-Level ICL Network Integration](#).”

As part of IP generation, the tool writes out the following files to the TSDB:

- **ICL File** — Consists of ICL module description for the LBIST controller and all EDT/Tessent LogicBIST blocks tested by this controller.
- **PDL File** — Contains iProcs at the core level that use the ICL modules written out.

During IP generation, the generated ICL file describes only the LogicBIST and EDT modules. The ICL file includes the core-level pin names and connectivity found from the core-level design netlist. The extracted ICL file is used during top-level pattern generation—see “[ICL Extraction and Pattern Retargeting](#).” Verilog patterns can be written out in this step and simulated to verify the test operation at the core level.

For complete information, see “[EDT and LogicBIST IP Generation](#).” In **steps 1 through 3** of the flow, new top-level test pins are added and/or existing top-level test pins controlled internally by the EDT/Tessent LogicBIST IP.

Logic Synthesis

You must synthesize all the EDT/Tessent LogicBIST blocks and the common LogicBIST controller. Synthesis is fully automated. You execute the run_synthesis command to synthesize

the controllers and test logic, located in the TSDB, and integrate them into the gate-level design. When the `run_synthesis` command completes successfully, a concatenated netlist of the design that contains the synthesized test logic and modified design modules is automatically created and placed in the `dft_inserted_designs` directory of the TSDB.

Refer to the [run_synthesis](#) command for more information.

LogicBIST Fault Simulation and Pattern Creation

During the fourth step of the flow, you perform the tasks of fault simulation and saving the parallel LogicBIST patterns.

The *corelevel* fault simulation run computes the test coverage for the core, MISR signature, and power consumption. The following files are generated and stored in the TSDB and are used later in top-level ICL extraction and pattern retargeting. Refer to “[logic_test_cores](#)” for more information.

- **PatternDB File** — Contains the pattern data and the relevant LBIST register values per pattern such as PRPG, MISR, and low-power registers.
- **Tessent Core Description file (TCD)** — Contains the description of the core that is relevant for the LogicBIST mode of operation.

For subsequent diagnosis, you can use Tessent Diagnosis to perform diagnosis with the EDT patterns.

See “[LogicBIST Fault Simulation and Pattern Creation](#)” for complete information.

Pattern Generation

In the fifth step of the flow, you generate core-level patterns for the bottom-up method and top-level patterns (including a Verilog test bench) for the LogicBIST controller for the top-down method. This step also generates the chip-level serial patterns that can be applied from the tester.

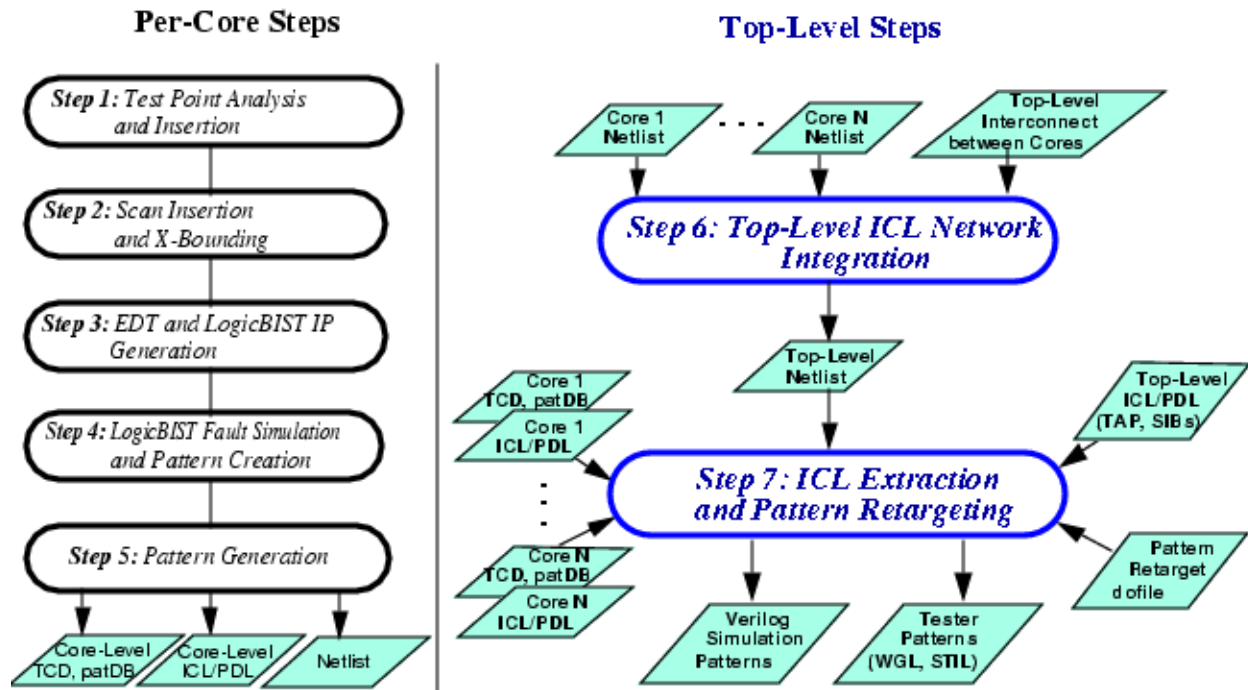
The tool supports all formats currently supported for ATPG. See “[Pattern Generation](#)” for complete information.

After you implement TK/LBIST on each core of your design, you must perform **steps 6-7** to bring information of your embedded structures to the top level. [Figure 1-2](#) illustrates the complete TK/LBIST hybrid flow when using the bottom-up method. Though not shown in the figure, all the input and output files are stored in the TSDB.

- On the left side — **Per-Core Steps**. **Steps 1-5** are repeated once for each core in your design. These are the steps that are discussed following [Figure 1-1](#).

- On the right side— **Top-Level Steps**. Steps 6-7 are performed once with the top-level netlist containing all the cores.

Figure 1-2. Complete Flow (Bottom-Up Method)



Top-Level ICL Network Integration

In the sixth step of the bottom-up flow, you use the top-level netlist that instantiates all the LogicBIST implemented cores.

You insert IJTAG-compliant SIBs to provide access to each of the LogicBIST cores as well as to connect these SIBs and cores to the top-level TAP controller. We recommend that you shadow each core by a separate SIB to provide maximum flexibility for test scheduling. You can also connect EDT control signals from the core to the top level at this time. Additionally, you can create a Tcl script that uses Tessent Shell netlist editing features to automate this task. See “[Top-Level ICL Network Integration](#)” for complete information.

ICL Extraction and Pattern Retargeting

In the seventh and final step of the bottom-up flow, you use the fully integrated top-level netlist, top-level ICL/PDL files for your IJTAG instruments such as the TAP controller and clock controller, and the per-core LogicBIST files generated at the core level.

ICL extraction can again be used here instead of manually creating the top-level ICL describing the SIB access network and connectivity between your instruments and the LogicBIST cores. The final tester patterns can be written out in any of the supported formats.

You must provide the pattern retargeting dofile that includes all LogicBIST cores and the desired scheduling. See “[ICL Extraction and Pattern Retargeting](#)” for complete information.

Top-Down Flow

When implementing the hybrid flow with the *top-down method*, the analysis is performed on the entire pre-DFT inserted view of your chip. Each core is analyzed within the respective context of its parents.

For the top-down method, you use the same steps as when you implement the hybrid TK/LBIST flow on the core level except without top-level ICL network insertion and extraction or ICL pattern generation (steps 6 and 7).

[Figure 1-1](#) illustrates the steps when using the top-down method. See “[Bottom-Up Flow](#)” for more information. You define blocks for insertion and run steps 1 through 5 as follows:

- [Test Point Analysis and Insertion](#) and [Scan Insertion and X-Bounding](#) — These steps are similar to those performed using the bottom-up method.
- [EDT and LogicBIST IP Generation](#) — EDT/LogicBIST hybrid IP generation is almost the same as the step used with the bottom-up method except that the TAP controller is present at the top level.
- [LogicBIST Fault Simulation and Pattern Creation](#) and [Pattern Generation](#) — These steps are similar to those performed using the bottom-up method.

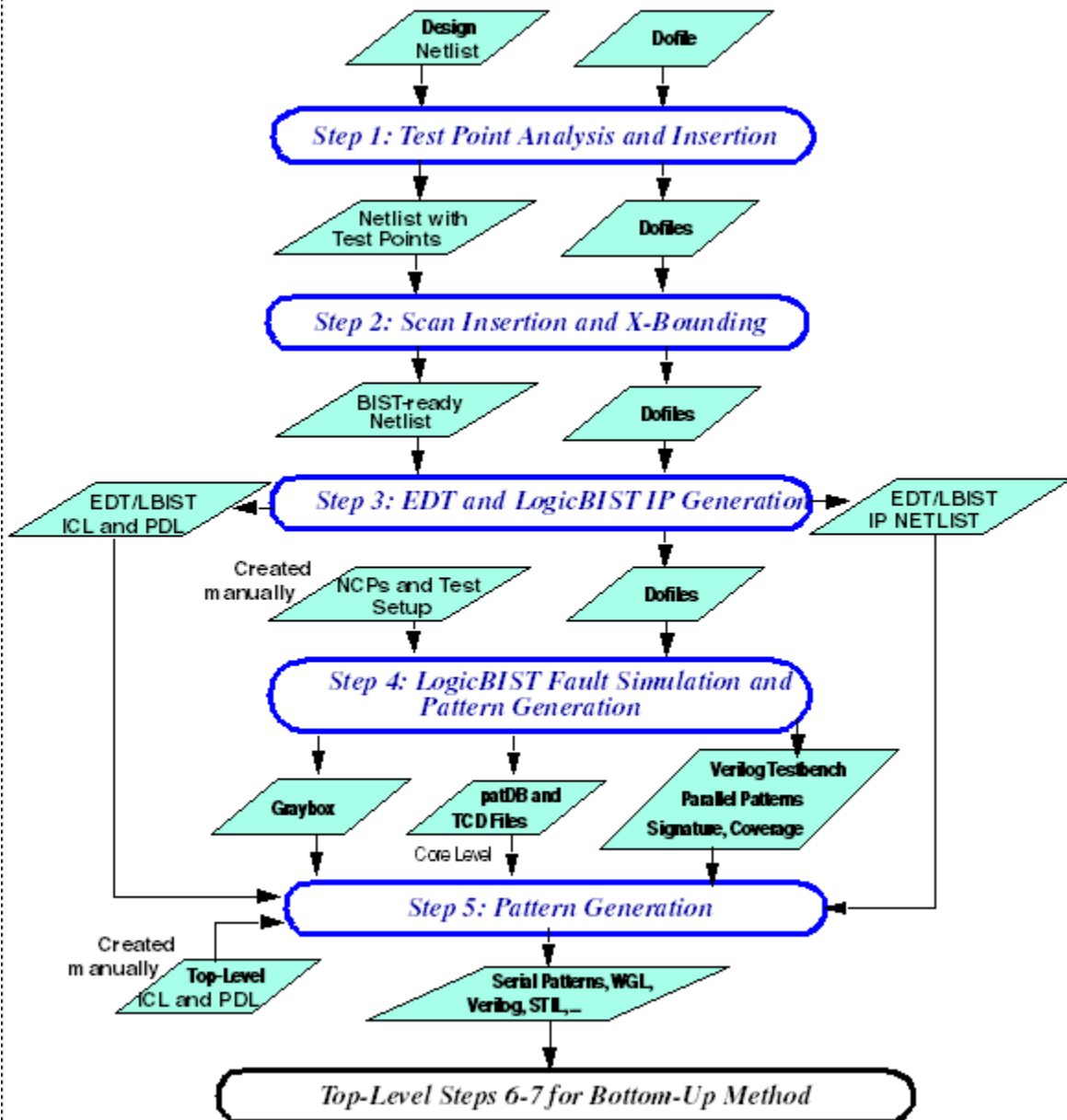
The Dofile Flow

The hybrid TK/LBIST flow supports a dofile flow that does not use the TSDB. In the dofile flow, the tool does not write output files to the TSDB. The five steps used for the TSDB bottom-up method remain the same, as do the additional two steps used for the top-down method.

The following figure shows the steps you perform on each of your cores with the dofile flow. Refer to “[Bottom-Up Flow](#)” for details. For the dofile flow the differences are:

- In step 3, the output EDT and LogicBIST ICL and PDL files, and the EDT/LogicBIST IP netlist are not stored in the TSDB. These files are direct inputs to pattern generation (step 5).
- In step 4, the output graybox, patternDB file, TCD file, testbenches, patterns, and so on, are direct inputs to pattern generation (step 5). They are not stored in the TSDB.
- During step 3, logic synthesis is not fully automated. To synthesize the EDT/Tessent LogicBIST blocks and the common LogicBIST controller, you must use the synthesis script that the tool generates. You use the synthesized gate-level netlist output from your synthesis tool (for example, the output of the logic synthesis step with Synopsys Design-Compiler) as input to the next step of the flow.

Figure 1-3. The Dofile Flow



Chapter 2

Test Point Analysis and Insertion

In the first step of the flow, you use Tessent Shell to read in a gate-level non-scan Verilog netlist of your design, and generate and insert test points.

Test Point Analysis and Insertion Step Overview. 23

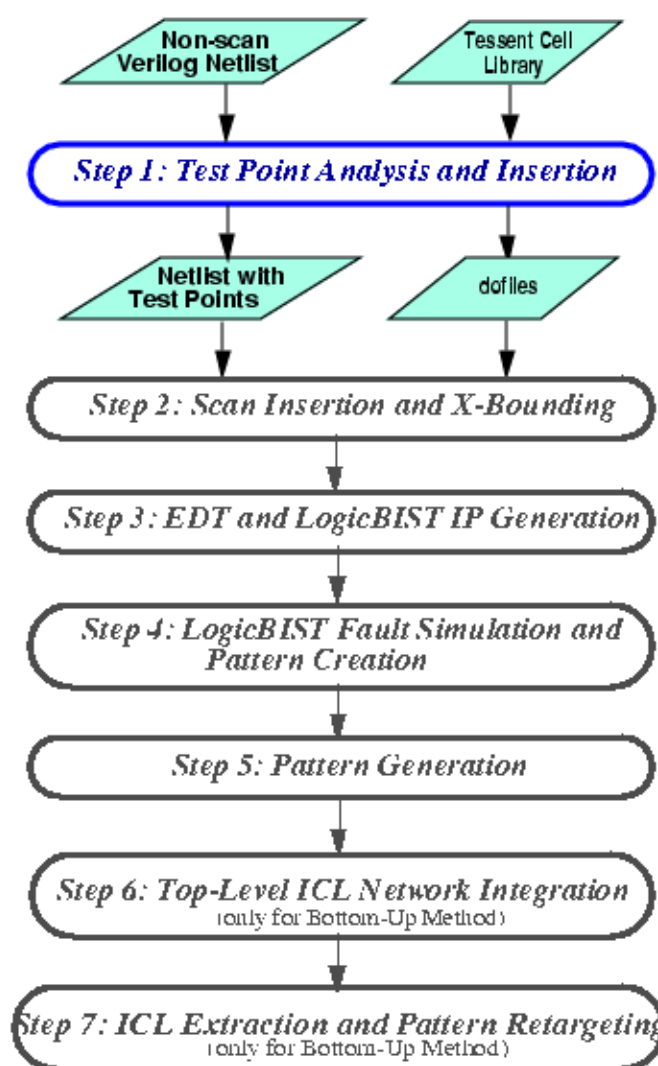
Test Point Analysis and Insertion Step Overview

In the test point analysis and insertion step of the hybrid TK/LBIST flow, you generate and insert test points into the netlist to achieve high test coverage.

During test point analysis and insertion, you add random pattern test points to certain locations in your design. By adding these test points, you can increase the testability of the design by improving controllability or observability.

[Figure 2-1](#) illustrates this step within hybrid TK/LBIST flow.

Figure 2-1. Test Point Analysis and Insertion



Note

 You generate and insert the test points as a separate step from scan insertion—you cannot perform the two operations together.

At the conclusion of test point analysis and insertion, you write out both the modified design netlist containing the test points and a dofile containing all the necessary steps for the next step of the flow—see “[Scan Insertion and X-Bounding](#).”

Requirements

Performing test point analysis and insertion has certain requirements. You must adhere to the following:

- Test point insertion needs a gate-level Verilog netlist and a Tessent Cell Library.

- You can read in functional SDC so the tool omits adding any test points to multicycle paths or false paths—see the [read_sdc](#) command.
- The tool identifies potential scan candidates for correct controllability/observability analysis. To ensure that eventual non-scan cells are not used as the destination for observe points or source for control points, you should declare all the non-scan memory elements during test point insertion using the [add_nonscan_instances](#) command.
- You should define black boxes using the [add_black_boxes](#) command so that test point analysis can incorporate this information.
- If you are performing test point analysis on a pre-scan netlist that has unconnected clock gaters, you should add the “[set_clock_gating on](#)” command to your dofile.

For more information, refer to “[Test Points for LBIST Test Coverage Improvement](#)” in the *Tessent Scan and ATPG User’s Manual*.

Chapter 3

Scan Insertion and X-Bounding

In the second step of the flow, you insert scan chains into the modified design containing the test points and then perform X-bounding.

Scan Insertion and X-Bounding Overview	27
X-Bounding	30
X-Bounding Control Signals (Existing or New Scan Cells)	30
Clock Selection	31
Multiple Clock Domain Handling	31
False and Multicycle Paths Handling	32
X-Sources Reaching Primary Outputs	33
X-Bounding and no_observe_point and no_control_point Attributes	33
EDT IP Handling	34
X-Bounding and the Tessent Memory BIST Controller	34
Performing Scan Insertion and X-Bounding	35
Scan Insertion and X-Bounding Command Summary	37

Scan Insertion and X-Bounding Overview

You insert scan chains into the modified netlist that contains the test points and then perform X-bounding using Tessent Shell (or a third-party tool).

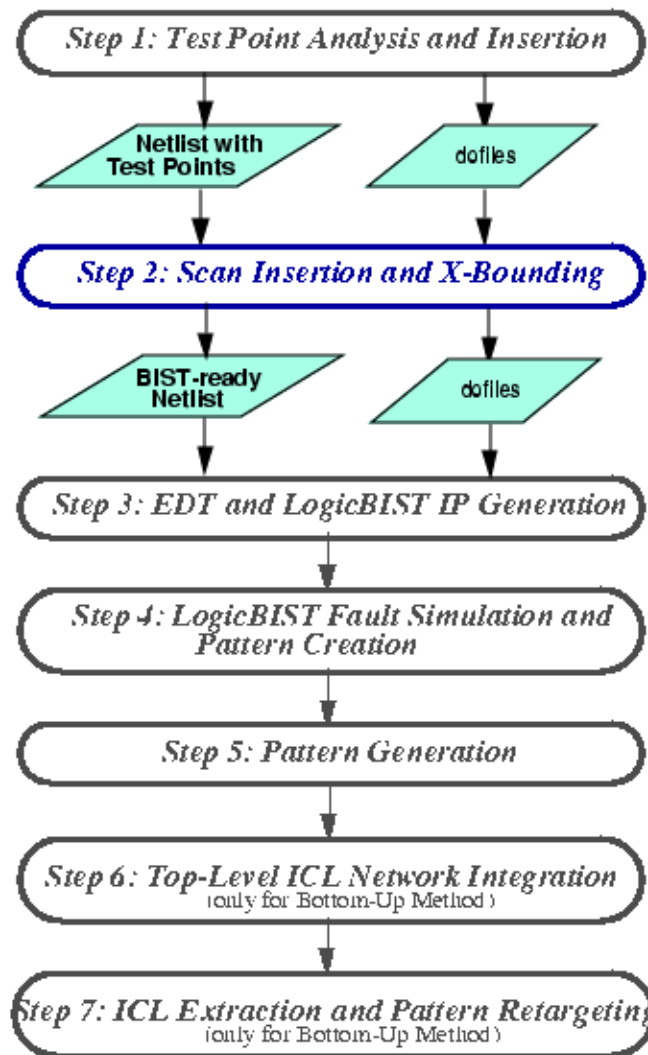
Note



The recommended flow is to always insert the X-bounding logic after you perform scan insertion.

Figure 3-1 illustrates the flow.

Figure 3-1. Scan Insertion and X-Bounding



You should adhere to the following during this step:

- **Inputs** — The modified design netlist with test points and the dofile you have generated using the Tessent Shell [write_scan_setup](#) command.
- **MCPs and Failing Paths** — You read in SDC to identify MCP/FP and add DFT for preventing capture of any transition on such paths.
- **Cross-Domain Paths** — The tool X-bounds cross-domain paths.
- **X-Bounding** — You must perform X-bounding. Perform this step after scan insertion.
- **Outputs** — A LogicBIST-ready netlist, test procedure file, and the dofile for the existing scan chains.

After you complete the scan insertion and X-bounding, you use Tessent Shell to write out the test procedure file and dofile for the existing design with scan chains that you use as input to the next step of the flow, “[EDT and LogicBIST IP Generation](#).”

Requirements for Using a Third-Party Scan Insertion Tool

Tessent Shell must make certain assumptions about which flip-flops in the design are converted to scan and which cells remain non-scan. If you are using a third-party scan insertion tool, then these assumptions might not be correct.

Specifically, the primary factors in determining the conversion of non-scan flops to scan flops are the S-rule DRC violations and the availability of a suitable scan model. In general, flops with S-rule violations are not converted to scan cells unless you use the following command:

set_test_logic -clock on -set on -reset on

You might need to read the design into Tessent Shell and go through DRC. You can subsequently post DRC use the [report_scan_elements](#) command to report the exact status for each flop in the design.

Scan Insertion and X-Bounding Limitations

Be aware of the following:

- If low power is used in the hybrid TK/LBIST flow, the scan chain length must be equal to or greater than the decompressor size. The minimum decompressor size in the hybrid flow is 31. Therefore, the scan chain length cannot be less than 31. The reason for this requirement is that the size of the low-power register and the size of the decompressor are the same; to initialize the low-power register, the shift length must be equal to or greater than the decompressor size. If the tool generates a larger decompressor, say a size of 62 bits, the required minimum shift length is 62.
- X-bounding analysis does not take into account wrapper cells identified with the `analyze_wrapper_cells` command and will insert X-bounding multiplexers at the primary input pins that feed the wrapper cells. The workaround is to perform wrapper chain identification and insertion in a separate pass, prior to X-bounding. In addition, you must constrain the scan enable signal for the input wrapper chains to keep these chains in “shift mode” during the capture cycles. This pin constraint prevents insertion of extra X-bounding multiplexers at the primary input pins that drive the wrapper cells in functional mode.

X-Bounding

After scan insertion, you *must* perform the X-bounding to prohibit all X generators (that is, non-scan cells, black boxes, and primary inputs) from reaching a scan cell. The source of the x-bounding mux could either be existing scan cells in the design or newly-inserted scan cells.

Tessent Shell performs the following operations during X-bounding:

- X-sources are identified as part of DRCs, specifically [E5 violations](#).
- All identified X-sources are bounded from reaching scan cells.
- A multiplexer is introduced to block propagation of an X.
- To increase controllability, the second input of the mux is driven by scan cell.
- Clocks for destination cells are analyzed to choose clocks for new scan cell or choose an existing scan cell.

Note

 (1) When an X-source does not reach a scan(able) flop, and feeds only a small number of combinational gates that may drive output pins, then this signal is not x-bounded, but all the affected logic is marked as `no_control_point` and `no_observe_point`, with `no_control_reason` and `no_observe_reason` set to `xbounding`.

(2) Sometimes, it is not possible to bound the signal directly at the source. In that case, any combinational logic that is driven by the X-source and is upstream of the X-bounding mux is also marked as `no_control_point` and `no_observe_point`, with `no_control_reason` and `no_observe_reason` set to `xbounding`.

X-Bounding Control Signals (Existing or New Scan Cells)	30
Clock Selection	31
Multiple Clock Domain Handling	31
False and Multicycle Paths Handling	32
X-Sources Reaching Primary Outputs	33
X-Bounding and <code>no_observe_point</code> and <code>no_control_point</code> Attributes	33
EDT IP Handling	34
X-Bounding and the Tessent Memory BIST Controller	34

X-Bounding Control Signals (Existing or New Scan Cells)

By default, the tool identifies existing scan cells with the correct clock domain and uses the output from these flip-flops to drive the test mode input of the X-bounding multiplexer.

Details about how the tool selects the clocks are in the “[Clock Selection](#)” section.

You also can choose to drive the test mode input of the new multiplexers from new scan cells using the `-connect_to new_scan_cell` switch of the [set_xbounding_options](#) command. The clock signal that drives the new scan cell is selected using the same algorithm that applies when using existing scan cells. The new scan cells are merged into existing chains whenever possible. The output of the flip-flops directly drives the data input of the new flip-flops.

Clock Selection

The tool analyzes the location of each X-bounding multiplexer to identify the clocks if a new flip-flop is chosen to drive the multiplexer. The tool looks at clocks for the controlling flip-flops that feed into this location and also the clocks for the flip-flops that are fed from this signal.

When only a single clock domain is involved, that clock is used to drive the test points. However, when multiple clock domains are involved, the behavior depends on the definition of the false or multicycle paths. X-bounding could potentially introduce a new false path that would not be bounded because the false path did not exist in the original netlist. Therefore, when an SDC file is loaded, the tool uses static bounding (forcing a constant 0 or 1 via an AND or OR gate) to avoid creating a new false path. When no false or multicycle paths are defined, the tool chooses the clock domain that is most frequently used by the memory elements in the fanout of the X-source.

Multiple Clock Domain Handling

By default, X-bounding by the tool does not guard cross-clock domain paths. However, if you reference these paths in the SDC file (for example, `set_false_path -from CLK1 -to CLK2`), then the tool adds the appropriate bounding hardware at the destination flop.

For cases when you create at-speed capture procedures that do not explicitly exercise these paths, none of these paths need to be bounded. The following command and switch:

`set_xbounding_options -exclude_sdc_cross_domain_path on`

modify the tool’s behavior such that paths that meet the following criteria are *not* X-bounded:

1. Identifies all the clock domains that feed into the test point location.
2. Identifies all the clock domains that observe the signal from the test point.
3. If there is no overlap between the clocks identified in (1) and (2), then this is considered a cross clock domain path that cannot be activated by pulsing only a single clock, and it is excluded from X-bounding.

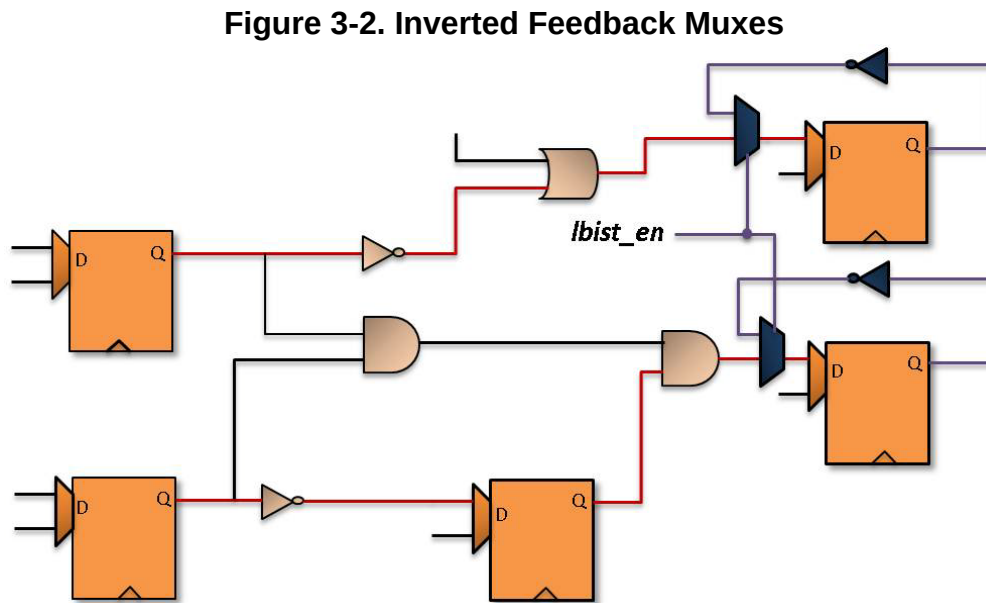
False and Multicycle Paths Handling

The tool can also insert logic to block false and multicycle paths to inhibit these from contributing Xs during LogicBIST as follows:

- All destination flops are identified from the SDC file you load into the tool using the `read_sdc` command.
- The destination flops are replaced with holding flops to prevent capturing Xs—see “Destination Flop Identification” in the following section.
- The holding mux has an inverter in the feedback loop to enable transition coverage for the downstream logic. In Figure 3-2, *lbist_en* is the signal to control the muxes. However, you can use a completely new enable signal to control the select line of these muxes. This would allow enabling this logic independent of the LogicBIST run.

The same enable signal (which defaults to *lbist_en* but can be overridden using the *mcp_bounding_en* dft signal) will be used to control the X-bounding of both false paths as well as multicycle paths.

Figure 3-2 shows the holding muxes in **black**.



Re-circulating muxes with an inverter in the feedback path are inserted on the D input of the destination scan cells of false and multicycle paths. The inversion ensures that the destination scan cell output has a transition during broadside test to achieve higher coverage.

Destination Flop Identification

Each false/multicycle path statement in the SDC file is analyzed in the region identified by that particular statement. The tool subsequently analyses the ability of signals from the marked

region to propagate to an observation point. These gates on the propagation path are marked as being in the false or multicycle path effect cone.

From this information, the tool looks at the data input(s) of scannable flops and wrapper cells to determine if they are marked in any false or multicycle path effect cone. These gate inputs are bounded using a mux and an inverted feedback loop. In addition, if the wrapper cells are constructed of separate library cells (for example a separate DFF and MUX), then the tool searches the fan-in of the DFF to find a non-scan path input that can be bounded. Note: this only works if the wrapper cells are part of existing traced scan chains since that allows the tool to differentiate between the shift path and the capture path.

In addition, the tool checks the set and reset ports of each flop and disables any set/reset ports that are also marked as false or multicycle paths. In this case, however, a combinational gate (either an AND gate or an OR gate) is used to force the set/reset port into the off state.

The inverting loop-back path that is inserted during X-bounding may still be considered as a false path during fault simulation if the data-input of the flop is defined as the “-to” gate for the false path. To avoid capturing an X at the input of this flop during fault simulation, the tool will search forward from the mcp bounding enable signal, identify these inverting loop-back paths, and modify the simulation such that these paths are no longer treated as false paths. The source of the mcp bounding enable signal is typically specified with the `set_dft_signals` command during the logic BIST insertion steps. In addition, the `mcp_bounding_en` port at the output of the logic BIST instrument is also specified in the `tcd` files and will be considered during this analysis.

X-Sources Reaching Primary Outputs

By default, the tool doesn't guard an X source that only reaches one or more primary outputs but doesn't reach any scan cell.

If there is a considerable amount of logic that can be observed at a primary output then inserting an observe point at such a primary output is beneficial for improving the test coverage of LogicBIST. The tool analyzes the amount of logic that can be observed at a primary output. If there are enough library cells that can be observed at a primary output then the tool will guard an X source that can reach this primary output. Test point analysis can then safely insert an observe point at this primary output.

X-Bounding and `no_observe_point` and `no_control_point` Attributes

X-bounding adds logic to block the propagation of the X-sources to scan cells.

The X-bounding muxes are inserted as close to the X-source as possible. However, there may be combinational logic between the X-source and the input of the X-bounding mux. If the test point analysis algorithm inserts an observe point at any of these locations, this observe point

would capture the signal from the X-source. Therefore, the X-bounding algorithm marks all gate-pins in the combinational fan-in cone of the functional mode input of the X-bounding mux and sets the `no_observe_point` attribute value to true at those locations. The `no_observe_reason` attribute for these gate-pins will return “xbounding” as the reason.

In addition to marking these locations with the `no_observe_point` attribute, the tool also sets the `no_control_point` attribute. Inserting a control point at any of these locations cannot improve test coverage during Logic BIST because all the logic in the fanout of the control point will be blocked by X-bounding muxes and is unobservable. The `no_control_reason` attribute for these gate-pins will also return “xbounding” as the reason.

EDT IP Handling

When EDT and LogicBIST IP have been inserted in the design the tool will use information about these cores to avoid guarding any “perceived” X-sources in this IP.

This usually happens when you are using the pre-synthesis flow where the EDT IP is inserted and synthesized together with the design. The X-bounding analysis happens only after scan stitching has been completed and the tool sees a gate-level design. For more information, see [ICL Extraction and Pattern Retargeting](#).

In the absence of such information, the tool will use information about the scan chains to avoid inserting bounding logic in the scan path and in the paths from the scan chain outputs to the primary outputs that are used as channel outputs. In some cases, particularly when third-party IP is involved, the tool may conservatively treat a functional primary output as a channel output. In such situations it will not guard an X source that reaches that primary output—see [X-Sources Reaching Primary Outputs](#). You should provide the TCD for the EDT IP so that the tool can easily identify these instruments and mark them as non-scan sources. Another way to identify the EDT instances in the design is to use the `set_edt_instances` command when the TCD flow is not used.

X-Bounding and the Tessent Memory BIST Controller

The Tessent Memory BIST controller has been modified to include a mechanism that disables all the clocks during capture. This is intended to avoid any requirement to add X-bounding hardware for false and multi-cycle paths.

To recognize this hardware during the X-bounding step, you must do the following:

- During the dft insertion stage (memory bist + logic bist), you must use "add_dft_signals" and specify the following:

add_dft_signals async_set_reset_static_disable

- During dft insertion, you must modify the `dft_specification` for the memory BIST controller, and set the `gate_bist_clock_in_functional_mode` property to "with_clock_gating_cell" in order to create a memory BIST controller that includes the `mcp_bounding_en` static dft signal.
- During the scan insertion stage, do the following:

```
set_static_dft_signal_values mcp_bounding_en 1
```

Note

 This is the signal to disable all clocks during capture.

```
set_static_dft_signal_values async_set_reset_static_disable 1
```

Note

 This prevents generating any transitions at the set/reset ports.

For more information about the `gate_bist_clock_in_functional_mode` property, refer to the [ControllerOptions](#) and [AdvancedOptions](#) sections of the [DftSpecification/MemoryBist](#) wrapper in the *Tessent Shell Reference Manual*.

Note

 Scannable flops inside the MemoryBIST controller that hold state during capture will be identified as constant and the tool will ignore them during X-bounding analysis.

Performing Scan Insertion and X-Bounding

Before inserting the scan and X-bounding logic, you characterize the scan signals with the `set_scan_signals` command and X-bounding with the `set_xbounding_options` command.

Prerequisites

- Design netlist with test points and dofile outputted by Tessent Shell during the “[Test Point Analysis and Insertion](#).”

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

After invocation, the tool is in unspecified setup mode. You must set the context before you can use the X-bounding and scan insertion commands.

2. Set the tool context to scan insertion using the [set_context](#) command as follows:

```
SETUP> set_context dft -scan
```

3. Load the non-scan gate-level Verilog netlist containing the test points using the [read_verilog](#) command.

SETUP> read_verilog my_modified_netlist.v

4. Load one or more cell libraries into the tool using the [read_cell_library](#) command.

SETUP> read_cell_library atpg.lib

5. Using the dofile command, load the Tessent Shell tool-produced dofile you generated during test point analysis and insertion. For example:

SETUP> dofile lbist_scan_setup.dofile

This dofile loads the netlist and the Tessent cell library. It also includes the necessary commands to set up the circuit for DRC.

6. If required in your design flow, load the SDC file using the [read_sdc](#) command.

SETUP> read_sdc my_sdc

The SDC file should describe false and multicycle paths that should be blocked during LogicBIST.

7. Set the scan insertion options using the [set_scan_signals](#) command.

SETUP> set_scan_signals -ten t_enable

8. Set the X-bounding options with the [set_xbounding_options](#) command. For example:

SETUP > set_xbounding_options -xbounding_enable my_enable1

This command specifies the name of the top-level pin that will enable the X-bounding signals.

9. Change the tool's system mode to analysis using the [set_system_mode](#) command.

SETUP> set_system_mode analysis

During the transition from setup to analysis mode, the tool performs design rule checking and inserts the scan chains.

10. Perform the X-bounding analysis using the [analyze_xbounding](#) command.

ANALYSIS> analyze_xbounding

The tool reports a summary of how many bounding muxes were added.

11. Optionally report the results of the X-bounding analysis using the [report_xbounding](#) command.

ANALYSIS> report_xbounding

12. Perform scan insertion and X-bounding logic insertion using the [insert_test_logic](#) command.

ANALYSIS> insert_test_logic

This command modifies the internal representation of the netlist to include the X-bounding muxes, as well as performing scan cell replacement and stitching. This command triggers a transition from analysis to insertion mode.

13. Write out the modified design using the [write_design](#) command. For example:

```
INSERTION> write_design -output design_with_scan.v
```

14. Write out the test procedure file and dofile using the [write_atpg_setup](#) command as follows:

```
INSERTION> write_atpg_setup my_scan_setup
```

You need this file for subsequent steps in the process.

Results

In this example, the tool produces the following two files that are to be used as input for the next step of the flow:

- *my_scan_setup.dofile*
- *my_scan_setup.testproc*

Now you are ready to perform “[EDT and LogicBIST IP Generation](#).”

Scan Insertion and X-Bounding Command Summary

Tessent Shell enables you to perform scan insertion and X-bounding.

[Table 3-1](#) lists the Tessent Shell scan insertion and X-bounding commands.

Table 3-1. Scan Insertion and X-Bounding Commands

Command	Description
analyze_xbounding	Performs X-bounding analysis.
insert_test_logic	Inserts the test structures you define into the netlist to increase the design’s testability.
read_cell_library	Loads one or more cell libraries into the tool.
read_sdc	Reads in the SDC file that describes the false and multicycle paths that should be blocked during LogicBIST.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
report_scan_elements	Reports information and testability data for the sequential instances in the design.
report_xbounding	Reports the X-sources and the scan cells used in bounding.

Table 3-1. Scan Insertion and X-Bounding Commands (cont.)

Command	Description
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.
set_scan_signals	Sets the pin names of the scan control signals.
set_system_mode	Specifies the operational state you want the tool to enter.
set_xbounding_options	Enables X-bounding and sets X-bounding parameters.
write_atpg_setup	Writes a test procedure file and the dofile for the existing scan chains.
write_design	Writes out the modified netlist.
write_scan_setup	Writes out a dofile containing all necessary steps for scan insertion.

Chapter 4

EDT and LogicBIST IP Generation

In the third step of the flow, you generate the shared EDT/Tessent LogicBIST RTL.

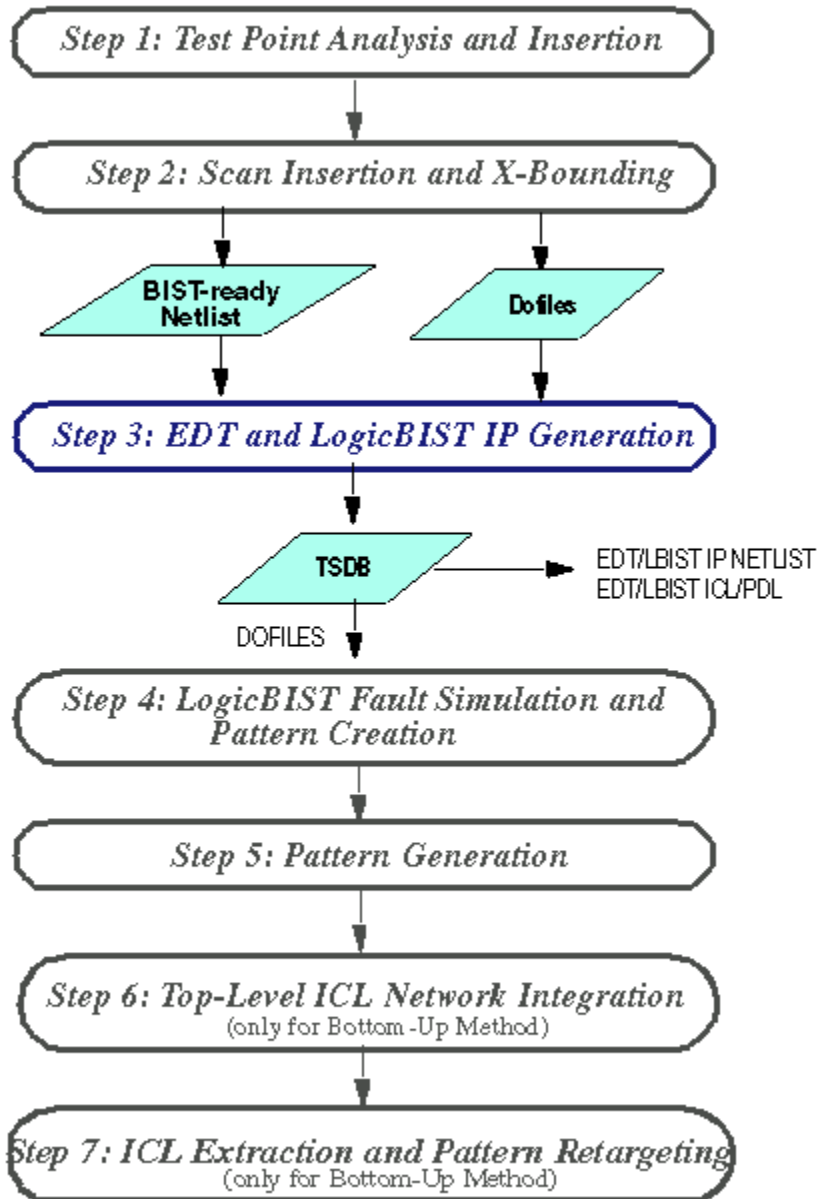
EDT and LogicBIST IP generation do not support writing out the EDT and LogicBIST IP in VHDL format. Currently, the RTL description of the IP can be written out only in Verilog format.

EDT and LogicBIST IP Generation Overview	40
TAP Controller and LogicBIST	41
Clock Controller Connections to the EDT/LogicBIST IP	43
EDT and LogicBIST IP	44
Low Pin Count Test Controller	47
Clock Control Logic and Named Capture Procedures	52
Programmable Registers	54
Programmable Capture Procedures	54
Programmable Shift and Capture Pause Cycles	55
Low-Power Shift	57
Warm-Up Patterns	57
Single Chain Mode Logic	64
Controller Chain Mode	64
IJTAG Network in EDT/LogicBist IP	70
Burn-In Test Mode	71
Generating the EDT and LogicBIST IP	74
Generating the EDT and LogicBIST IP (TSDB Flow)	74
Generating the EDT and LogicBIST IP (Dofile Flow)	76
Dual Compression Configurations for the Hybrid IP	84
Timing Constraints for EDT and LogicBIST IP	86
Timing Constraint Generation for the Dofile Flow	86
EDT and EDT-Bypass Timing Constraints	87
LogicBIST Timing Constraints	87
EDT and LogicBIST IP Generation Command Summary	90
ECO Implementation in the Hybrid TK/LBIST Flow	91

EDT and LogicBIST IP Generation Overview

EDT and LogicBIST IP generation stores several files in the TSDB. The dofiles are used for LogicBIST fault simulation and pattern creation (step 4) while the ICL, PDL, and IP netlist files are used as inputs for pattern generation (step 5).

Figure 4-1. EDT and LogicBIST IP Generation



You use Tessent Shell to generate the shared EDT/LogicBIST RTL. Modular EDT is supported in this step, where both shared EDT/LogicBIST IP per block and LogicBIST controller are inserted at the same time.

When using the bottom-up method, there is no TAP controller at the core level. The new core-level pins corresponding to the SIB control signals and TCK, and LogicBIST scan I/O are created at this step. The core-level Verilog patterns operate these pins directly. These pins will later be connected to the TAP controller at the top level of the design—see “[Top-Level ICL Network Integration](#).”

As part of IP generation, the following files are written out:

- The *ICL file* consists of ICL module description for the LogicBIST controller and all EDT blocks tested by this controller.
- The *PDL file* contains iProcs at the core level that use the ICL modules written out.

During the IP generation step, the generated ICL file describes only the LogicBIST and EDT modules. This extracted ICL includes the core-level pin names and connectivity found from the core-level design netlist. The extracted ICL is used during top-level pattern generation—see “[ICL Extraction and Pattern Retargeting](#).” Verilog patterns can be written out in this step and simulated to verify the test operation at the core level.

TAP Controller and LogicBIST	41
Clock Controller Connections to the EDT/LogicBIST IP	43
EDT and LogicBIST IP	44
Low Pin Count Test Controller	47
Clock Control Logic and Named Capture Procedures	52
Programmable Registers	54
Programmable Capture Procedures	54
Programmable Shift and Capture Pause Cycles	55
Low-Power Shift	57
Warm-Up Patterns	57
Single Chain Mode Logic	64
Controller Chain Mode	64
IJTAG Network in EDT/LogicBist IP	70
Burn-In Test Mode	71

TAP Controller and LogicBIST

If you plan on using a TAP controller with the LogicBIST controller, then you must configure your TAP to account for certain mandatory signals.

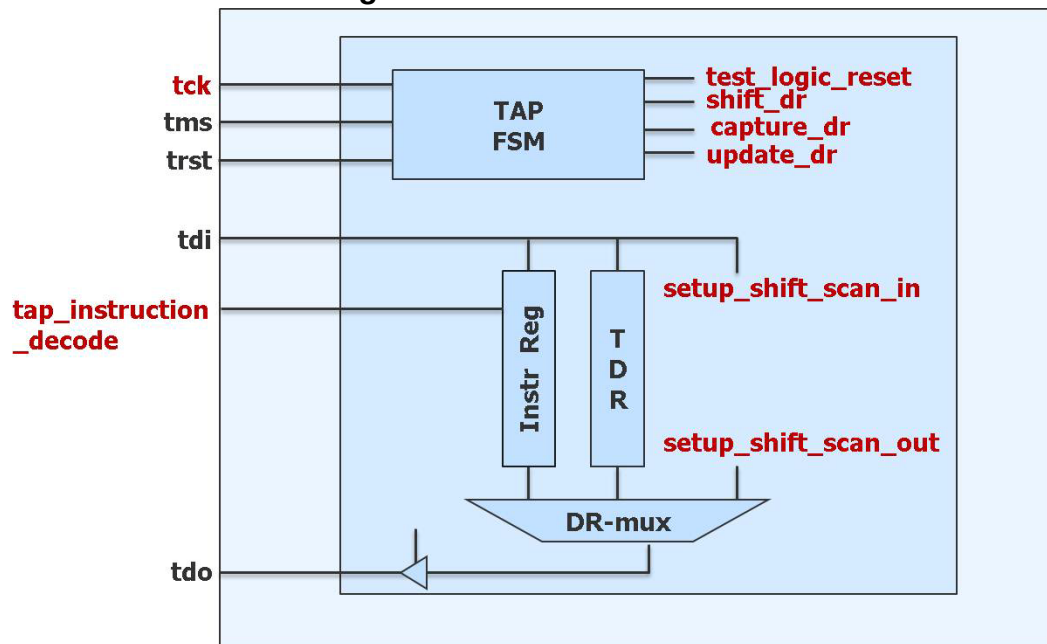
The mandatory signals are as follows:

- *tck*

- *setup_shift_scan_in*
- *setup_shift_scan_out*
- *capture_dr*
- *shift_dr*
- *update_dr*
- *test_logic_reset*
- *tap_instruction_decode*

Figure 4-2 shows the TAP controller with these signals highlighted in red.

Figure 4-2. TAP Controller



Note the following:

- During this step of the flow, you specify the LogicBIST pins for these TAP signals using the [set_lbist_pins](#) command before you create the hybrid LogicBIST controller.
- You must supply the ICL/PDL for the TAP—see “[Pattern Generation](#).”
- The tool automatically inserts a pipeline stage after the last SIB in the tool-produced ICL network for the LogicBIST controller to account for designs where the TAP already has a retiming flop on the *TDO* output pin. Consequently, you do not need to modify the ICL to account for the retiming flops at the output.

Clock Controller Connections to the EDT/LogicBIST IP

The clock controller interfaces only with *scan_enable* and *shift_clock* pins. You are responsible for creating and instantiating your clock controller in the input design. The clock controller should be pre-configured such that it is already usable for ATPG. The clock controller is expected to route *shift_clock* to its *clkout* pin when *scan_enable* is 1. It is expected to route the programmable internally-generated capture clock to its *clkout* pin when *scan_enable* is 0.

In this flow, the tool controls the *scan_enable/shift_clock* pins such that during ATPG mode the existing connections are used, but during LogicBIST mode they are driven by *~lbist_capture_en/lbist_shift_clock* respectively.

Figure 4-3 shows the Clock Controller before the EDT/LogicBIST IP has been generated.

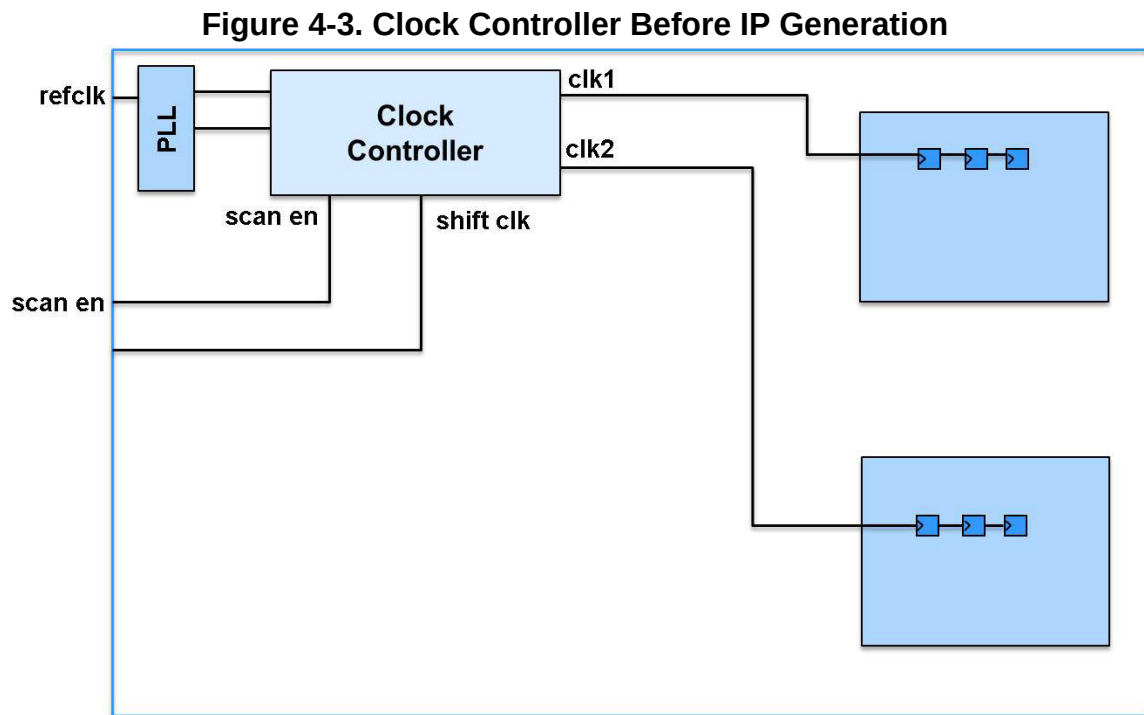
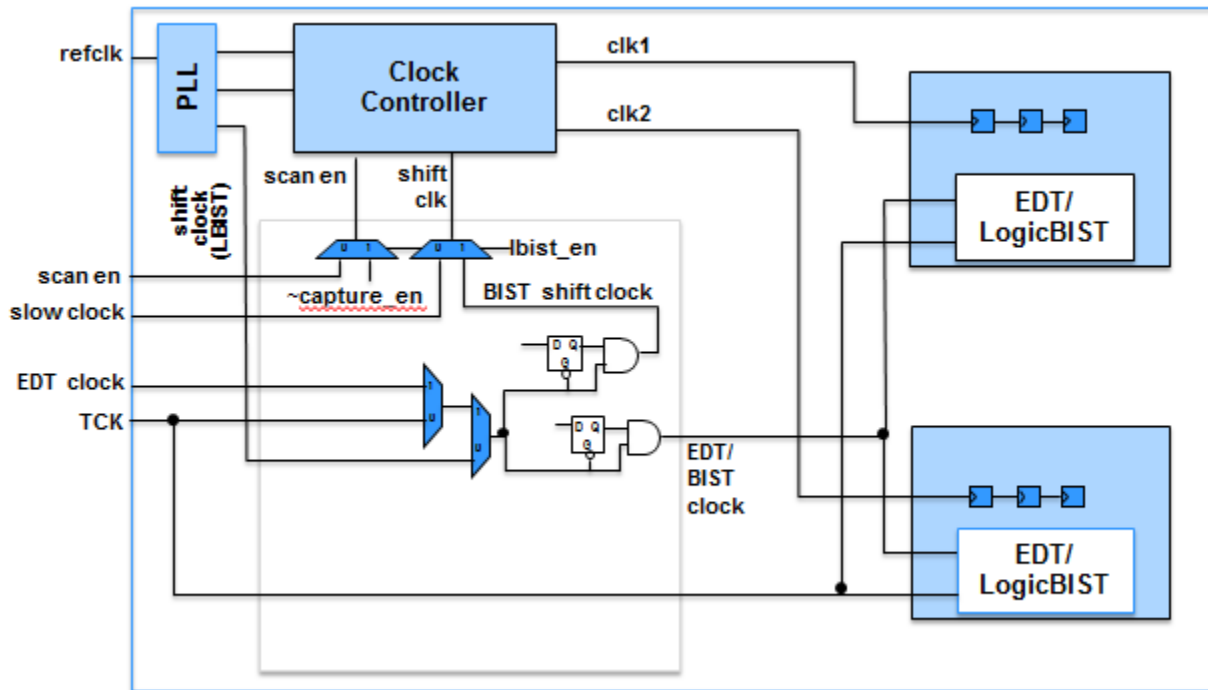


Figure 4-4 shows the clock controller connections to the BIST controller after EDT/LogicBIST IP generation.

Figure 4-4. Clock Controller After EDT/LogicBIST IP Generation



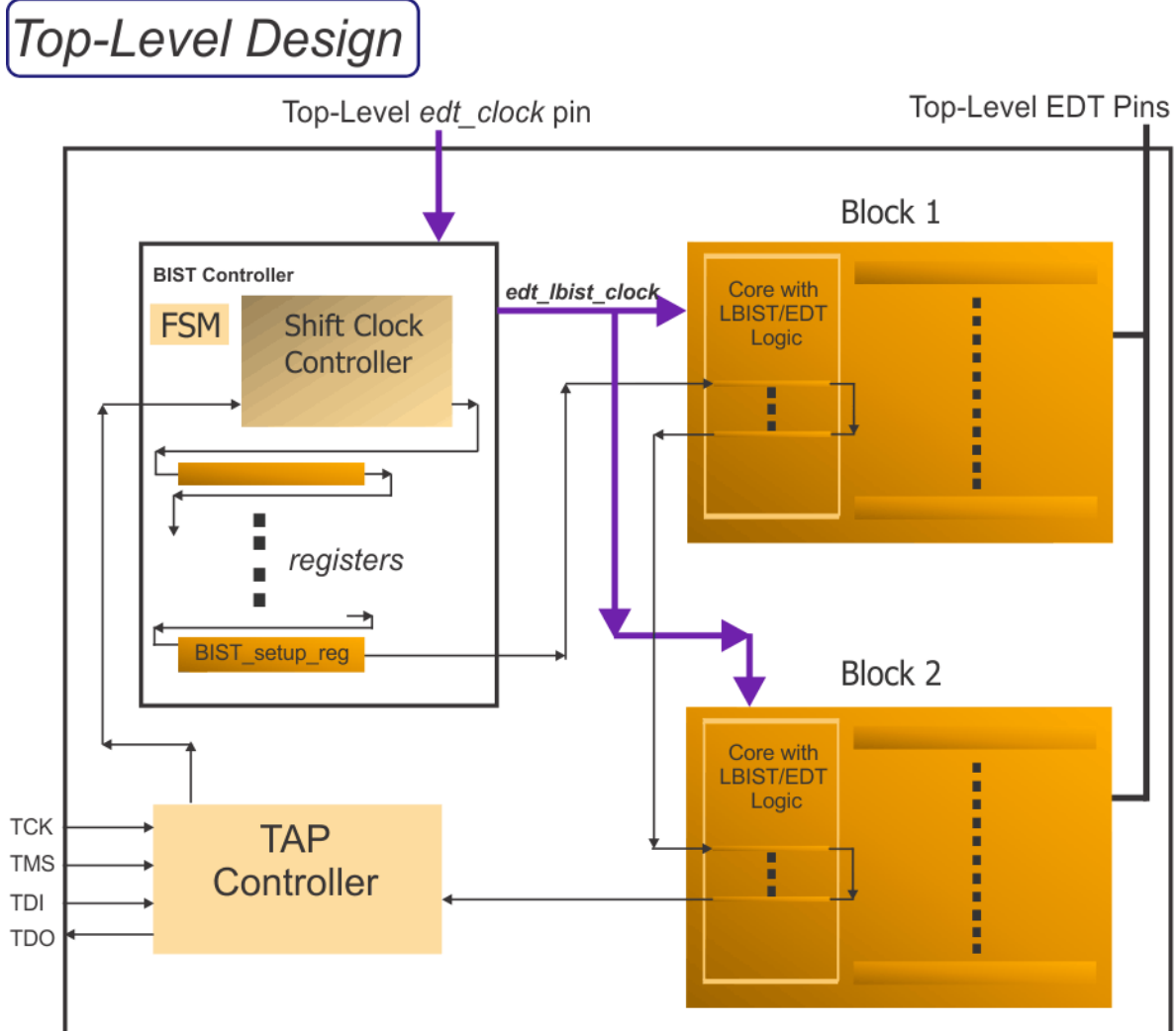
Alternatively, the clock controller could have *scan_enable/shift_clock_en* pins (instead of *shift_clock*). In this case, the tool muxes them with *~lbist_capture_en/lbist_shift_clock_en*. The clock controller is responsible for using the *lbist_shift_clock_en* as the clock-gating signal to internally generate the shift clock signal.

EDT and LogicBIST IP

An EDT block refers to an independent set of scan chains that are driven by a decompressor and observed by a compactor. The EDT block organization can be different from the block organization used for physical implementation. In particular, there can be multiple EDT blocks inside a single physical block.

Figure 4-5 illustrates a high-level block view of the EDT and LogicBIST IP.

Figure 4-5. Block-Level View of the EDT/LogicBIST IP



The EDT/LogicBIST hybrid IP reduces the area overhead compared to a separate implementation of EDT and LogicBIST IP. This is done by re-using parts of the IP for both EDT and LogicBIST modes. This hybrid logic controls input stimuli generation and output response comparison, and is implemented separately for each EDT block. The top-level controller including the LogicBIST FSM is then connected to these hybrid IP inserted EDT blocks.

For detailed description of Hybrid EDT/LogicBIST IP, refer to the following sections:

- “[Shared Logic](#)”
- “[Inserted Hybrid TK/LBIST IP](#)”
- “[Scan Chain Masking](#)”

- “New LogicBIST Control Signals”

Figure 4-6 shows the final top-level netlist after IP generation. During IP generation, the tool inserts muxes at the shift clock and scan enable clock controller pins and drive them from the LogicBIST controller.

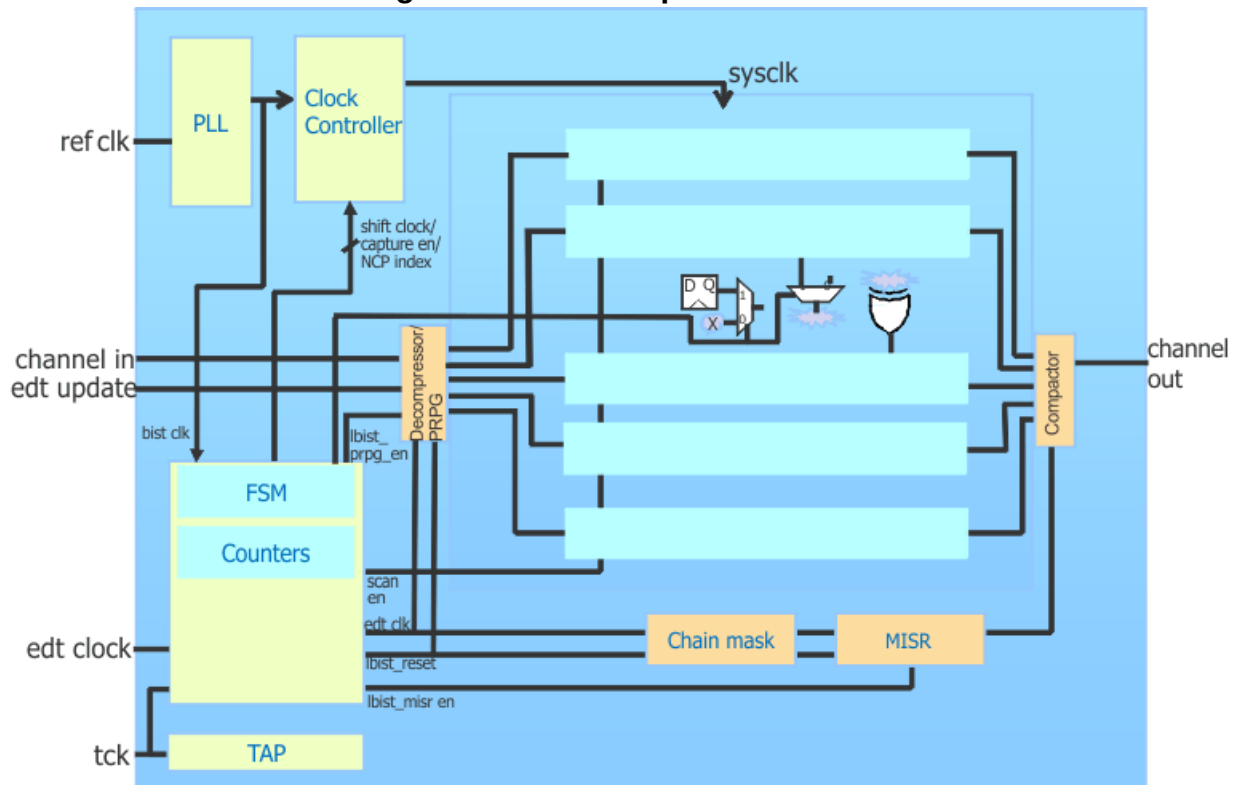
The clock controller’s *shift clock input* is driven by:

- The LogicBIST shift clock in LogicBIST mode,
- The pre-existing EDT clock connection during EDT mode.

The clock controller’s *scan enable input* is driven by:

- The inverted *lbist_capture_en* signal in LogicBIST mode,
- The pre-existing scan enable connection during EDT mode.

Figure 4-6. Final Top-Level Netlist



During EDT mode of operation, all EDT control signals except EDT clock, meaning, update, bypass, and low power enable are directly connected to the EDT logic without passing through the LogicBIST controller. The scan cells are shifted using a top-level shift clock. The Shift Clock controller chooses the EDT clock to be delivered to the EDT blocks.

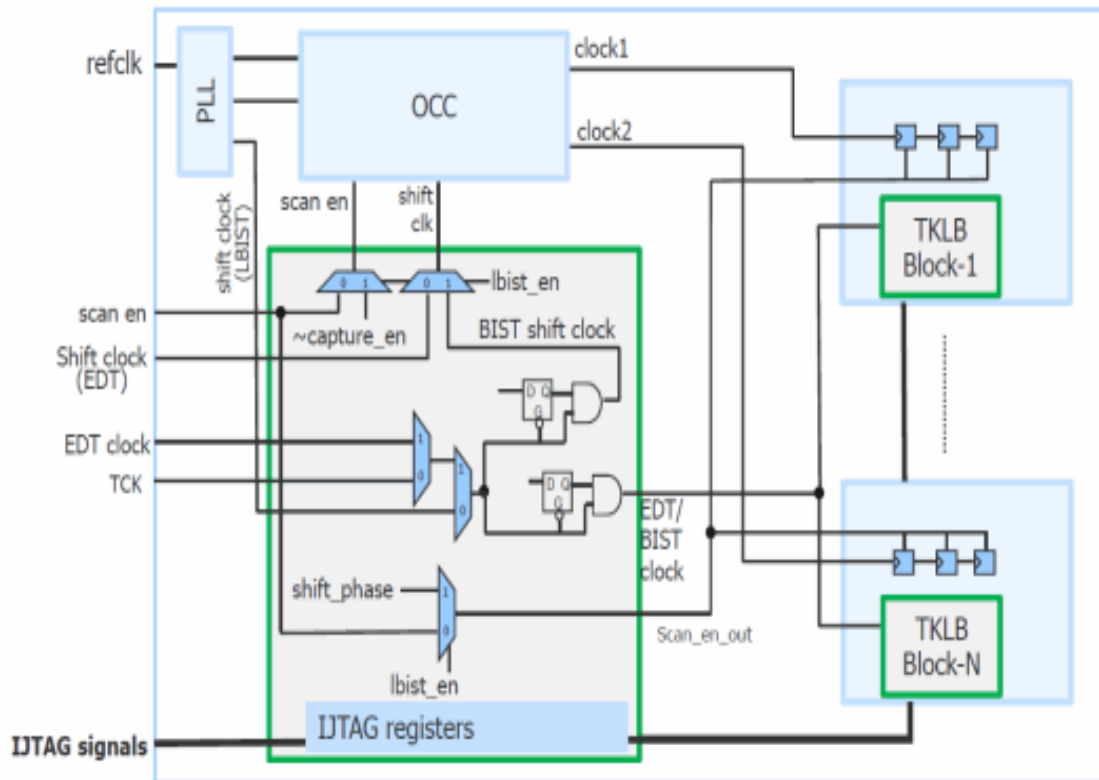
In addition, in EDT mode, the tool calls an automatically generated iProc procedure that initializes the hybrid IP.

During LogicBIST mode of operation, the scan cells are shifted using the shift clock output from the LogicBIST controller. The shift clock controller chooses a gated version of the LogicBIST clock to be delivered to the LogicBIST blocks. This gating ensures that the PRPG and MISR are not pulsed during capture as well as the transition between shift and capture modes.

The core scan cells receive the capture clock pulses from the clock controller logic. The LogicBIST clock input to the PRPG and MISR is turned off during capture.

Figure 4-7 shows the clock and EDT control signals in the LogicBIST IP.

Figure 4-7. Clocking During EDT Shift Mode of Operation



Low Pin Count Test Controller

The hybrid flow supports low pin count test (LPCT) type-1 and type-2 controllers but not type-3 LPCT controllers. The LPCT controller internally generates the EDT clock, and when operating in LBIST mode, the tool modified the controller to generate a free-running clock to allow using EDT clock for LBIST test. The LBIST controller's shift_clock_src and tck clock inputs are not impacted by using LPCT.

For hybrid-IP generation and insertion, you specify the scan enable and TAP controller pins using the set_lbist_pins command. Similarly, for LPCT controller generation and insertion, you

specify the scan enable and TAP controller pins when using type-2 LPCT using the `set_lpct_pins` command, as follows:

`set_lpct_pins scan_en`

When using LPCT with hybrid-IP, the `scan_enable` signal using the `set_lbist_pins` and `set_lpct_pins` are handled as follows:

- When only LPCT scan enable is specified, the value is also used for the LBIST controller.
- When only LBIST scan enable signals are specified, the first specified scan enable is used for the LPCT controller.
- When neither LPCT nor LBIST scan enable signals are specified, the tool issues an error message.
- For type-2 LPCT, the tool ignores the scan enable signals for the LBIST controller and instead uses the scan enable generated by the LPCT controller.

Similarly, the TAP pins for the LBIST and LPCT controllers are handled as follows:

- When either LBIST or LPCT pins only are specified for `tck`, `active-high` `test_logic_reset`, `shift_dr`, `capture_dr` and `update_dr` signals, they are also used for the other controller.
- When both LPCT and LBIST pins are specified, they are both used for connecting to the respective test logic IP.
- When neither LPCT nor LBIST pins are specified, default pin names are used for sharing between both controllers.
- The LBIST `tap_instruction_decode` signal is not shared with LPCT `test_mode` signal because these are controlled by separate TAP instructions or internal test logic.

The hybrid flow supports [Controller Chain Mode](#) (CCM) with both type-1 and type-2 LPCT controllers. You can use CCM with either `edt_clock` or `tck` with type-1 LPCT. The type-2 LPCT only supports the `edt_clock` option. This is because `tck` is used to operate the TAP controller to generate the scan enable, which will disturb the controller logic scan cells due to these extra `tck` pulses in pre-shift and post-shift cycles.

When using IJTAG with TAP controller, you must preserve the state of the IJTAG network using the following command:

`set_ijtag_retargeting_options -test_setup_network_end_state keep`

This requirement applies to both the hybrid IP and EDT-only type-2 LPCT controller. Since the network state is kept instead of reset, the BIST controller setup registers should be explicitly changed to the Idle state to enable EDT pattern generation. Refer to the following example for a pattern generation dofile.

Type-2 LPCT Controller Example

The following DftSpecification generates a TAP controller, excluding boundary scan cells for the primary IO pins. Three user TAP instructions are synthesized:

- An LPCT instruction to generate the LPCT test_mode signal and provide access to EDT channels using TDI/TDO.
- An LBIST instruction to provide the JTAG scan interface for the LBIST controller.
- A STATIC instruction that generates a TDR for controlling OCC and EDT static ports.

```
DftSpecification(m8051, tap) {
  use_rtl_cells: on;
  IjtagNetwork {
    HostScanInterface(dft) {
      Interface {
        tck: tck;
        tdo_en_polarity: active_high;
      }
      Tap(dft) {
        HostIjtag(lpct) {
        }
        HostIjtag(lbist) {
        }
        HostIjtag(static) {
          Tdr(static) {
            Interface {
              reset_polarity: active_high;
            }
            DataOutPorts {
              port_naming: test_mode, fast_capture_mode,
              capture_cycle_width[1:0], edt_bypass;
            }
          }
        }
      }
    }
  }
}

DftSpecification(m8051, occ) {
  use_rtl_cells: on;
  reuse_modules_when_possible: on;
  OCC {
    DefaultConnections {
      slow_clock: tck;
      StaticExternalControls {
        test_mode: m8051_tap_tessent_tdr_static_inst/test_mode;
        fast_capture_mode: m8051_tap_tessent_tdr_static_inst/
fast_capture_mode;
        capture_cycle_width:
          m8051_tap_tessent_tdr_static_inst/capture_cycle_width;
      }
    }
    capture_trigger: capture_en;
    static_clock_control: external;
    Controller(NX1) {
      clock_intercept_node: NX1g/Z;
    }
    Controller(NX2) {
      clock_intercept_node: NX2g/Z;
    }
  }
}
```

To configure the hybrid IP with a type-2 LPCT controller given the DftSpecification shown above, you would specify:

```
#
#EDT settings
set_edt_options -location internal -channel 1
set_edt_pins input 1 tdi
set_edt_pins output 1 tdo \
    m8051_tap_tessent_tap_dft_inst/host_lpct_from_so
set_edt_pins bypass - m8051_tap_tessent_tdr_static_inst/edt_bypass

#
#Type-2 LPCT settings
set_lpct_controller -generate_scan_enable on -tap_controller_interface on
set_lpct_pins clock tck
set_lpct_pins output_scan_en scan_en
set_lpct_pins reset - reset_inverter_dft/y
set_lpct_pins capture_dr - m8051_tap_tessent_tap_dft_inst/capture_dr_en
set_lpct_pins shift_dr - m8051_tap_tessent_tap_dft_inst/shift_dr_en
set_lpct_pins update_dr - m8051_tap_tessent_tap_dft_inst/update_dr_en
set_lpct_pins atpg_enable - \
    m8051_tap_tessent_tap_dft_inst/host_lpct_to_sel

#
#Tk/Lbist hybrid settings
set_lbist_controller_options -max_shift 100 -max_capture 7 \
    -max_pattern 100000 -capture_procedure 3
set_lbist_pins clock {- pll/pll_clock_0}
set_lbist_pins scan_en scan_en
set_lbist_pins tck tck
set_lbist_pins test_logic_reset {- reset_inverter_dft/y}
set_lbist_pins tap_instruction_decode {-
    m8051_tap_tessent_tap_dft_inst/host_lbist_to_sel}
set_lbist_pins shift_dr {- m8051_tap_tessent_tap_dft_inst/shift_dr_en}
set_lbist_pins update_dr {- m8051_tap_tessent_tap_dft_inst/
update_dr_en}set_lbist_pins capture_dr {-
    m8051_tap_tessent_tap_dft_inst/capture_dr_en}
set_lbist_pins setup_shift_scan_in tdi
set_lbist_pins setup_shift_scan_out {tdo
    m8051_tap_tessent_tap_dft_inst/host_lbist_from_so}
set_dft_enable_options -type xbounding -pin_name xbnd_en
set_clock_controller_pins capture_procedure_index -no_connection
```

The mapping flow for EDT pattern generation setup using TCD is shown below.

```
read_core_descriptions {created_m8051_edt.tcd created_m8051_lpct.tcd}
add_core_instances -module {*_tessent_occ *_edt *_lpct}

add_input_constraint RST -c0
add_input_constraint tms -c0
add_input_constraint trst -c1
add_clocks 0 tck -pulse_always          ;#reference clock to pll
add_clocks 0 pll/pll_clock_0 -pin_name fast_clock -pulse_always
set_procfile_name ../data/scan_setup.testproc

#
#Load LPCT instruction to allow EDT channels access to tdi/tdo
iProcsForModule m8051
iProc load_lpct_instruction {} {
    iWrite m8051_lbist_i.bist_setup Idle
    iWrite m8051_tap_tessent_tap_dft_inst.instruction HOSTIJTAG_LPCT
    iApply
}
set_test_setup_icall "load_lpct_instruction" -append
set_ijtag_retargeting_options -compare_constant_capture_values off \
    -test_setup_network_end_state keep
```

Clock Control Logic and Named Capture Procedures

In the EDT context, you must ensure the generation of proper shift and capture clocks. For LogicBIST mode, the controller generates the shift clock and capture enable signals, which are connected to the clock control logic. The scan enable output from the LogicBIST controller is de-asserted during capture.

Note

 NCPs are always required for LogicBIST operation. Other clock control techniques such as external clocks and clock_control definitions are not compatible with BIST application.

EDT Mode

All existing EDT capabilities for controlling capture mode clocking can be used in the shared EDT/LogicBIST architecture. For example, suppose you have a clock pin that is a design top-level pin controlled by the tester. When using a PLL, clock control register values for functional clocks can be shifted in during scan shift. You can control the number of capture cycles on a per pattern basis, either through a named capture procedure or clock_control definition.

LogicBIST Mode

You can describe specific capture clock sequences to be applied using NCPs using the following commands:

- [add_bist_capture_range](#)

- [set_clock_controller_pins](#), specifically, the `-capture_procedure_index` switch
- [set_lbist_controller_options](#), specifically, the `-capture_procedures` switch

In general, you associate the percentage of patterns for which each NCP is applied. This should reflect the number of faults that can be detected with the specified NCP. The number of cycles in capture should be uniform across all NCPs. You can specify the number of BIST clock cycles for capture. When not specified, the maximum value possible for the capture counter register in the hardware is used.

The LogicBIST controller consecutively applies the NCPs for the specified percentage of patterns, with the cycle repeating after 256 patterns. An additional output that identifies the index of the currently targeted NCP is available as a counter calculating the total number of NCPs. The NCP index is generated by decoding from the least significant bits of the pattern counter.

Again, you *must* ensure your design generates the correct clock sequence corresponding to the NCP based on this index signal bits.

If you define only a single NCP, then the NCP index output is not generated. You should do this in cases where the clock controller is initialized during `test_setup` to generate a particular capture procedure. All patterns in this session will use the same capture procedure.

LogicBIST Consideration When Using OCC for Fast Capture Mode

In the hybrid EDT/LBIST flow, you are responsible for ensuring that OCC generates capture clock pulses within the capture window (corresponding to the capture state of the LogicBIST FSM). When you configure Tessent OCC for fast capture mode, the OCC hardware requires one extra slow clock pulse to sample the `capture_en` trigger and two extra fast clock pulses for the synchronizer before the tool generates the capture pulses. When the fast clock and the slow clock have the same or similar frequencies, depending on clock balancing, it may take up to four pulses before the tool generates the capture pulses. This can result in clock pulses occurring outside the capture window, which can generate pattern mismatch errors.

To ensure that the tool accounts for the extra cycles, generate the LogicBIST capture width register with a bigger counter. For example, for an OCC with a four-bit shift register, configure the LogicBIST capture width counter to support up to eight cycles in the capture, as follows:

```
set_lbist_controller -max_capture_cycles 8
```

Likewise, during fault simulation, OCC is configured as having up to four pulses through NCPs, but you can load eight pulses into the capture width register by specifying the following command:

```
set_external_capture_options -fixed_cycles 8
```

Programmable Registers

Programming of the BIST registers is the same for both TAP and non-TAP cases. The BIST controller and EDT blocks use the same control interface.

When using a TAP, these signals are generated by the TAP controller. When *not* using a TAP, these signals are presented as top-level pins which will then be operated similar to the TAP controller output signals. This is done automatically in the next step of the flow—see “[Pattern Generation](#)” and “[Programmable Registers Inside Hybrid IP](#).”

Programmable Capture Procedures

The registers controlling the NCP bits are programmable, which means that you do not need to regenerate the IP if you want to change the percentage of patterns applied using a particular NCP after initial IP generation.

You can adjust the NCP activity level for fault simulation by issuing the [set_lbist_controller_options](#) command with the desired integer percentage values. The command allows you to provide a list of the NCP names with, optionally, the percentage values, or just the percentage values. Specifying only the percentage value is only allowed during IP creation.

When you know the percentages you want for the hardware default mode during IP creation, specify the list of percentages. Otherwise, specify only a count and the percentages will be equally distributed.

Note



If integer percentage values are not specified for IP generation, they must be specified for fault simulation or an error is issued.

The NCP names and order specified during IP generation are carried forward to fault simulation. The names specified during fault simulation are validated against those specified in IP generation. An error is generated if there is a mismatch between the two. If the NCPs specified during fault simulation are listed in a different order than those added during IP generation, the order is automatically changed to match the IP generation order. If an NCP specified during IP generation is not used during fault simulation, the NCP must be specified with “0” as the percentage.

To illustrate, suppose that four NCPs (clkseq1, clkseq2, clkseq3, and clkseq4) are declared without a percentage value in the IP generation dofile. When running LogicBIST fault

simulation, you decide that clkseq4 should be active 50% and the other three NCPs should be equally distributed for the remaining 50% of the time. Your dofile would have the following:

```
source ../<prefix>_lbist.dofile
...
set_lbist_controller_options -capture_procedures {clkseq 17 clkseq2 17
clkseq3 16 clkseq4 50}
```

The `set_lbist_controller_options` command will be internally converted to the following equivalent `add_bist_capture_range` commands:

```
add_bist_capture_range clkseq1 0 42 // 17%
add_bist_capture_range clkseq2 43 85 // 17%
add_bist_capture_range clkseq3 86 127 // 16%
add_bist_capture_range clkseq4 128 255 // 50%
```

These NCP counts are then stored in the Tessent Core Description (TCD) file for pattern generation.

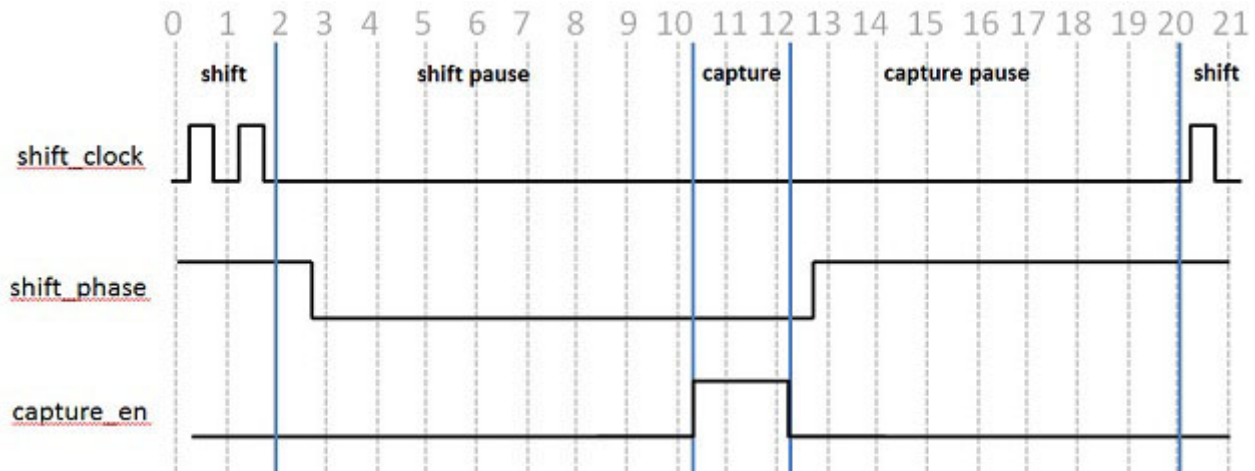
During pattern generation, the NCP counts stored in the TCD file are loaded into their respective ICL registers.

Programmable Shift and Capture Pause Cycles

During the LBIST controller operation, the tool applies a total of 16 dead cycles before and after the capture clocks are pulsed to help with the timing closure of the test logic. To reduce test overload due to the dead cycles, you can change the number of dead cycles by using the `set_lbist_controller_options` command.

The following figure shows the default cycle pause width for the dead cycles. The shift pause has 8 dead cycles from the last shift clock to the start of the capture, and the capture pause has 8 dead cycles from the end of the capture to the first clock edge in the shift. In addition, the scan enable (`shift_phase`) signal goes low after the last shift and goes high after the capture. This allows for maximum setup time transition.

Figure 4-8. Default Dead Cycle Pause Width



To change the number of dead cycles in the shift and capture pauses, specify the following command:

set_lbist_controller_options -pre_post_shift_dead_cycles *integer*

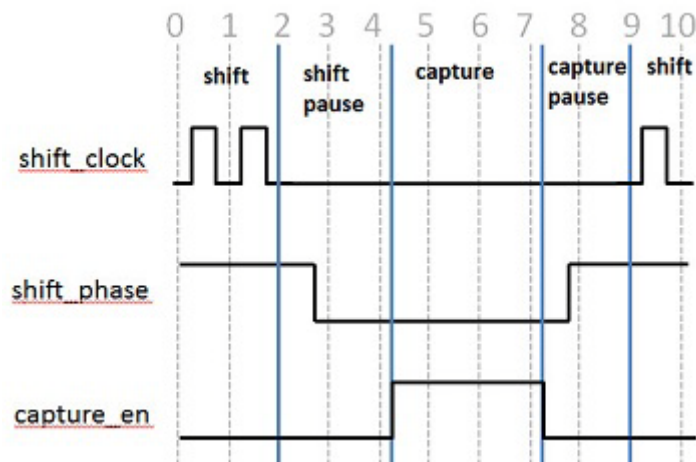
When using the DFT specification in the Tessent Shell flow, change the number of dead cycles by specifying the `pre_post_shift_dead_cycles` parameter, as shown below:

```
LogicBist {
  Controller(id) {
    parent_instance      : name ;
    leaf_instance_name   : name ;
    burn_in              : bool ; // default: off {symbols: on off}
    pre_post_shift_dead_cycles : int ; // default: 8
```

The option accepts integer values from 2 to 8, with 8 being the default. When specified, the hardware changes so that the requested number of dead cycles is used for both the shift and the capture pauses. A value of 1 is not allowed to prevent the capture enable and shift phase signals from toggling on the same clock edge.

The following figure shows the waveform when you have specified 2 for the dead cycle pause:

Figure 4-9. Dead Cycle Pause Width of 2



Low-Power Shift

During EDT/LogicBIST IP generation, you can configure the low-power scheme to control the switching activity during “shift” to reduce power consumption.

For a detailed description of the embedded structure inserted for this purpose, see “[Low-Power Shift Controller](#).”

The following Tessent Shell command sequence demonstrates creating the low-power shift controller for LogicBIST:

```
//Commands to configure EDT/LogicBIST blocks
add_edt_block B1
set_edt_options ...
set_lbist_controller_options ...
set_lbist_power_controller_options shift enabled -min_switching_threshold 10
```

Warm-Up Patterns

At the beginning of pattern generation, several patterns can elapse before the voltage in the chip stabilizes. This voltage droop can cause pattern mismatches. To remedy the impact of voltage droops, the tool inserts a warm-up pattern count register in the logic BIST controller. Warm-up patterns are those for which the scan chain unload values are not accumulated in the MISR, and for which no fault credit has been taken during fault simulation.

[Figure 4-12](#) shows the placement of the warm-up pattern count register within the logic BIST controller. By default, the tool creates a 10-bit register to allow up to 1024 warm-up patterns. A value of 0 loaded into the register indicates that you are not applying any warm-up patterns.

The logic BIST controller operates as follows: The first pattern that the tool uses for fault detection during fault simulation corresponds with `begin_pattern=0`. The PRPG value for this pattern is determined by the initial hardware PRPG seed, followed by applying the default number of warm-up patterns. When a non-default number of warm-up patterns is applied, the PRPG must be loaded such that it still reaches the same initial PRPG value for this first pattern. If `begin_pattern` is not zero, then the PRPG must be loaded such that it reaches the correct value that was specified for that pattern during fault simulation. When `begin_pattern` is zero, the tool can simply load the MISR with the value for that pattern (since the MISR does not update during warm-up patterns). However, when `begin_pattern` is non-zero, the tool must load the MISR with the value that was computed during fault simulation for that pattern.

The tool masks the scan chain unload values for warm-up patterns to allow the voltage in the chip to stabilize. You can specify the number of patterns to mask by setting the `warmup_pattern_count` argument when executing the `run_lbist_normal` or `scan_unload_register` iCall. If this argument is not provided with the iCall command, the iProc procedure defaults it to 0.

Refer to the [set_lbist_controller_options](#) -max_warmup_pattern_count option for information about how to change the maximum number of warm-up patterns, specify the number of hardware default warm-up patterns, or suppress warm-up pattern count register insertion.

Pattern Counts When Using Warm-Up Patterns

During fault simulation, the tool computes the PRPG value at the start of each warm-up pattern (without fault crediting and MISR accumulation) for the maximum number of warm-up patterns. Fault simulation is then performed for the number of patterns specified by `set_random_patterns` with fault crediting and MISR signature calculation.

Logic BIST patterns are typically applied starting with pattern 0. This is the default for the hardware, and is also the default for the `run_lbist_normal` ICL procedure. The most common reason for specifying a non-zero value for `begin_pattern` is during debug or diagnostic when you want to quickly apply a small range of patterns, or just a single failing pattern.

In the following example, the user wants to run LBIST starting from a specific pattern. The pattern specification instructs the logic BIST controller to run from patterns 100 to 199 with 16 warm-up patterns. Since there are 16 warm-up patterns, the tool seeds the PRPG with a seed corresponding to pattern 84, which is equal to:

`begin_pattern - warmup_pattern_count`

In this case, the controller executes 116 patterns but the MISR only observes the last 100 patterns.

```
PatternsSpecification(coreA,gate,signoff)
{
  Patterns(LogicBist_coreA){
    ClockPeriods {
      refclk : 10.00ns;
    }
    TestStep(serial_load){
      LogicBist{
        CoreInstance(.){
          run_mode : run_time_prog;
          begin_pattern : 100;
          end_pattern : 199;
          warmup_pattern_count : 16;
        }
      }
    }
  }
}
```

In the following example, the PRPG initial seed originates from the warm-up PRPG seed-only patterns. Beginning from pattern 0, the tool seeds the MISR with d'0, and the PRPG is based on the number of specified warm-up patterns. Suppose your pattern specification states:

```
PatternsSpecification(coreA,gate,signoff)
{
  Patterns(LogicBist_coreA){
    ClockPeriods {
      refclk : 10.00ns;
    }
    TestStep(serial_load){
      LogicBist{
        CoreInstance(.){
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 99;
          warmup_pattern_count : 16;
        }
      }
    }
  }
}
```

Assuming a maximum of 255 warm-up patterns, the PRPG seed starts with the 240th warm-up pattern.

With respect to warm-up patterns and the PRPG seeding, scan unload pattern operations function the same as described above for logic BIST patterns.

For the hardware default mode, the tool performs the warm-up period within the hardware so no additional JTAG access is required. The number of warm-up patterns, PRPG seed, and low-power register seeds are hard-coded in the iProc, and these values are checked against the PatternDB file to ensure they match. If the iProc values do not match the PatternDB, the tool generates an error and the hardware default patterns cannot be saved.

Fault Simulation

The fault simulation dofile generated during IP creation carries forward the size of the warm-up pattern count register, as shown in the example below, so that Tessent Shell can create the maximum number of patterns.

```
set_lbist_controller_options -version 1 -ncapture_cycles_reg_width 2  
-pattern_count_reg_width 10 -warmup_pattern_count_reg_width 6  
-ncp_index_reg_width 0 -programmable_capture_procedures off
```

You can use the `set_edt_options -decompressor_seed` command to change the initial seed of the PRPG. This seed applies to both the warm-up patterns and the actual patterns. If you do not specify a PRPG seed, the tool uses the value from IP generation. Using the PRPG seed, the tool calculates the PRPG seed for the first regular pattern, which it then stores in the Tessent Core Description file and flat model.

If you use a different decompressor seed than that used during IP generation, then the HW default mode cannot be used with the current pattern set.

The tool does not include warm-up patterns in the number of logic BIST patterns it simulates, which means these patterns do not contribute to fault coverage analysis. For example, if you specify `set_random_patterns 32000` and you have a maximum of 255 warm-up patterns, then the tool creates 255 warm-up patterns followed by 32000 logic BIST patterns.

When creating parallel patterns with the `write_patterns -parallel` command, the tool excludes the warm-up patterns because the parallel patterns are primarily used for validating logic BIST operation. However, the PatternDB file includes the warm-up patterns so that the tool can apply them during pattern retargeting.

The following Tessent Core Description file example shows a warm-up pattern register capable of 255 patterns.

```
Core(gpu) {  
  LbistMode(lbist) {  
    Registers {  
      PrpgRegister(gpu_edt_i.lfsm_vec) {  
        length : 31;  
        type : prpg;  
      }  
      WarmupPatternRegister(gpu_edt_i.warmup_pattern_count) {  
        length : 8;  
      }  
    }  
  }  
}
```

Example

In the following example, the logic BIST controller needs to support up to 300 warm-up patterns with 128 hardware default warm-up patterns. During IP generation, you would use the following command:

```
set_lbist_controller_options -max_warmup_pattern_count {300 128}
```

The Logic BIST controller generates a warm-up pattern counter register with 9 bits, allowing up to a maximum of 511 warm-up patterns. Using the following command, the tool forwards the register width to fault simulation through the Logic BIST dofile:

```
set_lbist_controller_options ... -warmup_pattern_count_reg_width 9
```

For the TSDB flow, the following example illustrates how you can apply the warm-up patterns for various scenarios.

```
PatternsSpecification(coreA,gate,signoff) {
  // Run the first 1K patterns with 300 warm-up patterns
  Patterns(warmup_300) {
    ...
    TestStep(serial_load) {
      LogicBist {
        CoreInstance(w2_A) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 1023;
          warmup_pattern_count : 300;
        }
      }
    }
  }
  // Unload the 274th pattern after applying 300 warm-up patterns
  Patterns(diag_warmup_300) {
    ...
    TestStep(diagnosis) {
      LogicBist {
        CoreInstance(w2_A) {
          run_mode : run_time_prog;
          begin_pattern : 274;
          end_pattern : 274;
          warmup_pattern_count : 300;
          DiagnosisOptions {
            extract_flop_data : on;
          }
        }
      }
    }
  }
  // Run the first 1K patterns with 16 warm-up patterns
  Patterns(warmup_16) {
    ...
    TestStep(serial_load) {
      LogicBist {
        CoreInstance(w2_A) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 1023;
          warmup_pattern_count : 16;
        }
      }
    }
  }
  // Run the HW default pattern, which contains 128 warm-up patterns
  Patterns(hw_default) {
    ...
    TestStep(hw_default) {
      LogicBist {
        CoreInstance(w2_A) {
          run_mode : hw_default;
        }
      }
    }
  }
  // Run two controllers in parallel with different
  // warm-up pattern ranges and number of warm-ups
}
```

```
Patterns(parallel_warmup) {  
  ...  
  TestStep(serial_load) {  
    LogicBist {      CoreInstance(w2_A) {  
      run_mode : run_time_prog;  
      begin_pattern : 0;  
      end_pattern : 1023;  
      warmup_pattern_count : 64;  
    }  
    CoreInstance(w2_B) {  
      run_mode : run_time_prog;  
      begin_pattern : 1024;  
      end_pattern : 2047;  
      warmup_pattern_count : 128;  
    }  
  }  
}  
}
```

For the dofile flow, the following pattern retargeting commands illustrate how you can call iProcs.

```
# Run the first 1K patterns with 300 warm-up patterns  
open_pattern_set warmup_300  
  iCall w2_A.run_lbist_normal lbist_clock 0 1023 lbist 300  
close_pattern_set  
write_patterns ...  
  
# Unload the 274th pattern after applying 300 warm-up patterns  
open_pattern_set diag_warmup_300  
  iCall w2_A.scan_unload_register lbist_clock 274 274 lbist 300  
close_pattern_set  
write_patterns ...  
  
# Run the first 1K patterns with 16 warm-up patterns  
open_pattern_set warmup_16  
  iCall w2_A.run_lbist_normal lbist_clock 0 1023 lbist 16  
close_pattern_set  
write_patterns ...  
  
# Run the HW default pattern, which contains 128 warm-up patterns  
open_pattern_set hw_default  
  iCall w2_A.run_lbist_hw_default lbist  
close_pattern_set  
write_patterns ...  
  
# Run two controllers in parallel with different warm-up pattern ranges  
# and number of warm-ups  
open_pattern_set warmup_16  
  iCall w2_A.run_lbist_normal lbist_clock 0 1023 lbist 64  
  iCall w2_B.run_lbist_normal lbist_clock 1024 2047 lbist 128  
close_pattern_set  
write_patterns ...
```

Single Chain Mode Logic

In the hybrid EDT/LBIST flow, LBIST pattern diagnosis is performed by loading values by the PRPG and shifting out the captured scan cell values. To allow this shift out, the tool inserts single chain mode logic that concatenates all the short scan chains across all the blocks. This logic is built by concatenating the EDT single bypass chain across all the EDT blocks.

By default, the tool inserts the logic for single chain mode during IP generation.

The single chain mode logic disables EDT bypass and single bypass pins to ensure they do not interfere with LBIST operation. During diagnosis, a TDR control bit within the logic provides access to the single chain mode for scan chain unload.

Note

 When you disable either EDT bypass or EDT single bypass in any EDT block, the tool does not generate the single chain mode logic. You can turn off bypass and single bypass by using the `set_edt_options -bypass off` or `set_edt_options -single_bypass off` commands, respectively.

Caution

 Turning these switches off is not recommended because doing so also turns off LBIST pattern diagnosis.

Controller Chain Mode

For some safety critical applications, you must test the test logic itself—that is, the LogicBIST and EDT IP blocks—in addition to testing the core design logic. Controller chain mode (CCM) allows you to generate ATPG patterns that target the hybrid EDT/LBIST blocks and LBIST controller in addition to the single chain mode logic and In-System Test controller.

You can generate CCM patterns within both the TSDB and dofile flows. However, the process differs as described in this section.

Using tck as the Clock Source Rather Than Default test_clock

The hybrid EDT/LBIST controller has a 3-to-1 clock mux that allows you to choose between `shift_clock_src`, `test_clock` and `tck` as the runtime clock for LogicBIST test. By default, during scan insertion and fault simulation, this clock mux is configured to use `test_clock`.

When implementing CCM, you can use either `test_clock` or `tck` as the clock. You must choose the clock at the time of IP generation. Select the `tck` clock as follows for the dofile flow or TSDB flow, respectively:

```
set_lbist_controller_options -controller_chain_clock tck

DftSpecification/LogicBist/Controller/clock : tck ;
```


When you implement CCM to use `tck`, perform scan insertion for the CCM mode chains in a separate run from regular scan cell insertion. When inserting the CCM mode chains, configure the LogicBIST clock mux to choose `tck` as the clock by setting the `controller_chain_mode` DFT signal to 1. This avoids the situation in which the source of the `edt_lbist_clock` at the LogicBIST controller output differs between scan insertion and pattern generation for CCM mode.

When you implement CCM with the default, `test_clock`, the scan insertion view remains consistent with the CCM pattern generation view, so a separate CCM mode scan insertion run is not required.

TSDB Flow Usage Details for Controller Chain Mode

To enable generation of CCM logic, specify the following property within the LogicBist wrapper:

```
DftSpecification/LogicBist/Controller/ControllerChain/present: on ;
```

You can modify the names of CCM-specific ports using the `DftSpecification/LogicBist/Controller/Interface` wrapper.

As described in “[RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST](#)” in the *Tessent Shell User’s Manual*, in the TSDB flow (also known as the Tessent Shell flow), you insert the hybrid IP before performing scan chain insertion. By default, the tool inserts CCM scan segments for each instrument but does not connect them into one chain. This allows for flexible scan chain stitching during scan insertion.

During scan insertion, configure the controller chains by creating a controller chain scan mode with the `add_scan_mode` command. This scan mode should only contain the CCM scan segments as valid scan elements.

The CCM scan segments in Tessent IP cores are not active by default in order to not be confused with standard scan elements (design flops, segments on sub_blocks, and so on). You must activate a `controller_chain_mode` scan mode on Tessent IP instances before adding them to the scan mode population. It is also recommended to reset the active child scan mode after adding CCM scan mode. These tasks can be achieved using the following commands:

```
set_attribute_value [get_ccm_tested_instances] \
    -name active_child_scan_mode \
    -value controller_chain_mode
add_scan_mode controller_chain_mode \
    -include_elements [get_scan_elements \
        -of_child_scan_mode controller_chain_mode]
reset_attribute_value [get_ccm_tested_instances] \
    -name active_child_scan_mode
```

Note



Recommendation: To benefit from flow automation, use the `controller_chain_mode` DFT signal as a scan mode enable signal.

Example 4-1. Dofile Example for Scan Chain Insertion with CCM (TSDB Flow)

```
# Load design data and set the design and X-bounding parameters

set_system_mode analysis

# X-bounding and wrapper analysis and insertion

# add modes to test design logic
analyze_wrapper_cells
add_scan_mode int_mode -type internal
add_scan_mode ext_mode -type external

# Utility procedure used to setup controller_chain_mode
# Can be implemented by user to suit specific needs
proc get_ccm_tested_instances {} {
    set scan_elements [get_scan_elements \
                        -of_child_scan_mode controller_chain_mode \
                        -state any]
    set modules [get_attribute_value_list $scan_elements \
                -name module_name]
    return [get_instances -of_module $modules]
}

set_attribute_value [get_ccm_tested_instances] \
    -name active_child_scan_mode \
    -value controller_chain_mode

add_scan_mode controller_chain_mode \
    -include_elements [get_scan_elements \
                      -of_child_scan_mode controller_chain_mode]

reset_attribute_value [get_ccm_tested_instances] \
    -name active_child_scan_mode

analyze_scan_chains
insert_test_logic
```

For details about scan chain insertion when you are using the TSDB flow, refer to “[Perform Scan Chain Insertion](#)” in the *Tessent Shell User’s Manual*.

For information about generating CCM patterns in the TSDB flow, refer to “[Performing Pattern Generation for CCM in the TSDB Flow](#)” on page 109.

To connect the CCM scan segments into one controller scan chain during IP generation, specify:

```
DftSpecification/LogicBist/Controller/ControllerChain \
    segment_per_instrument: off ;
```

Dofile Flow Usage Details for Controller Chain Mode

To enable generation of CCM logic, specify the following command:

```
set_lbist_controller_options -controller_chain_mode on
```

By default, the tool inserts CCM scan segments for each instrument but does not connect them into one chain. In the dofile flow, you perform scan chain insertion prior to inserting the IP, which means that if you want to configure the controller chains, you must perform an incremental scan insertion to create the CMM scan chain from the segments.

To automatically generate the EDT/LBIST IP with a single RTL scan chain assembled from the CCM scan segments in each IP block, specify:

```
set_lbist_controller_options -segment_per_instrument off
```

You can use the `set_lbist_pins` command to change the port names or share them with existing ports. For example:

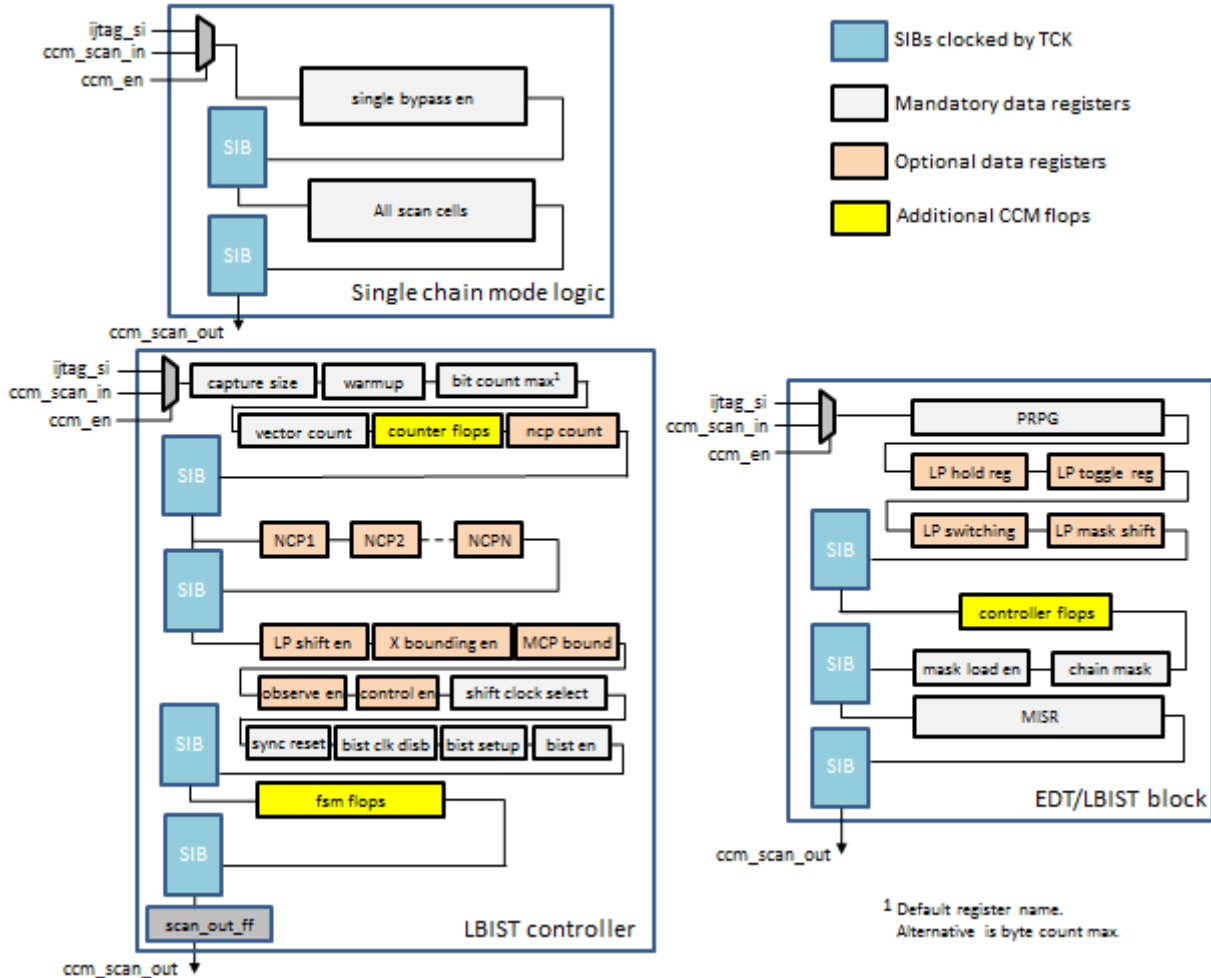
```
set_lbist_pins controller_chain_en {top_pin_name} [internal_pin_name]
set_lbist_pins controller_chain_scan_in {top_pin_name}
[internal_pin_name]
set_lbist_pins controller_chain_scan_out {top_pin_name}
[internal_pin_name]
```

IP generation results in two files, the `created_ccm.dofile` and the `created_ccm.testproc`, which are used as inputs for pattern generation. For information about generating CCM patterns in the dofile flow, refer to “[Performing Pattern Generation for CCM in the Dofile Flow](#)” on page 117 for details.

Controller Chain Mode Architecture

The default inserted design architecture is:

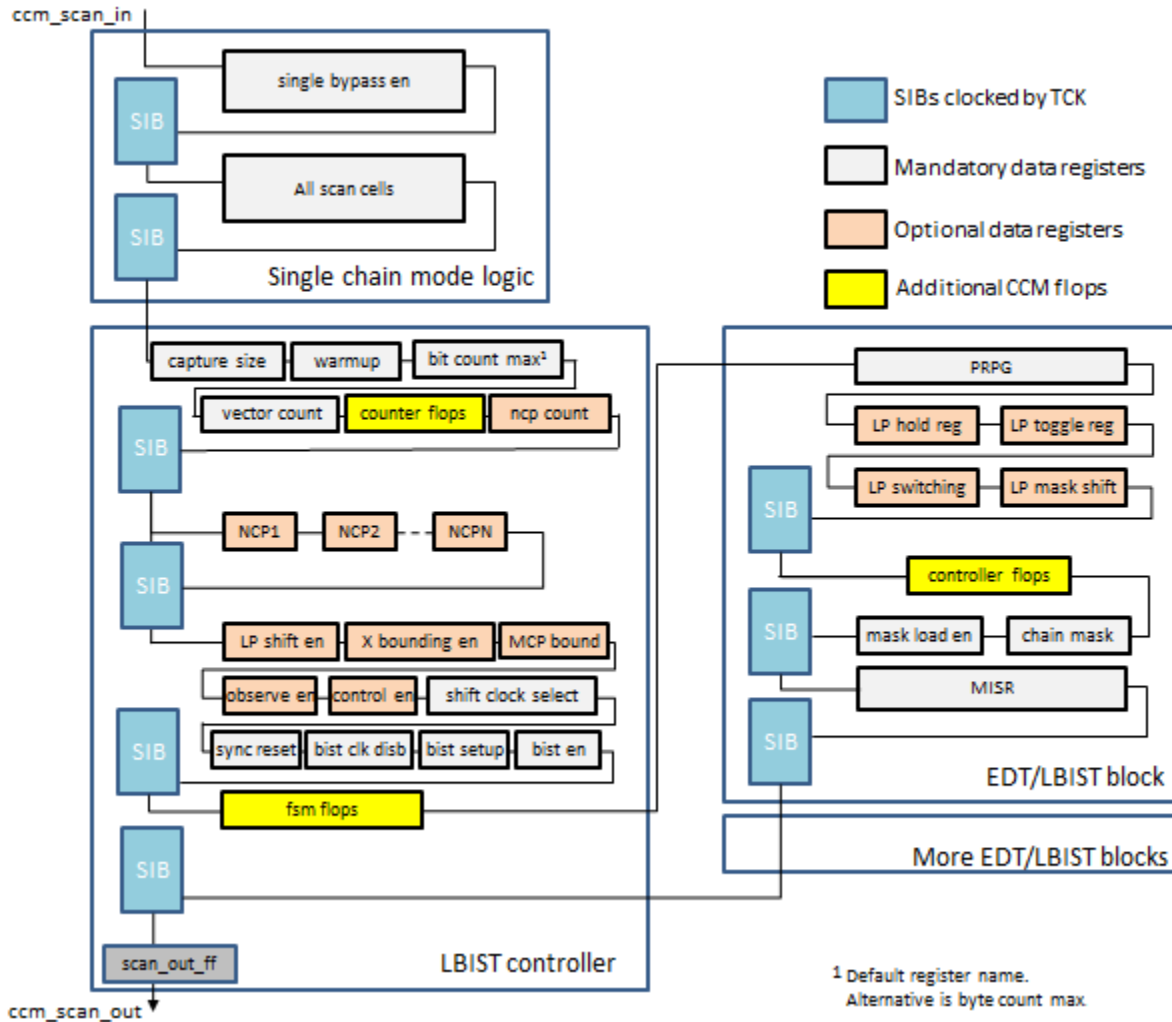
Figure 4-10. EDT/LogicBIST-Inserted Design for CCM, Segmented Scan Chains



The tool inserts CCM scan in and CCM scan out ports, and muxes controlled by the CCM enable signal, into each IP instrument that contains CCM scan segments. After IP insertion, the ccm_scan_in and ccm_en ports are tied to 0, and the ccm_scan_out is unconnected. The tool internally generates the ccm_scan_en signal within in each IP block by ANDing the ccm_en and scan_en signals. The input port (ccm_en) enables the CCM. The tool reuses the existing ATPG scan enable for CCM.

When you specify to turn off the default segmentation so that the tool automatically connects the scan segments into one controller scan chain, the inserted architecture design is:

Figure 4-11. EDT/LogicBIST-Inserted Design for CCM, Non-Segmented Scan Chains



CCM uses the existing ICL network chain that already contains most of the flops in the controllers. Flops that are not in the ICL network are multiplexed into it during the CCM shift. The result is a single chain that is accessible through a new scan input (ccm_scan_in) and output (ccm_scan_out).

Related Topics

[Performing Pattern Generation for CCM in the TSDB Flow](#)

[Performing Pattern Generation for the Dofile Flow](#)

IJTAG Network in EDT/LogicBist IP

SIBs are a mechanism to provide flexible access to data registers they control. For the Mentor Graphics version of a SIB, the data registers are accessible via the IJTAG network when the SIB controlling a data register is set to 1, and the data register is bypassed when the SIB is set to 0.

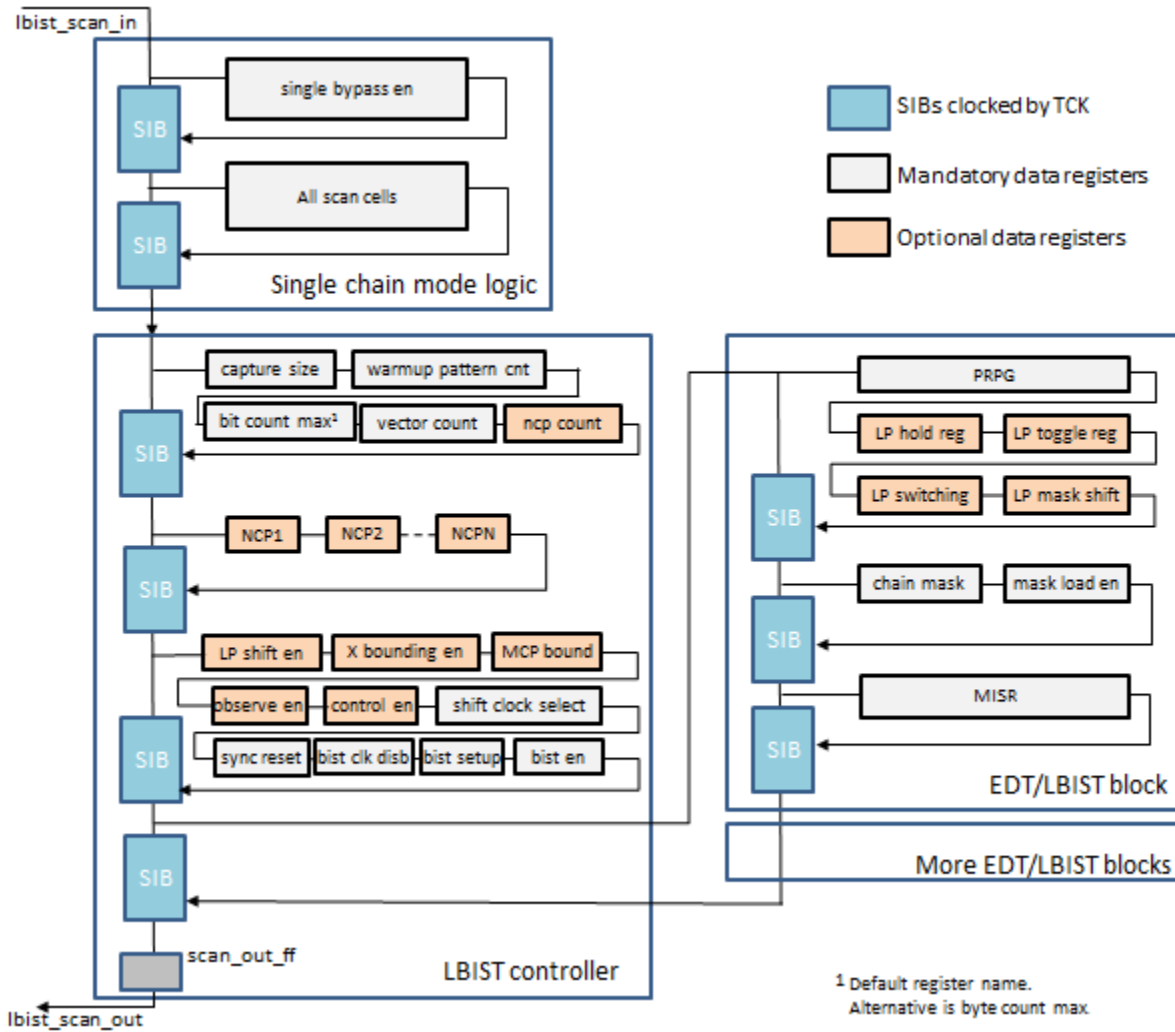
SIBs are clocked by TCK. To provide flexibility in accessing specified registers in the IJAG network at different times, the SIBs are arranged in a hierarchical tree structure. The top-level SIBs use the `lbist_tap_instruction` signal from the JTAG LogicBIST instruction as the SIB enable signal. The SIB enable signals for the child SIBs are controlled by their parents. This is how all the SIBs inside the LogicBIST controller, EDT/LogicBIST blocks and single chain mode logic are configured.

Each EDT/LogicBIST block contains four SIB registers. They provide access to the PRPG, EDT chain mask register, MISR, and programmable NCP count registers, respectively. The EDT SIBs are clocked by `tck`, and the data registers they control are clocked by `edt_clock`. The tool adds a lockup cell to avoid clock skew between these two clock domains. A SIB inside the LogicBIST controller controls access to the EDT SIBs. The enable output of this LogicBIST controller SIB is used as the input enable for the EDT SIBs.

For each specified NCP, an 8-bit register is created and inserted on the ICL network. [Figure 4-12](#) shows where the NCP count register is inserted. These registers are loaded at runtime so that the number of patterns applied for NCP is programmable. When an integer percentage is provided during IP generation, the NCP registers will reset to those values when `sib_reset` is asserted. Otherwise, these registers are reset to equal percentages across all NCPs. For more information, see the “[Programmable Capture Procedures](#)” section in the “[EDT and LogicBIST IP Generation](#)” chapter.

[Figure 4-12](#) shows the IJTAG network in a hybrid EDT/LBIST-inserted design.

Figure 4-12. SIBs Insertion and Integration of Cores



Burn-In Test Mode

The hybrid TK/LBIST flow supports wafer-level burn-in tests for use during board-level testing. When you enable burn-in test mode, the tool creates additional RTL to allow the LogicBIST IP to run for longer periods of time than is normally allowed by the size of the LogicBIST pattern counter.

When using the dofile flow for pattern generation, the retargeting dofile template will include a section to create a LogicBIST burn-in pattern. To enable burn-in test mode, specify the following command:

```
set_lbist_controller_options -burn_in on
```

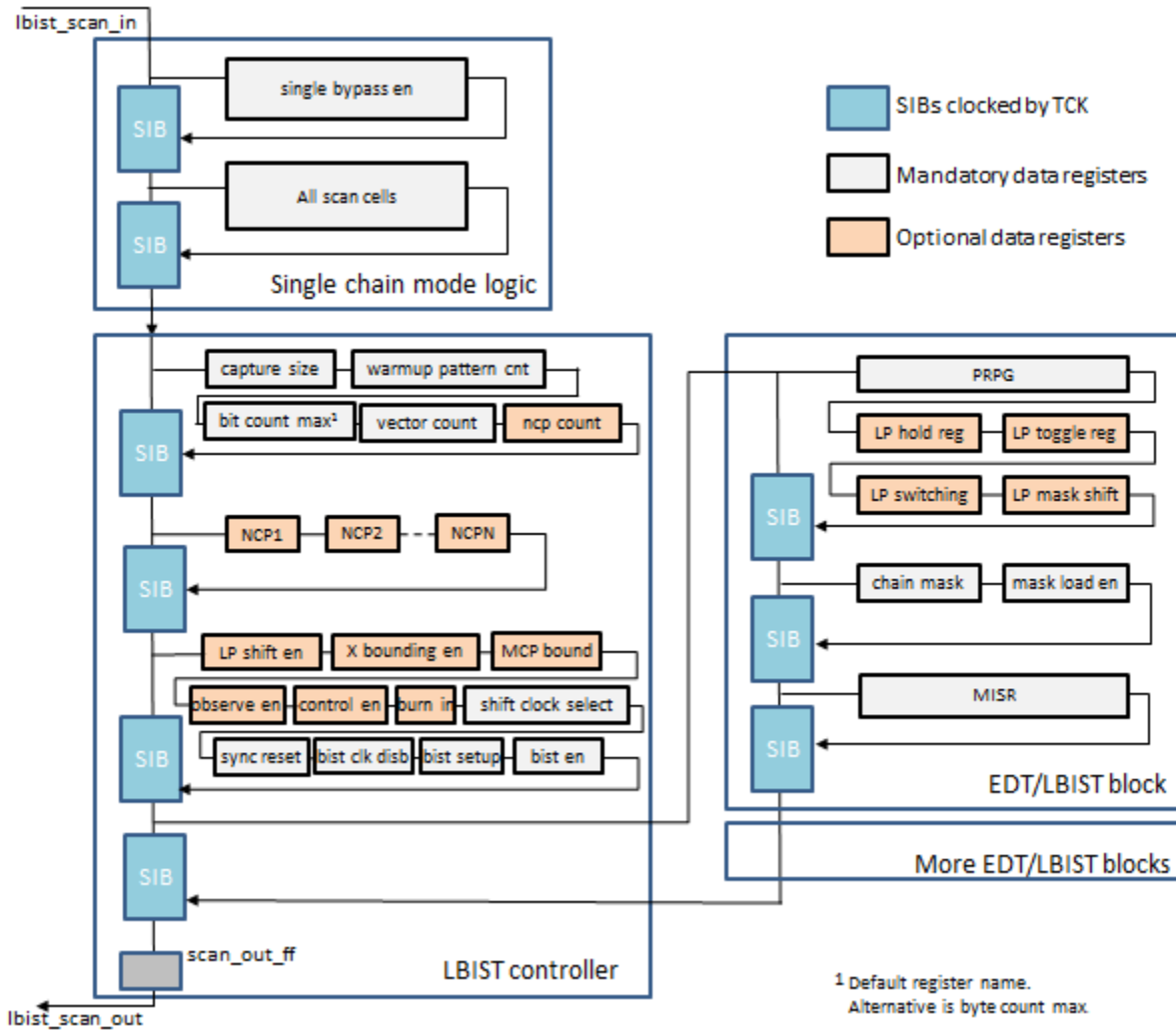
When using the DFT specification in the Tessent Shell flow, enable burn-in mode by specifying the `burn_in` property, as shown below:

```
LogicBist {  
  Controller(id) {  
    parent_instance           : name ;  
    leaf_instance_name       : name ;  
    burn_in                   : bool ; // default: off {symbols: on off}  
    pre_post_shift_dead_cycles : int ; // default: 8
```

When using the pattern specification in the TSDB flow, enable burn-in mode by specifying the `burn_in_time` property, as shown below:

```
{  
  CoreInstance(icl_instance_name) {  
    run_mode           : burn_in ;  
    burn_in_time       : tvalue ; // a time value in seconds
```

In burn-in mode, the hardware prevents the LogicBIST controller from reaching an end state in its FSM, which allows the PRPG to run continuously for the specified amount of time. Because the burn-in runtime can easily exceed the pattern counter, no MISR comparisons are performed when the test completes.

Figure 4-13. LogicBIST Controller with Burn-In Bit

Burn-In Only Fault Simulation

For fault simulation, you can create multiple burn-in configurations, such as different power configurations or decompressor seeds, by running a reduced fault simulation for the purposes of just creating the data needed to initialize/seed the LogicBIST registers.

To (re-)run fault simulation to generate a dedicated burn-in mode pattern set, do the following:

1. Change the current mode (set_current_mode).
2. Reduce the number of patterns to simulate (set_random_patterns).
3. Make any other changes as needed, such as changing the power configuration or using a different decompressor seed.

Generating the EDT and LogicBIST IP

You can generate the EDT and LogicBIST IP by using the dofile flow or the TSDB flow.

Generating the EDT and LogicBIST IP (TSDB Flow)	74
Generating the EDT and LogicBIST IP (Dofile Flow)	76

Generating the EDT and LogicBIST IP (TSDB Flow)

To generate the hybrid EDT/LBIST IP, you must create a LogicBIST wrapper, which specifies the information used by the `process_dft_specification` command to build the hybrid IP and insert the IP into the design.

Once you have nested the LogicBIST wrapper within the `DftSpecification` wrapper, you use Tessent Shell to validate and process the wrapper, and create the hybrid IP. The flow is the same as that used for EDT, with the addition of the LogicBIST wrapper. Refer to “[Validating the EDT Specification and Creating the EDT IP](#)” in the *Tessent TestKompress User’s Manual* for details.

The high-level syntax for the LogicBIST wrapper looks as follows:

```
DftSpecification (module_name,id) {
  LogicBist {
    ijtag_host_interface : name
    Controller(id) {           // configure the LBIST controller
      ...
      ControllerChain {       // configure controller chain mode
        ...
      }
      SingleChainForDiagnosis { // configure single chain mode logic
        ...
      }
      ShiftCycles {           // configure shift cycles
        ...
      }
      CaptureCycles {         // configure capture cycles
        ...
      }
      PatternCount {           // configure the pattern counter
        ...
      }
      WarmupPatternCount {     // configure warm-up pattern count
        ...
      }
      NcpOptions {             // configure NCP-related hardware options
        ...
      }
      Connections {           // configure controller connections
        ...
      }
    }
    NcpIndexDecoder {          // convert LogicBIST NCP index output into
      ...                      // clock sequences for Tessent OCC usage
    }
  }
}
```

Interactions

When you specify the LogicBist/Controller wrapper, the following interactions apply:

- **EDT.** All EDT controllers are converted into EDT/LBIST hybrid controllers.
- **OCC.** OCC signals are intercepted by the LogicBIST controller. For proper LogicBIST support, generate the OCC with capture enable trigger and static clock control enabled. When you generate OCC in the same pass as the LogicBIST controller, these properties are automatically configured to support LogicBIST.
- **NCP Index Decoder.** In the case where the NCP Index Decoder and the LogicBIST controller are specified within the same Dft Specification, the tool automatically populates the count for NcpOptions from the count specified by NcpIndexDecoder/Ncp wrapper. You cannot manually specify the NcpOptions count parameter.

Limitations

The following limitations apply when you are using the LogicBist wrapper to generate the hybrid IP:

- When the NcpIndexDecoder and LBIST controller are generated in the same pass, they are automatically connected. When not inserted together, the user is responsible for the connections.
- LogicBIST does not support child OCCs.
- Pre-inserted mini-OCCs, such as those created during MemoryBIST insertion or with boundary scan, may not be automatically intercepted. Ensure that you have explicitly specified these OCCs by using the `add_core_instances` command.
- For scan-chain clocking, input design scan chains must have first scan cell clocked by an LE clock edge and last scan cell clocked by an TE clock edge.
- Does not support Low Pin Count Controller.
- You must specify the LogicBist/Controller wrapper with its associated EDT wrappers, and you can only specify one LogicBIST controller. You must use the LogicBIST controller in conjunction with EDT to generate hybrid IP.

Generating the EDT and LogicBIST IP (Dofile Flow)

When generating the hybrid EDT and LogicBIST IP, you read in a TCD and bind it to a core instance in the design.

You can define dual compression configurations for the hybrid IP. See section “[Dual Compression Configurations for the Hybrid IP](#)” for an example.

Prerequisites

- You need the scan insertion dofile and BIST-ready netlist you created during [Scan Insertion and X-Bounding](#).

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

After invocation, the tool is in unspecified setup mode. You must set the context before you use the IP generation commands.

2. Set the tool context to EDT/LogicBIST generation using the `set_context` command as follows:

```
SETUP> set_context dft -edt -logic_bist -no_rtl
```

3. Set the [TSDB](#) location if necessary. For example:

```
SETUP> set_tsdb_output_directory top_level.tsdb
```

4. Load the LogicBIST-ready design netlist using the [read_verilog](#) command. For example:

```
SETUP> read_verilog my_modified_design.v
```

5. Load one or more cell libraries into the tool using the [read_cell_library](#) command:

```
SETUP> read_cell_library atpg.lib
```

6. Set the top design using the [set_current_design](#) command. For example:

```
SETUP> set_current_design top_module
```

7. Extract the ICL using the [extract_icl](#) command. For example:

```
SETUP> extract_icl
```

8. Read in the *design_name_tessent_occ.tcd* file from the TSDB directory using the [read_core_descriptions](#) command. For example:

```
SETUP> read_core_descriptions ./top_level.tsdb/instruments/  
my_design_occ.instrument/my_design_tessent_occ.tcd
```

9. Bind the core description in memory with the specified core instance in the design using the [add_core_instances](#) command. For example:

```
SETUP> add_core_instances -instance block_inst1/my_occ
```

The `add_core_instances` command also provides a method of specifying the core instance parameters. See “[Core Instance Parameters](#)” in the *Tessent TestKompress User’s Manual* for more information.

10. Read in the dofile you generated with Tessent Shell that contains the scan insertion and X-bounding information using the `dofile` command—see “[Scan Insertion and X-Bounding](#)”. For example:

```
SETUP> dofile my_setup.dofile
```

11. Set additional EDT/LogicBIST IP generation options depending on your design objective using the following commands:

- [set_clock_controller_pins](#)
- [set_edt_options](#)
- [set_lbist_controller_options](#)
- [set_lbist_instances](#)
- [set_lbist_pins](#)
- [set_lbist_power_controller_options](#)

- [set_lpct_controller](#)

See also “[Low-Power Shift](#), “[Controller Chain Mode](#)” and “[Low Pin Count Test Controller](#).”

12. Change the tool’s system mode to analysis using the [set_system_mode](#) command as follows:

SETUP> set_system_mode analysis

During the transition from setup to analysis mode, the tool performs design rule checking.

13. You can optionally report the clock controller pins, global LogicBIST controller configuration parameters, and/or specified pins results using the following commands:

- [report_clock_controller_pins](#)
- [report_lbist_configuration](#)
- [report_lbist_pins](#)

14. Use the [write_edt_files](#) command with the -tsdb option to create EDT/LBIST files and insert the generated hardware into the design. For example, the following command generates timing constraints and writes out the EDT/LBIST files into the instruments directory within the TSDB:

ANALYSIS> write_edt_files -tsdb

The tool transitions to insertion mode after inserting the EDT/LBIST logic into the design and writes the modified netlist to the dft_inserted_designs directory within the TSDB.

For the dofile flow, specify the write_edt_files command as follows:

ANALYSIS> write_edt_files my_edt_logic -timing_constraints

Results

The following files are generated in during this step:

Table 4-1. Output Files, EDT and LogicBIST IP Generation, TSDB Flow

File	Contents
<prefix>.synthesis_dictionary	A Tcl dictionary-formatted file that is used by the run_synthesis command. This command processes the dictionary to create a synthesis script compatible with the chosen synthesis tool.
<prefix>.tcd	File containing the IP core description.
<prefix>.icl	ICL file describing hybrid EDT/LBIST logic for EDT and LBIST modes — to be used during EDT pattern generation and BIST fault simulation.

Table 4-1. Output Files, EDT and LogicBIST IP Generation, TSDB Flow (cont.)

File	Contents
<prefix>.pdl	PDL file describing test_setup initialization of hybrid EDT/LBIST logic for EDT pattern generation and BIST fault simulation.
<prefix>.v	RTL for the hybrid TK/LBIST IP.

Table 4-2. Output Files, EDT and LogicBIST IP Generation, Dofile Flow

File	Contents
<prefix>_bypass_shift_sdc.tcl	EDT bypass shift mode timing constraints file.
<prefix>_dc_script.scr	Synopsys script for synthesizing EDT and BIST logic.
<prefix>_<design_name>_edt.tcd	File containing the EDT IP core description.
<prefix>_<design_name>_lbist.tcd	File containing the LBIST IP core description.
<prefix>_edt_fast_capture_sdc.tcl	Capture mode timing constraints file for EDT or EDT-bypass mode.
<prefix>_edt_shift_sdc.tcl	EDT shift mode timing constraints file.
<prefix>_edt_top_rtl.v	Gate-level netlist that instantiates the EDT and BIST logic.
<prefix>_ltest.icl	ICL file describing hybrid EDT/LBIST logic for EDT and LBIST modes — to be used during EDT pattern generation and BIST fault simulation.
<prefix>_ltest.pdl	PDL file describing test_setup initialization of hybrid EDT/LBIST logic for EDT pattern generation and BIST fault simulation.
<prefix>_lbist.v	Per-block MISRs and top-level BIST controller.
<prefix>_lbist_sdc.tcl	File containing all LogicBIST modes including LBIST setup, shift, capture, and single chain mode.

Examples

The following example shows a single EDT/LogicBIST block. The design has X-bounding, control, and observe points controlled by the same design pin named *lbist_en*. This design does not use low-power hybrid IP.

```
// Set context for generating hybrid EDT/LogicBIST IP
```

```
set_context dft -edt -logic_bist
```

```
// Read design and DFT library
```

```
read_verilog my_core_scan_xbound.v
read_cell_library atpg.lib
set_current_design

// Define clocks and pin constraints
add_clocks 0 occ/occNX1/U7/Z -internal -pin_name NX1 //Internally generated capture clock
add_clocks 0 occ/occNX2/U7/Z -internal -pin_name NX2 // Internally generated capture clock
add_clocks 0 shift_clock
add_clocks 0 refclk -pulse_always
add_input_constraint shift_clock -c0
add_input_constraint scan_en -c0

// Scan chains and test procedure file from scan insertion
add_scan_groups grp1 scan_setup.testproc
for {set i 1} {$i <= 16} {incr i} {
    add_scan_chains chain$i grp1 scan_in$i scan_out$i
}

// Configure EDT logic
set_edt_options -location internal

// Configure LogicBIST controller
set_edt_options -lbist_misr_input_ratio 4
set_lbist_controller -max_shift 400 -max_capture 3 -max_pattern 10000 \
    -capture {clk_once 100}

// Specify LogicBIST pins
set_lbist_pins clock refclk
set_lbist_pins scan_en scan_en
set_dft_enable_options -type xbounding_en -pin_name lbist_en
set_dft_enable_options -type control_point_en -pin_name lbist_en
set_dft_enable_options -type observe_point_en -pin_name lbist_en

// Specify clock controller pins
set_clock_controller_pins shift_clock occ/shift_clock
set_clock_controller_pins scan_en occ/scan_en
```



```
set system mode atpg
```

```
// Report user settings
```

```
report_edt_configuration
```

```
report_lbist_configuration
```

```
report_lbist_pins
```

```
report_clock_controller_pins
```

```
// Generate EDT files
```

```
write edt files -tsdb -replace
```

```
// For the dofile flow, do not use the -tsdb option. For example:
```

```
// write_edt_files created -replace
```

This is a modular IP generation example. The scan chains are defined at internal pins at the block boundary. The design has a TAP controller to which the LogicBIST controller is connected during IP generation. Separate control registers are synthesized in the LogicBIST controller for independently controlling X-bounding, control and observe points. There are multiple clock controllers in the design. The design has multiple NCPs that are used in LogicBIST test in the proportion specified.

```
// Set context for generating hybrid EDT/LogicBIST IP
```

```
set_context dft -edt -logic_bist
```

```
// Read BIST-ready block designs and TOP level
```

```
read_verilog TOP.v my_core_scan_xbound.v piccpu_scan_xbound.v
```

```
read_cell_library atpg.lib
```

```
set_current_design TOP
```

```
// Define clocks and RAM control signals
```

```
add_scan_groups grp1 TOP_scan_setup.testproc
```

```
add_clocks 0 NX1 NX2 clk ramclk
```

```
add_read_control 0 ramclk
```

```
add_write_control 0 ramclk
```

```
add_clock 0 XCLK -pulse_always
```

```
add_pin_constraint scan_en c0
```

```
add_pin_constraint RST c0
```

```
// Configure LogicBIST controller
```

```
set_lbist_controller_options -max_shift 400 -max_capture 2 -max_pattern 256000
set_lbist_controller_options -capture {pulseC1P 60 pulseC2P 35 pulseR 5}
set_edt_options -location internal
set_lbist_instances -controller_location /dftblk

// Define LogicBIST pins
set_lbist_pins tck tck
set_lbist_pins clock XCLK
set_lbist_pins scan_en scan_en
set_dft_enable_options -type xbounding_en -pin_name lbist_en
set_dft_enable_options -type mcp_bounding_en -pin_name mcp_bound_en
set_dft_enable_options -type control_point_en -pin_name tp_ctrl_en
set_dft_enable_options -type observe_point_en -pin_name tp_obs_en

// Define TAP pins for connecting to LogicBIST controller
set_lbist_pins setup_shift_scan_in tdi
set_lbist_pins setup_shift_scan_out { tdo jtag/scanCfgReg_so }
set_lbist_pins shift_dr { - jtag/shift_dr }
set_lbist_pins capture_dr { - jtag/capture_dr }
set_lbist_pins update_dr { - jtag/update_dr }
set_lbist_pins test_logic_reset { - jtag/tlr }
set_lbist_pins tap_instruction_decode { - jtag/scanCfgReg_en }

// Define clock controller pins
set_clock_controller_pins lbist_en {cc_clk/lbist_en cc_NX1/lbist_en cc_NX2/lbist_en}
set_clock_controller_pins shift_clock_en {cc_clk/shift_clock_en cc_NX1/shift_clock_en
cc_NX2/shift_clock_en}
set_clock_controller_pins scan_en {cc_clk/scan_en cc_NX1/scan_en cc_NX2/scan_en}
set_clock_controller_pins capture_procedure_index {NCPdecoder/i[1] NCPdecoder/i[0]}

// Define EDT block my_core_A of "my_core" design
add_edt_block my_core_A
for {set i 1} {$i <= 16} {incr i} {
    add_scan_chain -internal my_core_A_chain$i grp1 my_core_A/scan_in$i my_core_A/
    scan_out$i
}
```

```
set_edt_power_controller shift enabled -min_switching 15
set_lbist_power_controller shift enabled -min_switching 12
set_edt_instance -block_location /dftblk

// Define EDT block my_core_B of "my_core" design
add_edt_block my_core_B
for {set i 1} {$i <= 16} {incr i} {
    add_scan_chain -internal my_core_B_chain$i grp1 my_core_B/scan_in$i my_core_B/
    scan_out$i
}
set_edt_power_controller shift enabled -min_switching 15
set_lbist_power_controller shift enabled -min_switching 12
set_edt_instance -block_location /dftblk

// Define EDT block piccpu of design "piccpu"
add_edt_block piccpu
for {set i 1} {$i <= 8} {incr i} {
    add_scan_chain -internal piccpu_chain$i grp1 piccpu/edt_si$i piccpu/edt_so$i
}
set_edt_power_controller shift enabled -min_switching 25
set_lbist_power_controller shift enabled -min_switching 25
set_edt_instance -block_location /dftblk
report_lbist_configuration
report_lbist_pins
report_clock_controller_pins
report_edt_configuration -all_blocks
write_edt_files -tsdb -replace

// For the dofile flow, do not use the -tsdb option. For example:
// write_edt_files created -replace -timing_constraints
```

Dual Compression Configurations for the Hybrid IP

Using two compression configurations when setting up the EDT logic allows you to easily set up and reuse the EDT logic for two different test phases. For example, wafer test versus package test.

You can use hybrid IP with dual configuration in EDT mode in either the low or high compression configurations (similar to non-hybrid EDT IP). For the LBIST mode of operation, the IP is used in the low compression configuration. Specifically on the output side, the MISR input taps are always taken from the low compression spatial compactor.

For more information about dual compression configurations, see “[Dual Compression Configurations](#)” in the *Tessent TestKompress User’s Manual*.

The following example IP creation dofile enables dual compression configurations, where “LC” is the low compression configuration that uses 8 channels and “HC” is the high compression configuration that uses 2 channels. The `add_edt_configurations` command enables the dual compression functionality.

```

set_context dft -edt -logic_bist
read_verilog netlist.v
read_cell_library atpg.lib
set_current_design

add_scan_groups grp1 scan_setup.testproc

//... add clocks

//... add input constraints

// add scan chains
for {set chain 1} {$chain <= 160} {incr chain} {
    add_scan_chains chain${chain} grp1 scan_in${chain} scan_out${chain}
}

// EDT parameters
add_edt_configuration LC
set_edt_options -channels 8
add_edt_configuration HC
set_edt_options -channels 2

// LBIST parameters
set_lbist_controller_options -max_shift 160 -max_capture 4 -max_pattern
100000
set_lbist_controller_options -capture {single 100}
set_lbist_pins clock refclk
set_lbist_pins scan_en scan_en
set_dft_enable_options -type xbounding_en -pin_name lbist_en
set_clock_controller_pins shift_clock [get_pins occ*/shift_clock]
set_clock_controller_pins scan_en [get_pins occ*/scan_en]

// Generate IP
set system mode analysis
write edt_files -tsdb -replace

```

Timing Constraints for EDT and LogicBIST IP

Tessent Shell generates timing constraints for the Hybrid EDT/LogicBIST IP during IP generation in Synopsys Design Constraints (SDC) format. The SDC file contains timing constraints and exceptions for all modes of operation of the IP.

For the TSDB flow, the tool creates the SDC for the current design when you execute the [extract_icl](#) command, which does the following:

- Stores the SDC in Tcl procs.
- Writes the SDC to a single file named `design_name.sdc`, where `design_name` is the name of the current design loaded in Tessent Shell.

The SDC file is located next to the extracted ICL, which is typically in the `dft_inserted_designs` directory as follows:

```
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design
```

Every instrument type (for example, MBIST, IJTAG) of the current design and all sub-blocks that provide SDC constraints are represented by a separate proc in the SDC file. Constraints for logic test-related instruments such as OCC, EDT, or LBIST, including logictest-related DFT signals, are all grouped under similar placeholder “ltest” instrument procs.

Refer to “[Timing Constraints \(SDC\)](#)” in the *Tessent Shell User’s Manual* for details about specifying timing constraints for logic test instruments when you are using the TSDB flow.

The following sections describe how to generate timing constraints when you are using the dofile flow.

Timing Constraint Generation for the Dofile Flow	86
EDT and EDT-Bypass Timing Constraints.....	87
LogicBIST Timing Constraints	87

Timing Constraint Generation for the Dofile Flow

To generate timing constraint files (along with the other EDT output files), specify the command as follows:

```
write\_edt\_files prefix -timing_constraints
```

The tool generates SDC for the following Hybrid IP modes:

- EDT — EDT shift, slow and fast capture. The capture mode constraints are the same for EDT and EDT-bypass.
- EDT-bypass — Bypass shift, slow and fast capture. The capture mode constraints are the same for EDT and EDT-bypass.

- LogicBIST — LogicBIST setup, shift, capture and diagnostic modes. The diagnostic mode constraints are generated only when single chain mode logic is synthesized in the IP.

EDT and EDT-Bypass Timing Constraints

In the dofile flow, the EDT and EDT-bypass mode timing constraints are the same as for the EDT only (non-hybrid) IP. The constraints for all the LogicBIST modes are described in different top-level TCL procs in the SDC file.

The general structure (shown below for LogicBIST setup) is as follows:

```
proc create_lbist_clock {} ...
proc set_lbist_io_pin_delays {} ...
proc set_lbist_setup_mode_constraints {} ...
proc lbist_setup_mode {} {
#Top level proc that includes the above sub procs corresponding for
this mode }
```

This SDC only describes constraints and exceptions related to the IP. This should be used together with the constraints and exceptions for the core design that is generated during pattern generation or fault simulation as applicable for the given mode. Sourcing the SDC files only defines the TCL procs. The user should use a top-level PrimeTime script that reads and links the design and execute the desired TCL procedure.

LogicBIST Timing Constraints

In the dofile flow, the four LogicBIST modes have their own lists of timing constraints/exceptions.

LogicBIST Setup Mode

This mode is for seeding the BIST registers in the EDT blocks like PRPG, MISR and EDT chain mask registers as well as those in the LogicBIST IP like vector counter, shift counter etc. The BIST controller is clocked by tck for this mode of operation.

The TDI and TDO pins of the TAP controller are used for seeding, hence IO delays are defined for those pins. These delays are defined with respect to virtual clocks named force_pi and measure_po that reflect the timing described in the test procedures.

The following constraints/exceptions are specified for the LogicBIST Setup mode:

- Set LogicBIST enable TDR bit to 1. Set the BIST setup registers to “001” corresponding to the LongSetup mode of operation for the BIST controller.

- Set false paths from EDT control signals that are not used like EDT reset and EDT update.
- Set false paths from EDT channel input pins and to EDT channel output pins.
- Set false paths through TAP controller's LogicBIST instruction enable and test logic reset outputs. The TAP controller paths for shift, capture and update are enabled.
- EDT chain mask registers are active in this mode. The paths from these registers to the design scan cells, specifically the hold paths, where the source and destination are on different clock domains are not explicitly disabled. This works correctly because the destination design scan cells are not clocked in this mode.
- Set variables for sequentially propagating case analysis constraints through the user clock controller.
- Disable clock gating checks on the shift controller clock muxes. This is because only the tck path is selected in this mode.

LogicBIST Shift Mode

This mode describes scan chain shifting. The scan cells are driven by the internally generated LogicBIST shift clock and the PRPG/MISR are driven by the internally generated LogicBIST clock. The BIST controller is clocked by the free running shift clock source. IO pins are not involved in this, hence no IO pin delays are specified.

The following constraints/exceptions are specified for the LogicBIST Shift mode:

- Set LogicBIST enable TDR bit to 1.
- Constrain the internally generated LogicBIST scan enable that reaches the scan cells to ON. There is a 4-cycle window around the time the scan enable changes during which the design scan cells are not clocked, which allows the scan enable to be described as a constant.
- Constrain the prpg_en, misr_en, and scan_en signals to MCP 4, and constrain the lbist_reset signal to MCP 3.
- Constrain the internally generated clock controller scan enable to ON.
- Set the clock source for LBIST test. The shift mode constraints TCL procedure takes a clock_select parameter to choose from LBIST shift clock source, EDT clock or TCK. The default is LBIST clock. To analyze timing when either EDT clock or TCK is used for logic test, call the top-level procedure with the desired clock parameter. For example, lbist_shift_mode edt_clock. Both shift and capture modes run with the same clock, so clock_select should be consistent for shift and capture mode.
- Constrain or exclude EDT pins like EDT clock, update, reset and other control signals.
- Disable all paths from the TAP controller.

- EDT chain mask registers are static throughout test, so declare all paths from these registers as false.
- Disable all paths from SIBs inside the EDT logic. These SIBs are clocked by the faster lbist shift clock and hence excluded in this mode.
- Set variables for sequentially propagating case analysis constraints through the user clock controller.
- Disable clock gating checks on the shift controller clock muxes. This is because only the free running shift clock source is selected in this mode.

LogicBIST Capture Mode

This mode describes the capture. The clocking for the scan cells is described in the Named Capture Procedures. The BIST controller is clocked by the free running shift clock source. IO pins are not involved in this mode, hence no IO pin delays are specified.

The following constraints/exceptions are specified for this mode:

- Set LogicBIST enable TDR bit to 1.
- Constrain the internally generated LogicBIST scan enable that reaches the scan cells to OFF. There is a 4 cycle window around the time the scan enable changes during which the design scan cells are not clocked, which allows the scan enable to be described as a constant.
- Constrain the prpg_en, misr_en, and scan_en signals to MCP 4, and constrain the lbist_reset signal to MCP 3.
- Constrain the internally generated clock controller scan enable to OFF.
- Set the clock source for LBIST test. The shift mode constraints TCL procedure takes a clock_select parameter to choose from LBIST shift clock source, EDT clock or TCK. The default is LBIST clock. To analyze timing when either EDT clock or TCK is used for logic test, call the top-level procedure with the desired clock parameter. For example, lbist_shift_mode edt_clock. Both shift and capture modes run with the same clock, so clock_select should be consistent for shift and capture mode.
- Constrain or exclude EDT pins like EDT clock, update, reset and other control signals
- Disable all paths from the TAP controller
- EDT chain mask registers are static throughout test, so declare all paths from these registers as false.
- Set variables for sequentially propagating case analysis constraints through the user clock controller
- Disable clock gating checks on the shift controller clock muxes. This is because only the free running shift clock source is selected in this mode

LogicBIST Single Chain Mode

This mode describes shifting through the concatenation of the design scan cells through the single chain mode logic, used primarily for LogicBIST diagnostics. The BIST controller and the design scan cells are clocked by tck for this mode of operation. The TDI and TDO pins of the TAP controller are used for seeding, hence IO delays are defined for those pins. The delays are defined with respect to virtual clocks named force_pi and measure_po that reflect the timing described in the test procedures.

The following constraints/exceptions are specified for the LogicBIST Single Chain mode:

- Set LogicBIST enable TDR bit to 1. Set the BIST setup registers to 100 corresponding to the SingleChain mode of operation for the BIST controller.
- Constrain or exclude EDT pins like EDT clock, update, reset and other control signals.
- Set false paths from EDT channel input pins and to EDT channel output pins.
- Set false paths through TAP controller's LogicBIST instruction enable and test logic reset outputs. The TAP controller paths for shift, capture and update are enabled.
- EDT chain mask registers are static throughout test, so declare all paths from these registers as false.
- Set the single chain mode TDR bit to 1.
- Set variables for sequentially propagating case analysis constraints through the user clock controller.
- Disable clock gating checks on the shift controller clock muxes. This is because only the tck path is selected in this mode.

EDT and LogicBIST IP Generation Command Summary

Tessent Shell provides a numbers of commands for performing EDT and LogicBIST IP generation.

Table 4-3. EDT and LogicBIST IP Generation Commands

Command	Description
read_cell_library	Loads one or more cell libraries into the tool.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
report_clock_controller_pins	Reports the clock controller pins.

Table 4-3. EDT and LogicBIST IP Generation Commands (cont.)

Command	Description
report_lbist_configuration	Reports the global LogicBIST controller configuration parameters.
report_lbist_pins	Reports the pins specified using the set_lbist_pins command.
set_clock_controller_pins	Specifies the connection information for the clock controller pins.
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.
set_current_design	Specifies the top level of the design for all subsequent commands until reset by another execution of this command.
set_edt_options	Sets options for EDT IP creation.
set_lbist_controller_options	Specifies global options to configure the LogicBIST controller.
set_lbist_instances	Specifies the instance in which the LogicBIST controller or single chain mode logic is placed.
set_lbist_pins	Specifies the connection information for LogicBIST controller pins.
set_lbist_power_controller_options	Specifies creating the low-power shift controller for LogicBIST.
set_system_mode	Specifies the operational state you want the tool to enter.
set_tsdb_output_directory	Allows changing the TSDB directory name where the generated files and modified netlists are stored.
write_design	Writes out the modified netlist.
write_edt_files	Creates the files that implement EDT logic.

ECO Implementation in the Hybrid TK/LBIST Flow

As part of the IP creation step, the tool ensures that every scan chain starts with a leading edge (LE) flip-flop and ends with a trailing edge (TE) flip-flop. The retiming flip-flops are exposed as part of the scan chains. You must make sure that the clocking of the boundary flip-flops never changes; if you need to add a flip-flop into the design, you must add it in the middle of the chain, not at the beginning or end.

The same is true for deleting a flip-flop. Only remove a scan cell that is in the middle of a chain. Do not remove a scan cell that is at the boundary because it controls the retiming latches that have been added.

In addition, consider the following:

- During IP creation, the tool generates a shift counter that is used during LogicBIST mode. By default, the tool adds seven more clock cycles to the number of shift cycles. This allows additional flip-flops to be added later through an ECO. Note that this is across all scan chains, so many flip-flops can be added in ECO mode if you wish. However, you should not exceed seven per chain.
- You should specify the size of the pattern counter. Although not directly related to the ECO process, you should specify the hardware in such a way that you can double the number of LogicTest patterns that you think will be necessary during IP generation.
- Handle timing violations after placement and routing as follows:

Note



Make sure you declare only the paths giving timing violations that are discovered late. Do not provide the entire SDC or the tool will insert muxes for FP/MCPs that are already bounded.

- a. Use the “set_xbounding_options -xbounding_enable <pin_name>” command to declare the existing enable pin that the tool should use to drive the X-bounding mux. The enable signal must be constrained to 1 so that it activates the logic that was inserted in a previous path.
- b. Use the “set_system_mode analysis” command.
- c. Use the “add_nonscan_instances -all” command to remove any non-scan cells from consideration that the tool sees as scannable. At this point, the netlist is completely scanned, so any non-scan cells should remain as is.
- d. Use the analyze_xbounding command to determine where the muxes should be inserted. While this command analyzes whether any X-sources have been missed, none should have been at this stage.
- e. Use the report_xbounding command to show where the muxes will be inserted to block those paths.
- f. Either insert the muxes yourself as part of the ECO or use the insert_test_logic command to let the tool insert them.

Chapter 5

LogicBIST Fault Simulation and Pattern Creation

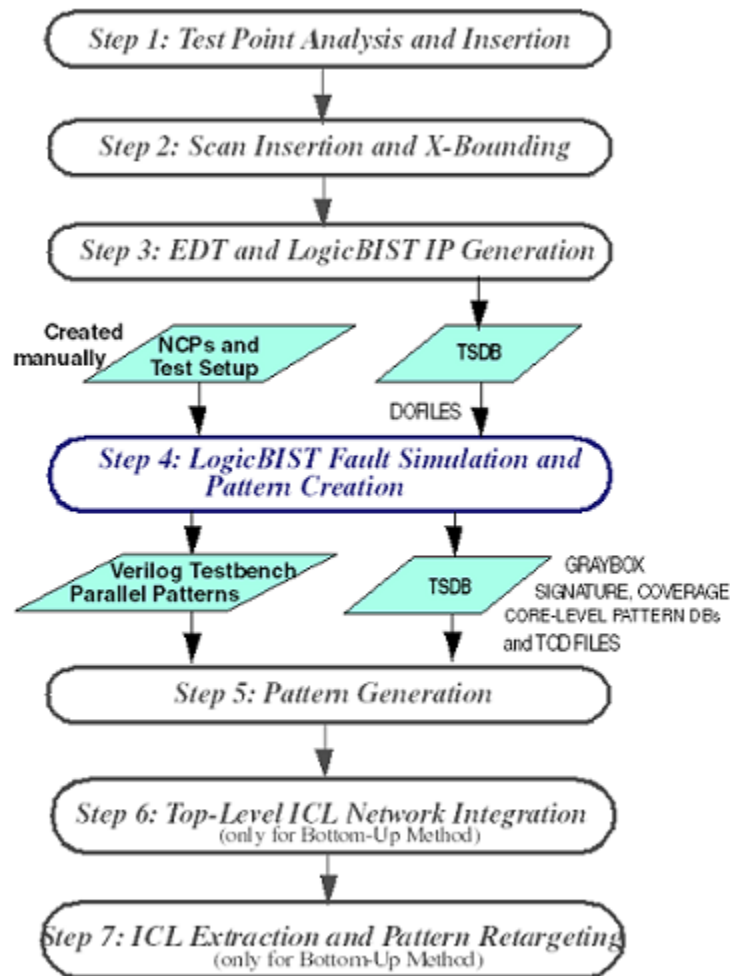
In the fourth step of the flow, you perform the tasks of fault simulation and saving the parallel LogicBIST patterns.

LogicBIST Fault Simulation and Pattern Creation Overview	93
Initial Static DFT Signal Values.....	95
Performing LogicBIST Fault Simulation and Pattern Creation.....	96
Fault Simulation When There Are Inversions	98
Example Dofiles for Core-Level Simulation	99
Fault Coverage Report for the Hybrid IP.....	100
Fault Simulation and Pattern Creation Command Summary	102

LogicBIST Fault Simulation and Pattern Creation Overview

During fault simulation with Tessent Shell, the tool generates files in the TSDB for pattern re-targeting, such as the PatternDB files. Tessent Shell performs fault simulation at the core level, generates the signatures, and computes test coverage. The tool also writes out a parallel test bench.

The following figure illustrates fault simulation and pattern creation in this flow.



The *core level* fault simulation run computes the test coverage for the core, MISR signature, and power consumption. Running the `write_tsdb_data` command during this step generates the following output files:

- **PatternDB** — Contains the relevant LogicBIST register values per pattern like PRPG, MISR, and low-power registers.
- **Tessent Core Description (TCD)** — Contains the description of the core in the LogicBIST mode of operation.
- **Flat model** — Contains the flattened circuit model, the scan trace, and all DRC-related information to a specific binary file.
- **Fault list** — Contains the fault information from the current fault list.

Low-Power Shift Simulation

If you specified a low-power controller during EDT and LogicBIST IP generation, you can specify the amount of switching activity desired during fault simulation with the following commands. For example:

```
set_lbist_control shift on -switching_threshold 15
set_system_mode analysis
...
```

Control and Observe Points Disabling and Enabling

During fault simulation, you can optionally disable/enable the control signals for the control points and the observe points using the `set_dft_enable_options` command.

For more information, see the [set_dft_enable_options](#) command.

Initial Static DFT Signal Values

Prior to fault simulation, the tool initializes DFT signals created during IP insertion to the values required for fault simulation, unless they were user specified with the `set_static_dft_signal_values` or `set_test_setup_icall` commands. This ensures that the signals are properly constrained during fault simulation.

The following table lists the DFT signals that are required for fault simulation and their default initialization values for fault simulation.

Table 5-1. Initial Static DFT Signals for Fault Simulation

DFT Signal	Value
async_set_reset_static_disable	1
control_test_point_en	1
ext_ltest_en	0
int_ltest_en	1
ltest_en	1
mcp_bounding_en	0
memory_bypass_en	1
observe_test_point_en	1
se_pipeline_en	0
x_bounding_en	1

The `x_bounding_en` signal must be set to 1 during LogicBIST operation.

The `mcp_bounding_en` signal is set to 0 to obtain optimal test coverage when performing fault simulation for stuck-at faults. To maintain the disabled state for MCP bounding, the following criteria must be met:

- There are no NCPs that will generate sequential patterns.
- The clocks located in cross-domain paths are not pulsed at the same time.
- There are enough cycles from last shift and first capture.

When performing fault simulation for transition faults, the recommended `mcp_bounding_en` signal value is 1.

Performing LogicBIST Fault Simulation and Pattern Creation

For LogicBIST, the external mode for NCPs is not relevant and is discarded with a warning message. When generating parallel patterns during fault simulation, only the `-mode_internal` patterns are valid. These patterns are the default. An error message is issued when trying to save `-mode_external` patterns.

Prerequisites

- Use the following files that you have generated during [EDT and LogicBIST IP Generation](#) using the [write_edt_files](#) command and during Logic Synthesis:
 - `tsdb_outdir/dft_inserted_designs/gpu_gate.dft_inserted_design/gpu.vg`
 - `tsdb_outdir/instruments/gpu_gate_edt_lbist.instrument/*.v`
 - `tsdb_outdir/instruments/gpu_gate_lbist.instrument/*.v`

Procedure

1. Use this Tessent Shell procedure to perform fault simulation:
2. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

After invocation, the tool is in unspecified setup mode. You must set the context before you use the fault simulation commands.

3. Set the tool context to fault simulation using the [set_context](#) command as follows:

```
SETUP> set_context patterns -scan
```

4. Open the TSDB. For example:

```
SETUP> open_tsdb ../tsdb_outdir
```


5. Load the design netlist using the [read_verilog](#) command. For example:

```
SETUP> read_verilog tsdb_outdir/dft_inserted_designs/  
gpu_gate.dft_inserted_design/gpu.vg
```

6. Load one or more cell libraries into the tool using the [read_cell_library](#) command.

```
SETUP> read_cell_library atpg.lib
```

7. Set the top design using the [set_current_design](#) command as follows:

```
SETUP> set_current_design top_module
```

8. Extract the ICL using the [extract_icl](#) command. For example:

```
SETUP> extract_icl
```

9. Add the binded TK/LBIST TCD core instances using the [add_core_instances](#) command. For example:

```
SETUP> add_core_instances -module *tessent_lbist  
SETUP> add_core_instances -module *tessent_edt_lbist
```

10. Specify the capture procedure names with the [set_lbist_controller_options](#) command as follows; be sure to include all of the NCPs that will be used in the LBIST mode:

```
SETUP> set_lbist_controller_options -capture_procedures {clkseq1 clkseq2  
clkseq3 clkseq4}
```

11. Change the tool's system mode to fault using the [set_system_mode](#) command as follows:

```
SETUP> set_system_mode analysis
```

During the transition from setup to analysis mode, the tool performs design rule checking.

12. Add faults using the [add_faults](#) command as follows:

```
ANALYSIS> add_faults -all
```

13. Specify the number of random patterns the tool will simulate using the command as follows:

```
ANALYSIS> set_random_patterns 100
```

14. Set the pattern source to LogicBIST and execute fault simulation using the [simulate_patterns](#) command as follows:

```
ANALYSIS> simulate_patterns -source bist -store_patterns all
```

15. Save the TCD, PatternDB, flat model, and fault list needed for the next step, [Pattern Generation](#), using the [write_tsdb_data](#) command as follows:

```
ANALYSIS> write_tsdb_data -replace
```

By default, the `write_tsdb_data` command only saves the scan chain data for the first 1024 patterns. To save more patterns—for example, if you discover during diagnosis that additional patterns are required—use the `-max_scan_load_unload_size` option.

16. Write out the parallel testbench using the `write_patterns` command as follows:

```
ANALYSIS> write_patterns lbist_patt_parallel.v -verilog -parallel \  
-mode_internal -param ../data/paramfile
```

Results

Now you are ready to perform [Pattern Generation](#).

Fault Simulation When There Are Inversions

Third-party tools may add inversions between the PRPG/phase-shifter and scan chains on the input side, and between scan chains and the spatial compactor on the output side. When Tessent Shell is unaware of the inversions, simulation of the Verilog patterns may result in MISR signature mismatches. To make Tessent Shell aware of these inversions, there are a couple of tasks you can perform prior to LogicBIST fault simulation.

Automatically import the inversions from the TCD. To do this, first perform EDT DRC (K rules) so that Tessent Shell learns the inversions, and then specify `write_core_description` to save the information into the TCD. Prior to running fault simulation, issue the `read_core_description` command to import the inversions. You must specify `read_core_description` before `import_scan_mode`.

Tessent Shell attempts to match each EDT block declared in the tool (by `add_core_instances`, `add_edt_block`, or through `import_scan_mode`) with those described in the TCD by matching the SCI and SCO pins. If at least one AtpgMode section is found for the current top module, but not all EDT blocks are successfully matched in the TCD, then the tool issues a warning similar to the following:

```
Could not match the following EDT instances in the TCD file and, as a  
result, the decompressor to scan chain input and scan chain output to  
compactor inversions are not imported for them:  
    <instance_name1>,  
    <instance_name2>  
You can specify the inversions manually with the set_scan_chain_options  
command or provide a TCD that includes all EDT instances.
```

If you do not run the K rules, then you can specify the inversions manually using the `set_scan_chain_options` command. For example, the following example indicates that there is an inversion between the decompressor and SCI at scan chain “chain1”:

```
set_scan_chain_options -chain_name chain1 -decompressor_to_scan_in_inversion on
```

When an inversion for the same scan is specified by `set_scan_chain_options` as well as the TCD, the user-specified inversion has priority. If the values do not match, Tessent Shell issues a warning.

Example Dofiles for Core-Level Simulation

In the TSDB flow, the tool uses the `write_tsdb_data` command to write out TCD and PatternDB files that will be used in pattern generation and ICL extraction and pattern retargeting. In the dofile flow, the tool uses the `write_core_description` and `write_patterns` commands.

For more information, see “[Pattern Generation](#)” and “[ICL Extraction and Pattern Retargeting](#).”

For the TSDB flow, the following example shows core-level fault simulation for core *alu*.

```
set_context patterns -scan
read_verilog tsdb_outdir/dft_inserted_designs/gpu_gate.dft_inserted_design/gpu.vg
read_cell_library atpg.lib
set_current_design
extract_icl
read_core_description cpu_tklbist_tessent_edt_lbist.tcd
add_core_instances -module *tessent_lbist
add_core_instances -module *tessent_edt_lbist
set_lbist_controller_options -capture_procedures {clkseq1 clkseq2 clkseq3 clkseq4}
set_system_mode analysis
add_faults -all
set_random_patterns 100
simulate_patterns -source bist
write_tsdb_data -replace
```

The following example shows the same process, but using the dofile flow.

```
set_context patterns -scan
read_verilog alu_edt_top_gate.v
read_cell_library atpg.lib
set_current_design
dofile alu_lbist.dofile
set_lbist_controller_options -capture_procedures {clkseq1 clkseq2 clkseq3 clkseq4}
set_system_mode analysis
add_faults -all
```

```
set_random_patterns 100
simulate_patterns -source bist
write_core_description alu.tcd -replace
write_patterns alu.patdb -patdb -replace
```

Fault Coverage Report for the Hybrid IP

The fault coverage report provides statistics about the total faults and the relevant faults found during LogicBIST fault simulation. The total faults are those found within the hybrid IP controller instances. The relevant faults are those found within the core logic, excluding the hybrid IP controller faults.

If the relevant coverage percentile is too low, you can improve the test by adding test points, running more patterns, and so on. Faults found within the hybrid IP controller cannot be detected or improved upon in this way.

As shown in the example below, the default fault classification for faults found within the hybrid IP controller are designated as ATPG untestable (AU) faults with sub-class hybrid LBIST (LBIST) if they have not already been identified with another designator, such as unused (UU), tied (TI), or blocked (BL).

```
// command: report_statistics

Statistics Report
Stuck-at Faults

-----
Fault Classes                                #faults      #faults
                                           (total)      (total relevant)
-----
FU (full)                                37280        29117
-----
UC (uncontrolled)                        50 ( 0.13%)  same ( 0.17%)
UO (unobserved)                          870 ( 2.33%) same ( 2.99%)
DS (det_simulation)                      22790 (61.13%) same (78.27%)
DI (det_implication)                     3444 ( 9.24%) same (11.83%)
PT (posdet_testable)                     53 ( 0.14%)  same ( 0.18%)
UU (unused)                              106 ( 0.28%) same ( 0.36%)
TI (tied)                                11 ( 0.03%)  same ( 0.04%)
BL (blocked)                              2 ( 0.01%)  same ( 0.01%)
AU (atpg_untestable)                     9954 (26.70%) 1791 ( 6.15%)
-----
Fault Sub-classes
-----
AU (atpg_untestable)
PC* (pin_constraints)                    616 ( 1.65%)  same ( 2.12%)
TC* (tied_cells)                         95 ( 0.25%)  same ( 0.33%)
MPO (mask_po)                           1039 ( 2.79%) same ( 3.57%)
LBIST (hybrid_lbist)                8163 (21.90%) deleted
Unclassified                             41 ( 0.11%)  same ( 0.14%)
*Use "report_statistics -detailed_analysis" for details.
-----
Coverage
-----
test_coverage                            70.67%        90.56%
fault_coverage                           70.44%        90.19%
atpg_effectiveness                       97.46%        97.46%
-----
#test_patterns                           10000
#clock_sequential_patterns               10000
#simulated_patterns                      10000
CPU_time (secs)                          7.8
-----
```

The following example shows that these faults appear as AU.LBIST faults in the report generated after LogicBIST fault simulation completes.

```

FaultInformation {
  version : 1;
  FaultType (Stuck) {
    FaultList {
      FaultCollapsing : FALSE;
      Format : Identifier, Class, Location;
      Instance ("") {
        0, DI.SCAN,  "/u1/STATD_reg_1_/Q";
        1, DI.SCAN,  "/u1/STATD_reg_1_/Q";
        ...
        1, UU,       "/occ/occNX2/clk_enable_latch_reg/Q";
        1, EQ,       "/occ/occNX2/U7/B";
        0, AU.LBIST,  "/m8051_single_chain_mode_logic_i/tdr_sib_i/sib_reg/Q";
        1, AU.LBIST,  "/m8051_single_chain_mode_logic_i/tdr_sib_i/sib_reg/Q";
        ...
        0, AU.LBIST,  "/m8051_single_chain_mode_logic_i/tdr_single_bypass_reg/Q";
        1, AU.LBIST,  "/m8051_single_chain_mode_logic_i/tdr_single_bypass_reg/Q";
        0, AU.LBIST,  "/m8051_edt_i/m8051_edt_decompressor_i/m8051_edt_sib_i/U8/A";
        0, EQ,       "/m8051_edt_i/m8051_edt_decompressor_i/m8051_edt_sib_i/U8/B";
        ...
        0, AU.LBIST,  "/m8051_edt_i/m8051_edt_misr_i/m8051_edt_sib_i/sib_latch_reg/Q";
        1, AU.LBIST,  "/m8051_edt_i/m8051_edt_misr_i/m8051_edt_sib_i/sib_latch_reg/Q";
        0, AU.LBIST,  "/m8051_lbist_i/m8051_lbist_ctrl_i/
m8051_lbist_capture_phase_size_reg_i/U9/Z";
        1, EQ,       "/m8051_lbist_i/m8051_lbist_ctrl_i/
m8051_lbist_capture_phase_size_reg_i/U9/A";
        ...
        0, AU.LBIST,  "/m8051_lbist_i/lbist_scan_out_reg/CP";
        1, AU.LBIST,  "/m8051_lbist_i/lbist_scan_out_reg/CP";    }
      }
    }
  }
}

```

When you are not reading ICL during fault simulation, the single chain mode logic instance faults are not classified as AU.LBIST.

Optionally, specify the [set_relevant_coverage](#) -include AU.LBIST command if you want to include the AU.LBIST faults in the relevant fault coverage.

AU.LBIST faults are not reclassified when you use the [reset_au_faults](#) command to reclassify AU faults.

Fault Simulation and Pattern Creation Command Summary

Tessent Shell provides variety of fault simulation and pattern creation commands.

Table 5-2. Fault Simulation and Pattern Creation Commands

Command	Description
add_bist_capture_range	Associates the capture procedure that is used for a specific set of patterns.
add_chain_masks	Specifies the scan chains that are masked during fault simulation and their load and unload values. Used to work around design issues.
add_faults	Adds faults to the current fault list, discards all patterns in the current test pattern set, and sets all faults to undetected (actual category is UC).
read_cell_library	Loads one or more cell libraries into the tool.
read_sdc	Reads in the SDC file that describes the false and multicycle paths.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
report_misr_connections	Reports the MISR connections.
set_bist_debug	Sets up a trace of PRPG and MISR values during a pattern's shift cycles.
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.
set_current_design	Specifies the top level of the design for all subsequent commands until reset by another execution of this command.
set_dft_enable_options	Enables and/or disables the control or observe points.
set_edt_options	Sets options for EDT IP creation and LogicBIST fault simulation.
set_lbist_controller_options	Specifies global options to configure the LogicBIST controller.
set_lbist_power_controller_options	Specifies creation of the low-power shift controller for LogicBIST.
set_power_control	Specifies the switching threshold in the patterns for fault simulation.
set_random_patterns	Specifies the number of random patterns the tool simulates.
set_system_mode	Specifies the operational state you want the tool to enter.
simulate_patterns	Performs simulation by applying the specified pattern source.
write_patterns	Writes out parallel patterns and PatternDB files.

Table 5-2. Fault Simulation and Pattern Creation Commands (cont.)

Command	Description
write_tsdb_data	Writes the <i>design_name</i> .lbist[_ <i>mode_name</i>] directories that contain all the files needed to use LogicBIST test mode during PatternsSpecification processing and to perform diagnosis of the failures.

Chapter 6

Pattern Generation

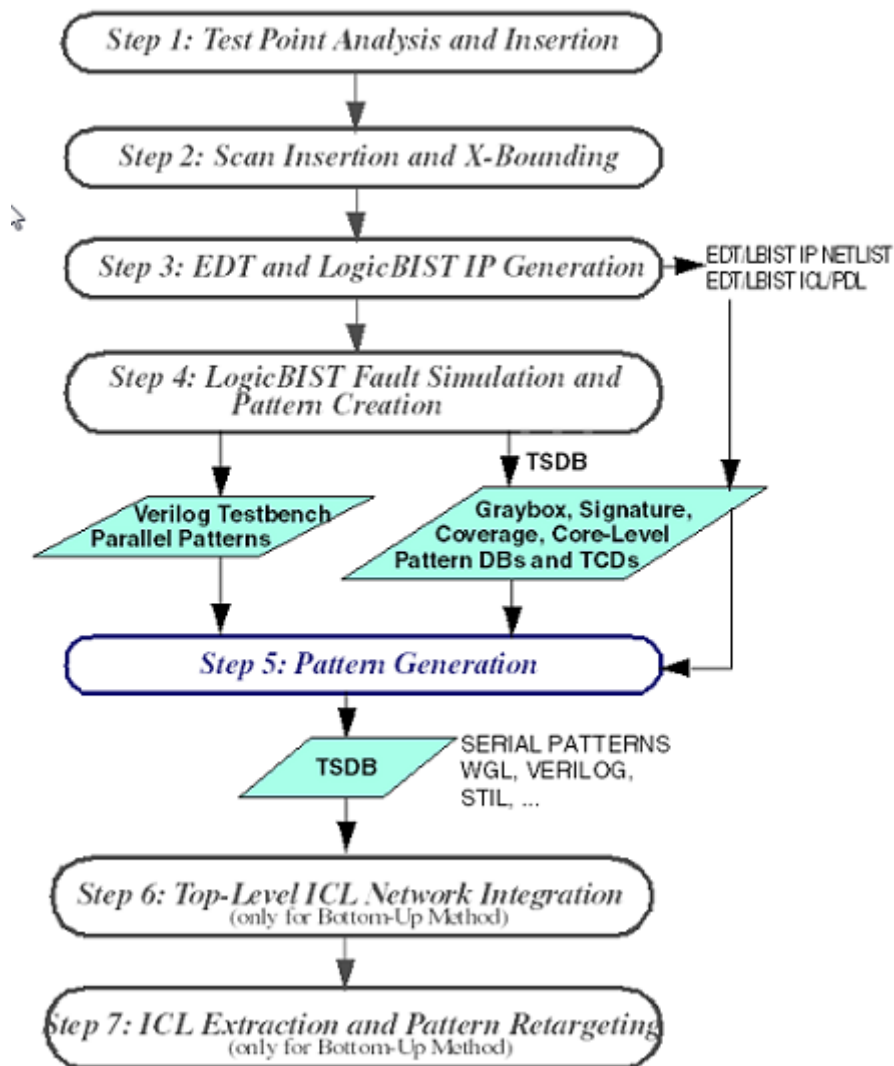
In the fifth step of the flow, you generate core-level patterns for the bottom-up method and top-level patterns (including a Verilog test bench) for the LogicBIST controller for the top-down method.

Pattern Generation Overview	105
Pattern Generation for the TSDB Flow	107
Performing Pattern Generation for the TSDB Flow	107
Performing Pattern Generation for CCM in the TSDB Flow	109
Pattern Generation for the Dofile Flow	113
Performing Pattern Generation for the Dofile Flow	115
Performing Pattern Generation for CCM in the Dofile Flow	117
Pattern Generation Command Summary	120
Single Chain Mode Diagnosis	121
Pattern Mismatch Debugging	123
Debug Based on MISR Signature Divergence	123
Debug Based On Scan Cell Monitoring	129
Usage Examples	131

Pattern Generation Overview

For the bottom-up flow, pattern generation creates core-level patterns. For the top-down flow, it creates top-level patterns (including a Verilog testbench) for the LogicBIST controller. These are the chip-level serial patterns that can be applied from the tester.

Figure 6-1. Pattern Generation



The tool supports all the formats currently supported for ATPG.

Required Inputs

To program the LogicBIST controller, you use Tessent Shell to retarget the patterns and create hardware default mode testbench/vectors and pattern_range specific vectors.

The information you need to program the LogicBIST controller is stored in the TSDB, specifically in the ICL and PDL files created during [EDT and LogicBIST IP Generation](#):

- **ICL** — The ICL file consists of ICL module description for the LogicBIST controller and all EDT blocks tested by this controller.
- **PDL** — The PDL file contains iProcs at the core level that use the ICL modules written out.

Pattern Generation for the TSDB Flow

The TSDB stores the various files required for IJTAG and CCM pattern generation. These files include the ICL, PDL, and design netlist.

Performing Pattern Generation for the TSDB Flow 107

Performing Pattern Generation for CCM in the TSDB Flow 109

Performing Pattern Generation for the TSDB Flow

The procedure for generating chip-level serial patterns uses the extracted ICL files that are stored in the TSDB. Generate the patterns using the `create_patterns_specification` and `process_patterns_specification` commands.

Prerequisites

- Modified design netlist found in the TSDB.
- The LogicBIST instruments PDL data file found in the TSDB.
- The LogicBIST instruments ICL data file found in the TSDB.
- Top-level ICL describing how the signals at the interface of the LogicBIST controller are connected to chip-level pins.
- A PDL that describes the test setup at the chip level if there is any. For example, if there is a TAP controller at the top level, then the tool requires an ICL and, optionally, PDL for the TAP controller.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

2. Set the tool context to IJTAG mode as follows:

```
SETUP> set_context patterns -ijtag
```

3. Open the TSDB if it is not already open. For example:

```
SETUP> open_tsdb tsdb_outdir
```

4. Unless it is already in memory, read the current design's extracted ICL. For example:

```
SETUP> read_icl ./tsdb_outdir/dft_inserted_designs/m01_gate.dft_inserted_design/
m01.icl
```

5. Set the current design.

```
SETUP> set_current_design m01
```

6. Define the top-level clocks. For example:

```
SETUP> add_clocks refclk -period 10 ns -free_running
```

```
SETUP> add_clocks 0 tck
```

7. Define the pin constraints. For example:

```
SETUP> add_input_constraints RST -c0
```

8. Change to analysis mode to generate patterns:

```
SETUP> set_system_mode analysis
```

9. Generate the patterns:

```
SETUP> create_patterns_specification
```

```
SETUP> process_patterns_specification
```

10. Point to the simulation library sources so all design files can be found. For example:

```
SETUP> set_simulation_library_sources -y ./techlib -extensions { v }
```

11. Simulate the logicBIST testbenches with the following command:

```
SETUP> run_testbench_simulations
```

12. As needed, monitor or check the simulation with the following command:

```
SETUP> check_testbench_simulations
```

Results

Upon completion, Tessent Shell outputs the testbench/vectors for the entire pattern set, range specific vectors, or Hardware default mode as specified in the dofile.

- If you are using the [Top-Down Flow](#), you finished all necessary steps in this flow.
- If you are using the [Bottom-Up Flow](#), you are ready to perform the [Top-Level ICL Network Integration](#).

Examples

In the following example, the core-level IJTAG dofile uses the core-level ICL to create a pattern specification that is then processed to generate a Verilog testbench for core-level pattern verification.

```
set_context patterns -ijtag
read_cell_library atpg.lib
read_icl tsdb_outdir/dft_inserted_designs/
    alu_gate.dft_inserted_design/alu.icl
set_current_design
add_clocks refclk -period 10 ns -free_running
set_system_mode analysis
create_patterns_specification
process_patterns_specification
run_testbench_simulations
check_testbench_simulations
exit
```

Performing Pattern Generation for CCM in the TSDB Flow

Controller chain mode allows you to generate ATPG patterns that target the hybrid-IP logic so that you can test the test logic itself.

For details, refer to “[Controller Chain Mode](#)” on page 64.

Note



When generating CCM patterns, do not use `add_core_instances` for the EDT/LogicBIST/OCC instruments. The presence of this command infers non-hybrid EDT pattern generation or LogicBIST fault simulation, as applicable.

Prerequisites

- Modified design netlist found in the TSDB.
- The LogicBIST instruments ICL data file found in the TSDB.
- Top-level ICL describing how the signals at the interface of the LogicBIST controller are connected to chip-level pins.
- A PDL that describes the test setup at the chip level if there is any. For example, if there is a TAP controller at the top level, then the tool requires an ICL and, optionally, PDL for the TAP controller.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

2. Set the tool context to ATPG pattern generation:

```
SETUP> set_context pattern -scan
```

3. Read in the design and libraries. For example:

```
SETUP> read_design piccpu  
SETUP> read_cell_library ../library/tessent/adk.tcelllib ../data/picdram.atpglib  
SETUP> set_current_design
```

4. Import the controller scan chain mode that you created during scan insertion. For example:

```
SETUP> import_scan_mode controller_chain_mode
```

Refer to the second example below for a dofile that shows the flow when you have turned off segmented controller chain generation in favor of connecting the controller chain scan segments into one chain during IP generation. See “[TSDB Flow Usage Details for Controller Chain Mode](#)” on page 65 for details.

5. Turn off all core clock and reset activity. Set these constraints because the faults in the design are not targeted during CCM.

```
SETUP> add_input_constraints clk -c0  
SETUP> add_input_constraints reset -c0  
SETUP> add_input_constraints shift_capture_clock -c0
```

6. Change the system mode to analysis:

```
SETUP> set_system_mode analysis
```

7. Target the faults to the EDT/LogicBIST logic. For example:

```
ANALYSIS> add_faults piccpu_rtl_tessent_lbist \  
piccpu_rtl_tessent_edt_lbist_c0_inst \  
piccpu_rtl_tessent_single_chain_mode_logic
```

8. Create and save the CCM patterns. For example:

```
ANALYSIS> create_patterns  
ANALYSIS> write_patterns ccm_patt.v -verilog -replace -serial
```

Examples

Example 1: Generating Patterns for Controller Chain Mode, Default Flow for TSDB

```
set_context pattern -scan
read_design piccpu
read_cell_library ../library/tessent/adk.tcelllib ../data/picdram.atpglib
set_current_design

import_scan_mode controller_chain_mode

add_input_constraints clk -c0
add_input_constraints reset -c0
add_input_constraints shift_capture_clock -c0

set_system_mode analysis
add_faults piccpu_rtl_tessent_lbist \
    piccpu_rtl_tessent_edt_lbist_c0_inst \
    piccpu_rtl_tessent_single_chain_mode_logic
create_patterns
write_patterns ccm_patt.v -verilog -replace -serial
```

Example 2: Generating Patterns for Controller Chain Mode, Non-Default Flow for TSDB

The following dofile example generates CCM patterns when you specify to connect the controller chain scan segments into one chain during IP generation.

```
set_context pattern -scan
read_design piccpu
read_cell_library ../library/tessent/adk.tcelllib ../data/picdram.atpglib
set_current_design

# Add edt_clock or tck as the primary clock source for CCM.
# Specify the same clock (edt_clock or tck) that you specified during
# IP generation with the set_lbist_controller_options command. If the
# edt_clock was derived from test_clock or previously specified as a
# DFT signal, then you do not need to specify it with the add_clock
# command
add_clocks 0 edt_clock

# Define a scan group. Assuming you defined scan_en as a DFT signal when
# you generated the IP, you can define a scan group without a test
# procedure file
add_scan_groups grp1

# Define scan chains on ports specified during IP creation
add_scan_chains chain1 grp1 control_chain_scan_in control_chain_scan_out

# Define the pin constraint for the ccm_en signal
add_input_constraints control_chain_enable -c1

# Turn off all core clock and reset activity.
add_input_constraints clk -c0
add_input_constraints reset -c0
add_input_constraints shift_capture_clock -c0
```

```
# For retargeting, specify to pulse the edt_clock during shift.
# The following command is only required when you use edt_clock for CCM
# and edt_clock is a top-level port, or when you use tck for CCM.
# The tool automatically generates a test procedure file that configures
# the scan_en and shift_capture_clock DFT signals. If the edt_clock is
# derived from test_clock, do not specify the
# set_procedure_retargeting_options command
set_procedure_retargeting_options -pulse_during_shift edt_clock

set_system_mode analysis

add_faults piccpu_rtl_tessent_lbist \
    piccpu_rtl_tessent_edt_lbist_c0_inst \
    piccpu_rtl_tessent_single_chain_mode_logic

create_patterns
write_patterns ccm_patt.v -verilog -replace -serial
```

Related Topics

[Controller Chain Mode](#)

Pattern Generation for the Dofile Flow

Generating IJTAG patterns with the dofile flow requires a pattern retargeting dofile template. This dofile is a template with Tcl-style comments that you must modify with the information specific to your design before using the dofile in Tessent Shell. Generating CCM patterns requires the created_ccm.dofile that was generated during IP creation.

For IJTAG pattern generation, you can use the following pattern retargeting dofile template.

```
#####
#Set the following variables before sourcing this dofile:
# Required variables:
#     set edt_lbist_icl_file_list    <filenames>
#     set edt_lbist_pdl_file_list   <filenames>
#     set edt_lbist_tcd_file        <filenames>
#     set edt_lbist_patdb_file_list <filenames>
#     set edt_lbist_reference_clock <clockname>
#
# Optional variables:
#     set edt_lbist_tester_clock_period    <period>
#     set edt_lbist_user_iproc             <procname>
#     set edt_lbist_write_hw_default_patterns <0|1: default=0>
#     set edt_lbist_write_diag_patterns    <0|1: default=0>
#####
#
#Set context to ijtag pattern retargeting
set_context pattern -ijtag

#
#Read library and design
read_cell_library ../data/atpg.lib
read_verilog created_edt_top_gate.v
#
#Read fault simulation data
read_config_data $edt_lbist_tcd_file

#
#Read ICL files
read_icl $edt_lbist_icl_file_list
#
#Set top level module for ijtag retargeting
set_current_design m8051

#
#Define top-level clocks
if {[info exists edt_lbist_tester_clock_period]} {
    add_clocks $edt_lbist_reference_clock -period
    $edt_lbist_tester_clock_period ns -free_running
}
```

```
} else {
    add_clocks $edt_lbist_reference_clock -free_running
}
add_clocks 1 tck

#
#Define pin constraints
add_input_constraints RST -c0
add_input_constraints edt_reset -c0

#
#Change to analysis mode to generate patterns
set_system_mode analysis
if {[get_context -extraction]} {
    write_icl -o m8051.icl -replace
}
foreach patdb_file $edt_lbist_patdb_file_list {
    open_patdb $patdb_file
}
report_open_patdb

#
#Read PDL files
foreach pdlfile $edt_lbist_pdl_file_list {
    source $pdlfile
}

#
#Write regular LBIST patterns
set begin_pattern 0
set end_pattern 31
set warmup_pattern_count 0
open_pattern_set lbist_normal
if {[info exists edt_lbist_user_iproc]} {
    iCall $edt_lbist_user_iproc
}
iCall run_lbist_normal lbist_clock $begin_pattern $end_pattern lbist
$warmup_pattern_count
close_pattern_set
write_patterns m8051_lbist_normal_${begin_pattern}_${end_pattern}.v -
pattern_set lbist_normal -verilog -replace

#
#Write hardware default LBIST patterns
if {[info exists edt_lbist_write_hw_default_patterns]&&
$edt_lbist_write_hw_default_patterns]} {
    open_pattern_set lbist_hw_default
    if {[info exists edt_lbist_user_iproc]} {
        iCall $edt_lbist_user_iproc
    }
    iCall run_lbist_hw_default lbist
    close_pattern_set
    write_patterns m8051_lbist_hw_default.v -pattern_set
lbist_hw_default -verilog -replace
}

#
#Write diagnostic LBIST patterns
```

```

set diag_begin_pattern 0
set diag_end_pattern 1
set diag_warmup_pattern_count 0
if {[info exists edt_lbist_write_diag_patterns] &&
$edt_lbist_write_diag_patterns} {
    open_pattern_set lbist_diag
    if {[info exists edt_lbist_user_iproc]} {
        iCall $edt_lbist_user_iproc
    }
    iCall scan_unload_register lbist_clock $diag_begin_pattern
$diag_end_pattern lbist $diag_warmup_pattern_count
    close_pattern_set
    write_patterns
m8051_lbist_diag_${diag_begin_pattern}_${diag_end_pattern}.v -pattern_set
lbist_diag -verilog -replace
}

exit

```

Performing Pattern Generation for the Dofile Flow 115

Performing Pattern Generation for CCM in the Dofile Flow 117

Performing Pattern Generation for the Dofile Flow

The procedure for generating chip-level serial patterns uses a pattern retargeting dofile that you have created for your design. The `write_patterns` command specified in the dofile writes out the PatternDB files.

Prerequisites

- Modified design netlist.
- PDL data file, *my_edt_logic_ltest.pdl*, produced by the `write_edt_files` command.
- ICL data file, *my_edt_logic_ltest.icl*, produced by the `write_edt_files` command.
- Top-level ICL describing how the signals at the interface of the LogicBIST controller are connected to chip-level pins.
- A PDL that describes the test setup at the chip level if there is any. For example, if there is a TAP controller at the top level, then the tool requires an ICL and, optionally, PDL for the TAP controller.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

2. Set the tool context to IJTAG mode using the [set_context](#) command as follows:

```
SETUP> set_context patterns -ijtag
```

3. Execute the modified pattern retargeting dofile. For example:

```
SETUP> dofile ./test_retargt.dofile
```

Results

Upon completion, Tessent Shell outputs the testbench/vectors for the entire pattern set, range specific vectors, or Hardware default mode as specified in the dofile.

- If you are using the [Top-Down Flow](#), you finished all necessary steps in this flow.
- If you are using the [Bottom-Up Flow](#), you are ready to perform the [Top-Level ICL Network Integration](#).

Examples

Example 1

In the following example, the core-level ICL IJTAG dofile uses the TCD and PatternDB files to generate a Verilog testbench for core-level pattern verification.

```
set_context patterns -ijtag
read_verilog alu_edt_top_gate.v
read_cell_library atpg.lib
read_icl alu_ltest.icl
set_current_design
set_system_mode analysis
source alu_ltest.pdl
// save the extracted ICL, will be used at the top level
write_icl -o alu.icl -replace
open_pattern_set alu_core_patt
iCall run_lbist_normal
close_pattern_set
write_patterns alu_core_patt.v -verilog -replace
```

Example 2

When you perform an iCall to the same controller within the same pattern, you receive an error because some iReadVar variables are not uniquified across the pattern set. The following example shows two iCalls that are applied to the same LBIST controller in the same pattern set and the resulting error.

```
open_pattern_set lbist_normal
  iCall run_lbist_normal lbist_clock 6 31 lbist
  iCall run_lbist_normal lbist_clock 6 31 chain_test
Error: iReadVar variable 'bist_done_start[0]' already exists in ICL
instance 'piccpu_lbist_i'.
Error: The iCall of iProc 'run_lbist_normal', which is associated with the
ICL module 'piccpu', failed.
```

To prevent this type of error, use the `teststep_name` argument in the `iProc`. This argument adds a unique string for each `iRead`. For example:

```
open_pattern_set lbist_normal
  iCall run_lbist_normal lbist_clock 6 31 lbist      0 1 lbist_mode
  iCall run_lbist_normal lbist_clock 6 31 chain_test 0 1 chain_test_mode
close_pattern_set
write_patterns piccpu_lbist_normal_6_31.v -pattern_set lbist_normal
-verilog -replace
exit
```

Related Topics

[Controller Chain Mode](#)

Performing Pattern Generation for CCM in the Dofile Flow

When generating CCM patterns using the dofile flow, you must use the CCM dofile, `create_ccm.dofile`, and the CCM `.testproc` file, `created_ccm.testproc`, that are generated during IP creation.

For details, refer to “[Controller Chain Mode](#)” on page 64.

Prerequisites

- Modified design netlist.
- ICL description of the current design.
- A PDL that describes the test setup at the chip level if there is any. For example, if there is a TAP controller at the top level, then the tool requires an ICL and, optionally, PDL for the TAP controller.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

2. Set the tool context to ATPG pattern generation:

```
SETUP> set_context pattern -scan
```

3. Read in the design and libraries. For example:

```
SETUP> read_verilog created_edt_top_gate.v  
SETUP> read_cell_library ../data/atpg.lib
```

4. Source the created_ccm.dofile generated during IP creation:

```
SETUP> dofile created_ccm.dofile
```

The created_ccm.dofile calls the CCM .testproc file, created_ccm.testproc, that was also generated during IP creation. For details, refer to the examples that follow.

5. Change the system mode to analysis:

```
SETUP> set_system_mode analysis
```

6. Add the CCM faults.

```
ANALYSIS> add_ccm_faults
```

The add_ccm_faults() TCL proc is contained within the created_ccm.dofile. This proc targets the faults to the hybrid IP.

7. Create and save the CCM patterns. For example:

```
ANALYSIS> create_patterns  
ANALYSIS> write_patterns ccm_patt.v -verilog -replace -serial
```

Examples

The follow example generates CCM patterns.

```
set_context pattern -scan  
  
read_verilog created_edt_top_gate.v  
read_cell_lib ../data/atpg.lib  
  
dofile created_ccm.dofile  
  
set_system_mode analysis  
  
add_ccm_faults  
  
create_patterns  
write_patterns ccm_patt.v -verilog -replace -serial
```

Example created_ccm.dofile

The following example shows a snippet of the CCM dofile, created_ccm.dofile, that was generated during IP creation. When a TAP is present, the tool places the TAP controller in the run-test-idle state to ensure that the IJTAG control signals are in a known state. IJTAG control signals that are top-level pins are constrained to their inactive values, as shown below.

```

add_scan_groups grp1 created_ccm.testproc
add_scan_chains ccm_chain grp1 ccm_scan_in ccm_scan_out
add_clocks 0 tck

add_input_constraints ijtag_reset c0
add_input_constraints ijtag_sel c0
add_input_constraints ijtag_ce c0
add_input_constraints ijtag_se c0
add_input_constraints ijtag_ue c0
add_input_constraints ccm_en c1

proc add_ccm_faults {} {           // Adding faults only on the hybrid logic
    add_faults {lbist_instance}
    add_faults {edt_instances}
    add_faults {single_chain_mode_logic_instance}
}

```

Example CCM .testproc file

The generated CCM .testproc file, created_ccm.testproc, is identical to the LBIST and EDT .testproc files except for minor modifications for use with CCM. Most notably, CCM .testproc files do not include internal signals (internal_shift_clock, internal_scan_en, internal_capture_en) and capture procedures. See the annotations in the following example.

```

set time scale 1.000000 ns ;
set strobe_window time 100 ;

timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 100 ;
    pulse clk_ 200 100;           //No internal_shift_clock
    pulse ramclk 200 100;
    pulse refclk 200 100;
    pulse reset 200 100;
    pulse shift_clock 200 100;
    pulse tck 200 100;           //Controller chain clock set to default tck
    period 400 ;
end;

always =
    pulse refclk ;
end;

```

```

procedure test_setup =           //Capture procedures are not necessary
timeplate gen_tpl ;
  // cycle 1 starts at time 0
cycle =
  force edt_clock 0 ;
  force refclk 0 ;
  force reset 0 ;
  force scan_enable 0 ;
  force shift_clock 0 ;
  force test_logic_reset 1 ;
  pulse refclk ;
end ;
  // cycle 2 starts at time 400
cycle =
  force test_logic_reset 0 ;
  pulse refclk ;
end;
end;

procedure shift =
scan_group grp1 ;
timeplate gen_tpl ;
  // cycle 1 starts at time 0
cycle =
  force_sci ;
  force tck 0 ; //Force tck to 0 rather than internal_shift_clock to 1
  measure_sco ;
  pulse refclk
  pulse shift_clock ;
  pulse tck ;    //Pulsing tck rather than internal_shift_clock
end;
end;

procedure load_unload =
scan_group grp1 ;
timeplate gen_tpl ;
  // cycle 1 starts at time 0
cycle =
  force clk_0 ;
  force edt_clock 0 ;    //no internal_capture_en, internal_scan_en,
  force ramclk 0 ;      //and internal_shift_clock
  force refclk 0 ;
  force reset 0 ;
  force scan_enable 1 ;/
  force shift_clock 0 ;
  force tck 0 ;
  force test_logic_reset 0 ;
  pulse refclk ;
end ;
  apply shift 16;
end;

```

Pattern Generation Command Summary

Tessent Shell provides a variety of pattern generation commands.

Table 6-1. Pattern Generation Commands

Command	Description
check_testbench_simulations	Checks the status of simulations that were previously launched by the <code>run_testbench_simulations</code> command.
create_patterns_specification	Generates a pattern specification for the specified usage. The usage is either <code>signoff</code> or <code>manufacturing</code> .
process_patterns_specification	Validates and processes the content of the <code>PatternsSpecification</code> wrapper.
read_cell_library	Loads one or more cell libraries into the tool.
read_icl	Reads ICL files into the internal database.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
run_testbench_simulations	Invokes a simulation manager to run a set of simulation test benches.
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.
set_current_design	Specifies the top level of the design for all subsequent commands until reset by another execution of this command.

Single Chain Mode Diagnosis

Single chain mode diagnosis pinpoints the location of failed scan cells in a design under test.

For a design with one or more blocks, diagnosis for LogicBIST patterns consists of two steps:

1. Identify the first failing pattern on which a MISR mismatches.
2. Use the `scan_unload_register` iProc to shift in the failing pattern stimuli using the PRPG, capture the results, and shift out the results through the single chain mode logic.

By default, during IP generation the tool adds additional SIB logic to each EDT block so that their scan cells can be bypassed as needed. During fault simulation, you specify scan chains that should be masked. During pattern generation, the tool uses the added SIB to skip EDT blocks that contain at least one masked chain. The `scan_unload_register` iProc automatically skips those blocks during single chain mode diagnosis. Single chain diagnosis cannot be performed on those blocks, but the MISR signature can still be used even in the presence of masked scan chains. This allows the tool to achieve a functional diagnosis on the rest of the blocks in the design.

The tool issues an error message when all the EDT blocks in a design have at least one masked chain. You can turn off the default behavior with the [set_lbist_controller_options](#) -single_chain_mode_skip_edt_blocks switch or the DftSpecification/LogicBist/Controller/[SingleChainForDiagnosis](#) skip_edt_blocks property.

Note



You cannot use single chain mode diagnosis on a design in which MISR mismatches occur in a block that contains masked scan chains. Because the scan_unload_register iProc does not know about the MISR mismatches, this iProc cannot detect and issue errors in this scenario. It is your responsibility to determine whether you can use single chain mode diagnosis based on the failing EDT block and masked scan chain information.

Pattern Mismatch Debugging

At the pattern generation stage, most simulation mismatches occur because the clocks were not configured properly during PDL retargeting. Rather than waiting through a time-consuming serial pattern simulation to verify that the clocks are working as expected, the simulation mismatch debugging flow provides a means to verify the clocks prior to running serial simulation.

Debug Based on MISR Signature Divergence	123
Debug Based On Scan Cell Monitoring	129
Usage Examples	131

Debug Based on MISR Signature Divergence

You can verify the BIST registers and clocks at the same time, which in turn allows you to identify the patterns corresponding to mismatches (such as MISR) as the mismatches occur.

Restrictions and Limitations

- Once the TCD file is created as described below, you cannot alter the core hierarchy (such as by ungrouping the LogicBIST controller). Altering the core hierarchy causes the list of monitor points in the TCD file to become out of sync.

Prerequisites

- You have performed the hybrid TK/LBIST flow through the pattern generation step.

Procedure

Note



The following procedure describes how to debug pattern mismatch results for both the TSDB flow and the dofile flow.

1. Verify the clocks.

TSDB flow. Enable clock verification by setting the SimulationOptions/ logic_bist_debug property in the PatternsSpecification wrapper, as follows:

```
PatternsSpecification(CHIP,gate,signoff) {
  Patterns(LogicBist) {
    ClockPeriods { ... }
    SimulationOptions {
      logic_bist_debug : setup_and_clock_verify;
    }
  }
  ...
}
```

The setup_and_clock_verify value exercises the full 256 pattern NCP count range. If you want to run a single pattern per NCP, specify the

setup_and_clock_verify_one_per_ncp value instead. This is equivalent to specifying “one_pattern_per_ncp 1” in the dofile flow.

Dofile flow. Modify the default LogicBIST retargeting dofile template that the tool creates during fault simulation (with the write_lbist_register_data command). The following example shows the default LogicBIST retargeting dofile template. To verify the clocks, uncomment and set the edt_lbist_setup_and_clock_verification_patterns variable to 1.

```
#####
#Set the following variables before sourcing this dofile:
# Required variables:
#     set edt_lbist_icl_file_list      <filenames>
#     set edt_lbist_pdl_file_list     <filenames>
#     set edt_lbist_tcd_file          <filenames>
#     set edt_lbist_patdb_file_list   <filenames>
#     set edt_lbist_reference_clock   <clockname>
#
# Optional variables:
#     set edt_lbist_tester_clock_period <period>
#     set edt_lbist_user_iproc         <procname>
#     set edt_lbist_write_hw_default_patterns <0|1: default=0>
#     set edt_lbist_write_diag_patterns  <0|1: default=0>
#     set edt_lbist_setup_and_clock_verification_patterns <0|1: \
#         default=0>
#####

...
#
#Write setup and clock verification LBIST patterns
if {[info exists edt_lbist_setup_and_clock_verification_patterns]
&& $edt_lbist_setup_and_clock_verification_patterns} {
    set one_pattern_per_ncp 1
    open_pattern_set lbist_setup_and_clock_verification
    if {[info exists edt_lbist_user_iproc]} {
        iCall $edt_lbist_user_iproc
    }
    iCall cpu_gates_tessent_occ_NX1_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall cpu_gates_tessent_occ_NX2_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall cpu_gates_tessent_occ_NX3_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall lbist_setup_and_clock_verification lbist_clock lbist \
    $one_pattern_per_ncp
    close_pattern_set
    write_patterns cpu_lbist_setup_and_clock_verification.v -
pattern_set lbist_setup_and_clock_verification -verilog -replace
}

exit
```

The syntax for the lbist_setup_and_clock_verification iProc is:

```
iProc lbist_setup_and_clock_verification { {clock_select lbist_clock}
{mode_name lbist} {one_pattern_per_ncp 0} {teststep_name ""} } {...}
```

As shown in the example iCall, the generated template sets the `one_pattern_per_ncp` variable to 1 by default. This means that if you have four NCPs, the tool runs four tests, one for each NCP.

The default pattern run is 256. To exercise the full pattern range, in the generated template, change the line “`set one_pattern_per_ncp 1`” from 1 to 0.

2. If clock verification fails, investigate and fix possible causes, such as:
 - a. The On-Chip Clock Controller (OCC) is not set up correctly. Ensure that the arguments are specified correctly if you are using an iCall to setup the OCC.
 - b. Ensure that you have correctly set any primary input pin asserts that are needed for the clocks to operate.
 - c. You may find that while debugging the clock verification failures the capture window is too small. That is, capture clock pulses occur outside the window where “`capture_en=1`.” If this is the case, return to LogicBIST IP generation and specify the following command to increase the capture window:

```
set_lbist_controller_options -max_capture_cycles #
```

3. Run Verilog simulation with the LogicBIST debugging feature enabled as shown below, and identify any failing LogicBIST patterns.

TSDB flow. Set the `SimulationOptions/logic_bist_debug` property in the `PatternsSpecification` wrapper as follows:

```
...
SimulationOptions {
    logic_bist_debug : bist_registers_and_clock_verify;
...

```

For both flows, the resulting transcript will include mismatch statements such as those shown in bold below. The statements tell you at which pattern the MISR signature started to diverge from the expected value.

Note



To display the passing data, specify “`+show_passing_regs`” when you start the simulator.

```
...
Setting up controller TLB_coreB_I1.coreB_lbist_i
  Number of patterns      : 5 (5 + 0 warm-up patterns)
  Pattern Length         : 40
  Shift Clk Select       : 0b01
  Capture Phase Width    : 0x3 Shift Clock Cycles
  PRPG Seed              : 0x66241da0
  MISR Seed              : 0x000000
Starting controller TLB_coreB_I1.coreB_lbist_i in Normal mode,
patterns 0 to 3
  Checking that the controller TLB_coreB_I1.coreB_lbist_i DONE
signal is NO at the beginning of the test
  Mismatch at pattern 2 for TLB_coreB_I1.coreB_edt_lbist_i.misr:
Expected = 83ab37 Actual = 7854d0
  Mismatch at pattern 3 for TLB_coreB_I1.coreB_edt_lbist_i.misr:
Expected = 9b96e3 Actual = 5e161a
Test Complete for controller TLB_coreB_I1.coreB_lbist_i
  Checking that signal DONE is YES for controller
TLB_coreB_I1.coreB_lbist_i
Checking results of controller TLB_coreB_I1.coreB_lbist_i
  Expected Signature for controller TLB_coreB_I1.coreB_lbist_i :
0x9b96e3
Turning off LogicBist controller TLB_coreB_I1.coreB_lbist_i
...
```

Dofile flow. Enable the debugging feature by specifying `sim_monitor` with the `run_lbist_normal` iProc. The syntax is:

```
iProc run_lbist_normal { {clock_select lbist_clock} {begin_pattern 0} {end_pattern 31}
  {mode_name lbist} {warmup_pattern_count 0} {misr_compares 1}
  {sim_monitor off} {teststep_name ""} } {...}
```

For example:

```
set sim_monitor 1
open_pattern_set lbist
  iCall run_lbist_normal lbist_clock 980 999 lbist 0 0 $sim_monitor
close_pattern_set
```

4. Re-run the simulation so that you can identify the failing flop associated with the particular pattern where the MISR started to diverge.

TSDB flow. Regenerate the LogicBist wrapper with the Diagnosis Options/`extract_flop_data` property enabled, as shown below, and execute the `run_testbench_simulations` command.

```
PatternsSpecification(CHIP,gate,signoff) {
  Patterns (LogicBist) {
    ClockPeriods { ... }
    ...
    TestStep (diagnostic) {
      LogicBist {
        CoreInstance(CHIP) {
          run_mode : run_time_prog ;
          begin_pattern : 2;
          end_pattern : 2;
          DiagnosisOptions {
            extract_flop_data : on ;
          }
        }
      }
    }
    ...
  }
}
```

Examine the results of the re-run simulation to identify the failing flops, noting the Verilog simulation results against what fault simulation predicted. Use Tessent Visualizer to trace the flops to the cause of the failure. For detailed usage examples, refer to “[Usage Examples](#).”

Dofile flow. Use the scan_unload_register iProc, as shown in the following sample dofile template. Set the edt_lbist_write_diag_patterns variable to 1, and adjust the diag_begin_pattern and diag_end_pattern settings accordingly. For example, if the MISR signature failed at pattern 2, you would set the begin and end patterns to 2.

```
#####
#Set the following variables before sourcing this dofile:
# Required variables:
#     set edt_lbist_icl_file_list      <filenames>
#     set edt_lbist_pdl_file_list     <filenames>
#     set edt_lbist_tcd_file          <filenames>
#     set edt_lbist_patdb_file_list   <filenames>
#     set edt_lbist_reference_clock   <clockname>
#
# Optional variables:
#     set edt_lbist_tester_clock_period <period>
#     set edt_lbist_user_iproc         <procname>
#     set edt_lbist_write_hw_default_patterns <0|1: default=0>#
set edt_lbist_write_diag_patterns      <0|1: default=0>
#     set edt_lbist_setup_and_clock_verification_patterns <0|1:
#         default=0>
#####
...
#
#Write diagnostic LBIST patterns
set diag_begin_pattern 2
set diag_end_pattern 2
set diag_warmup_pattern_cnt 0
if {[info exists edt_lbist_write_diag_patterns] &&
$edt_lbist_write_diag_patterns} {
    open_pattern_set lbist_diag
    if {[info exists edt_lbist_user_iproc]} {
        iCall $edt_lbist_user_iproc
    }
    iCall cpu_gates_tessent_occ_NX1_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall cpu_gates_tessent_occ_NX2_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall cpu_gates_tessent_occ_NX3_inst.setup fast_capture_mode
on capture_window_size 2 static_clock_control external
    iCall scan_unload_register lbist_clock $diag_begin_pattern
$diag_end_pattern lbist $diag_warmup_pattern_cnt
    close_pattern_set
    write_patternscpu_lbist_diag_${diag_begin_pattern}_
${diag_end_pattern}.v -pattern_set lbist_diag -verilog -replace
}
```

Results

The following transcript example shows a mismatch at an lbist_scan_out pin.


```

...
300ns: piccpu MISR Seed : 0x0000000
49300ns: Starting controller TLB_coreB_I1.coreB_edt_lbist_i in Normal
mode, patterns 0 to 0
51000ns: Checking that the controller TLB_coreB_I1.coreB_edt_lbist_i
DONE signal is NO at the beginning of the test
62800ns: Test Complete for controller TLB_coreB_I1.coreB_edt_lbist_i
69000ns: Scanning out capture results of vector 0 for controller
TLB_coreB_I1.coreB_edt_lbist_i
180024ns: Mismatch at pin 0 name lbist_scan_out,
Simulated x, Expected 0
180100ns: Corresponding ICL register:
TLB_coreB_I1.coreB_edt_single_chain_mode_logic_i.TLB_coreB_I1.coreB_edt_i
nternal_scan_registers_i.coreB_A_chain1[18]
180100ns: Corresponding design object: coreB_A/u11/PRB_reg/DFF1
181700ns: Turning off LogicBist controller TLB_coreB_I1.coreB_edt_lbist_i

```

Debug Based On Scan Cell Monitoring

You can have the tool monitor the scan chains and return information about scan cells associated with unexpected unload values.

Restrictions and Limitations

- Once the TCD file is created as described below, you cannot alter the core hierarchy (such as by ungrouping the LogicBIST controller). Altering the core hierarchy causes the list of monitor points in the TCD file to become out of sync.

Prerequisites

- You have performed the hybrid TK/LBIST flow through the pattern generation step.

Procedure

Note



The following procedure describes how to debug pattern mismatch results for both the TSDB flow and the dofile flow.

1. Verify the clocks as described in step 1 of “[Debug Based on MISR Signature Divergence](#)” on page 123.
2. If clock verification fails, investigate and fix possible causes as described in step 2 of “[Debug Based on MISR Signature Divergence](#)” on page 123.
3. Run Verilog simulation with the monitor_scan_cells LogicBIST debugging feature enabled as shown below. When specified, the tool monitors the scan chain output pins, detects when an unexpected value is unloaded, and reports which shift cycle and scan cell failed.

TSDB flow. Set the SimulationOptions/logic_bist_debug property in the PatternsSpecification wrapper as follows:

```
...
SimulationOptions {
    logic_bist_debug : monitor_scan_cells;
...

```

Dofile flow. Specify sim_monitor with the run_lbist_normal iProc and include the monitor_scan_cell argument. For example:

```
set sim_monitor 1
open_pattern_set lbist
    iCall run_lbist_normal lbist_clock 980 999 lbist 0 0 monitor_scan_cell \
$sim_monitor
close_pattern_set
```

Results

When a mismatch occurs the tool first reports the scan chain output pin where the mismatch was observed, and then maps the mismatch to a pattern, shift cycle, and scan cell. For both messages it reports the simulated and expected values. If there is inversion between the scan cell and the scan out, the simulated/expected values on these two lines will be different. If the failing scan cell is within a sub-chain of a hard module, then the message will only report the scan cell and not the pin of the scan cell that failed.

The following transcript example shows mismatches when the wrong values are observed on scan chain cells.

```
#ns: Pattern_set serial_load
#ns: Setting up controller xtea_tk_lbist_ip_tessent_lbist_i
#ns: Number of patterns : 3 (3 + 0 warm-up patterns)
#ns: Pattern Length : 2 #ns: Shift Clk Select : 0b00
#ns: Capture Phase Width : 0x20 Shift Clock Cycles
#ns: PRPG Seed : 0x3e0a
#ns: MISR Seed : 0x000000
#ns: Starting controller xtea_tk_lbist_ip_tessent_lbist_i in Normal mode,
patterns 0 to 2
#ns: Checking that the controller xtea_tk_lbist_ip_tessent_lbist_i DONE
signal is NO at the beginning of the test

#ns: Mismatch at pin xtea_tk_lbist_ip_tessent_edt_lbist_c0_inst/
tessent_persistent_cell_edt_scan_out_0_buf/Y, Simulated x, Expected 1
#ns: Corresponding scan cell for pattern 0 at shift cycle 0:
hard_mod2_inst1/OUT_R_reg_0_, Simulated x, Expected 0

#ns: Mismatch at pin xtea_tk_lbist_ip_dft_tessent_edt_lbist_c0_inst/
tessent_persistent_cell_edt_scan_out_0_buf/Y, Simulated x, Expected 0
#ns: Corresponding scan cell for pattern 0 at shift cycle 11:
hard_mod2_inst1/IN2_R_reg_3_, Simulated x, Expected 1

#ns: Mismatch at pin xtea_tk_lbist_ip_tessent_edt_lbist_c1_inst/
tessent_persistent_cell_edt_scan_out_2_buf/Y, Simulated x, Expected 1
#: Corresponding scan cell for pattern 1 at shift cycle 0: B_R_reg_2_/Q,
Simulated x, Expected 1
```

Usage Examples

Your debugging efforts may include debugging clock verification and MISR signature mismatches. In addition, to help with debugging, you can display passing capture clocks and BIST register values.

Example 1: Clock Verification Debugging

The following example illustrates how you can use the `setup_and_clock_verification` functionality to quickly find a problem with the clocking. Assume the design contains a PLL that generates three clocks. The tool inserts Tessent OCCs on each clock along with the following NCP Index Decoder that declares the NCPs, the clocks, and how many pulses occur in each NCP.

```
DftSpecification(cpu, gates2) {
  LogicBist {
    NcpIndexDecoder {
      Ncp(CLK1) {
        cycle(0): cpu_gates_tessent_occ_NX1_inst;
        cycle(1): cpu_gates_tessent_occ_NX1_inst;
      }
      Ncp(CLK2) {
        cycle(0): cpu_gates_tessent_occ_NX2_inst;
        cycle(1): cpu_gates_tessent_occ_NX2_inst;
      }
      Ncp(CLK3) {
        cycle(0): cpu_gates_tessent_occ_NX3_inst;
        cycle(1): cpu_gates_tessent_occ_NX3_inst;
      }
      Ncp(ALL) {
        cycle(0): cpu_gates_tessent_occ;
        cycle(1): cpu_gates_tessent_occ;
      }
      Ncp(ALL_1p) {
        cycle(0): cpu_gates_tessent_occ;
      }
    }
  }
}
```

During IP generation, the NCPs CLK1, CLK2, CLK3, ALL, and ALL_1p are specified to operate at 10%, 10%, 10%, 60%, and 10%, respectively.

```
set_lbist_controller_options -capture_procedures {CLK1 10 CLK2 10 CLK3 10
ALL 60 ALL_1p 10}
```

After LogicBIST fault simulation, the dofile template for LogicBIST pattern retargeting includes the following pattern set to perform the setup and clock verification. Suppose that you had inadvertently changed the `capture_window_size` argument of one of the OCC iCall statements, as shown below.

```
set edt_lbist_setup_and_clock_verification_patterns 1
...
#
#Write setup and clock verification LBIST patterns
if {[info exists edt_lbist_setup_and_clock_verification_patterns] &&
$edt_lbist_setup_and_clock_verification_patterns} {
    set one_pattern_per_ncp 1
    open_pattern_set lbist_setup_and_clock_verification
    if {[info exists edt_lbist_user_iproc]} {
        iCall $edt_lbist_user_iproc
    }
    iCall cpu_gates_tessent_occ_NX1_inst.setup fast_capture_mode on
capture_window_size 1 static_clock_control external
    iCall cpu_gates_tessent_occ_NX2_inst.setup fast_capture_mode on
capture_window_size 2 static_clock_control external
    iCall cpu_gates_tessent_occ_NX3_inst.setup fast_capture_mode on
capture_window_size 2 static_clock_control external
    iCall lbist_setup_and_clock_verification lbist_clock lbist
$one_pattern_per_ncp
    close_pattern_set
    write_patterns cpu_lbist_setup_and_clock_verification.v -pattern_set
lbist_setup_and_clock_verification -verilog -replace
```

The simulation transcript (with time units removed) looks as follows. Note that:

- Four NCPs are being tested, as requested during LogicBIST fault simulation, so there are four invocations of the LogicBIST controller.
 - CLK1 NCP tested at pattern 0, active from patterns 0:25.
 - CLK2 NCP tested at pattern 26, active from patterns 26:51.
 - CLK3 NCP tested at pattern 52, active from patterns 52:76.
 - ALL NCP tested at pattern 77, active from patterns 77:255.
- During the CLK1 and ALL NCPs, the tool detects that the clock generated by the NX1 OCC was expecting two clock pulses but only one was observed.

```

1 # Pattern_set lbist_setup_and_clock_verification
2 # Applying pattern to test NCP 'CLK1'
3 # Setting up controller cpu_gate_tessent_lbist_i
...
...
...
46 # Applying pattern to test NCP 'ALL'
47 # Setting up controller cpu_gate_tessent_lbist_i
48 #   Number of patterns   : 1 (1 + 0 warm-up patterns)
49 #   Pattern Length      : 16
50 #   Shift Clk Select     : 0b01
51 #   Capture Phase Width : 0x2 Shift Clock Cycles
52 #   PRPG Seed           : 0x1e7f97a1
53 #   MISR Seed           : 0xc17203
54 # Starting controller cpu_gate_tessent_lbist_i in Normal mode,
patterns 77 to 77
55 #   Checking that the controller cpu_gate_tessent_lbist_i DONE
signal is NO at the beginning of the test
56 # Mismatch at pattern 77 (NCP 'ALL') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_cl
ock_out_mux.y': Expected 2 pulses but observed 1
57 # Mismatch at pattern 78 (NCP 'ALL') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_cl
ock_out_mux.y': Expected 2 pulses but observed 1
58 # Test Complete for controller cpu_gate_tessent_lbist_i
59 #   Checking that signal DONE is YES for controller
cpu_gate_tessent_lbist_i
60 # Turning off LogicBist controller cpu_gate_tessent_lbist_i
61 #
62 #
63 # Simulation finished at time                      93361
64 # Number of miscompares =                          4
65 # Number of 0/1 compares =                       1143
66 # Number of Z compares  =                          0

```

In Tessent Visualizer, in the GTKWave viewer application, you can zoom in on the CLK1 NCP capture window and see that there is indeed only a single capture pulse. For more information, see “[Pin Data Window](#)” in the *Tessent Shell User’s Manual*.

Similarly, if you were to zoom in on ALL NCP, you would see that NX1’s OCC generates only a single pulse while the NX2 and NX3 OCCs have two pulses.

Because you know there is a problem with the NX1 OCC, you can check how the retargeting dofile was initialized with the corresponding iCall and see that the capture_window_size is set to 1 instead of 2.

Example 2: MISR Signature Mismatch Debugging

The following example illustrates how to debug a MISR signature mismatch. For purposes of this example, a MISR signature mismatch is triggered by using a Verilog force statement on the gate pin A0, which fans into the register pin uINTR/SERVICE_LEVEL_0_reg/D. This emulates a stuck-at fault starting at pattern 97.

Suppose you have a testbench that runs from pattern 0 to 99 with LogicBIST debugging enabled with the iProc bist_registers_and_clock_verify argument. For example:

```
open_pattern_set lbist_normal
  if {[info exists edt_lbist_user_iproc]} {
    iCall $edt_lbist_user_iproc
  }
  iCall cpu_gates_tessent_occ_NX1_inst.setup ...
  iCall cpu_gates_tessent_occ_NX2_inst.setup ...
  iCall cpu_gates_tessent_occ_NX3_inst.setup ...
  iCall run_lbist_normal lbist_clock 0 99 lbist 0 1
bist_registers_and_clock_verify close_pattern_set
write_patterns cpu_lbist_normal_0_99.v -pattern_set lbist_normal -verilog
-replace
```

The simulation transcript for this pattern set looks as follows. Note that:

- Without enabling simulation debug, the only failure you would see is the final MISR signature that is scanned out and compared at the end of the pattern. To isolate the pattern at which the MISR started to fail, you would have to rerun the simulation multiple times, possibly with a binary search. This can be time consuming for serial simulations for large designs.
- Simulation debug provides two mismatches on the MISR register, observed after the capture windows at patterns 98 and 99. Because the MISR comparison occurred before the pattern 98 scan chains were unloaded into the MISR, the MISR signature failure actually corresponds to the previous pattern, 97.

```
# Pattern_set lbist_normal
# Setting up controller cpu_gate_tessent_lbist_i
#   Number of patterns   : 100 (100 + 0 warm-up patterns)
#   Pattern Length      : 72
#   Shift Clk Select     : 0b01
#   Capture Phase Width : 0x2 Shift Clock Cycles
#   PRPG Seed           : 0x597fc27a
#   MISR Seed           : 0x000000
# Starting controller cpu_gate_tessent_lbist_i in Normal mode, patterns 0 to 99
# Checking that the controller cpu_gate_tessent_lbist_i DONE signal is NO at the
beginning of the test
# Mismatch at pattern 98 for cpu_inst.cpu_gate_tessent_edt_lbist_i.misr: Expected =
0x11504d Actual = 0x665059
# Mismatch at pattern 99 for cpu_inst.cpu_gate_tessent_edt_lbist_i.misr: Expected =
0xc71675 Actual = 0x33473f
# Test Complete for controller cpu_gate_tessent_lbist_i
#   Checking that signal DONE is YES for controller cpu_gate_tessent_lbist_i
# Checking results of controller cpu_gate_tessent_lbist_i
#   Expected Signature for controller cpu_gate_tessent_lbist_i: 0x397758
# Mismatch at pin          1 name          SIB_SCAN_OUT, Simulated 1, Expected 0
# Previous scan out : pin SIB_SCAN_OUT = cpu_gate_tessent_edt_lbist_i.misr[0]
# Mismatch at pin          1 name          SIB_SCAN_OUT, Simulated 1, Expected 0
# Previous scan out : pin SIB_SCAN_OUT = cpu_gate_tessent_edt_lbist_i.misr[1]
# Mismatch at pin          1 name          SIB_SCAN_OUT, Simulated 1, Expected 0
...
# Previous scan out : pin SIB_SCAN_OUT = cpu_gate_tessent_edt_lbist_i.misr[21]
# Mismatch at pin          1 name          SIB_SCAN_OUT, Simulated 1, Expected 0
# Previous scan out : pin SIB_SCAN_OUT = cpu_gate_tessent_edt_lbist_i.misr[22]
# Turning off LogicBist controller cpu_gate_tessent_lbist_i
```

You can now identify the failing flop associated with pattern 97 by creating a diagnostic LogicBIST pattern, as follows:

```
open_pattern_set lbist_diag
  if {[info exists edt_lbist_user_iproc]} {
    iCall $edt_lbist_user_iproc
  }
  iCall cpu_gates_tessent_occ_NX1_inst.setup ...
  iCall cpu_gates_tessent_occ_NX2_inst.setup ...
  iCall cpu_gates_tessent_occ_NX3_inst.setup ...
  iCall scan_unload_register lbist_clock 97 97 lbist 0 1
bist_registers_and_clock_verify
close_pattern_set
write_patterns cpu_lbist_diag_97_97.v -pattern_set lbist_diag -verilog -
replace
```

The simulation transcript for this diagnostic pattern looks as follows:

```
# Pattern_set lbist_diag
# Setting up controller cpu_gate_tessent_lbist_i
#   Number of patterns : 1 (1 + 0 warm-up patterns)
#   Pattern Length     : 72
#   Shift Clk Select   : 0b01
#   Capture Phase Width : 0x2 Shift Clock Cycles
#   PRPG Seed          : 0x5c953748
#   MISR Seed          : 0x6f2d3a
# Starting controller cpu_gate_tessent_lbist_i in Normal mode, patterns
97 to 97
#   Checking that the controller cpu_gate_tessent_lbist_i DONE signal is
NO at the beginning of the test
# Test Complete for controller cpu_gate_tessent_lbist_i
# Scanning out capture results of vector 97 for controller
cpu_gate_tessent_lbist_i
# Mismatch at pin          1 name          SIB_SCAN_OUT, Simulated 0, Expected
1
# Corresponding ICL register:
cpu_gate_tessent_single_chain_mode_logic_i.cpu_gate_tessent_edt_internal_
scan_registers_i.chain2[27]
# Corresponding design object: uINTR/SERVICE_LEVEL_0_reg
# Turning off LogicBist controller cpu_gate_tessent_lbist_i
```

The diagnostic pattern identifies uINTR/SERVICE_LEVEL_0_reg as the failing flop. You can find the cause of the failure by comparing the simulation waveform results against the LogicBIST fault simulation prediction in Tessent Visualizer.

Example 3: Displaying Passing Capture Clocks and BIST Register Values

By default, the LogicBIST simulation debug feature only displays a message when it detects a capture clock and/or BIST register mismatch. Using two simulator plusargs, you can direct the simulator to display passing capture clock and BIST register values as well.

To show the passing BIST register comparisons, use the +show_passing_regs plusargs. As the simulation runs, the tool displays the MISR, PRPG, and low-power (if present) values for each pattern. For example:

```
# Pattern_set lbist_normal
# Setting up controller cpu_gate_tessent_lbist_i
#   Number of patterns   : 3 (3 + 0 warm-up patterns)
#   Pattern Length      : 72
#   Shift Clk Select     : 0b01
#   Capture Phase Width : 0x2 Shift Clock Cycles
#   PRPG Seed           : 0x2070a3dc
#   MISR Seed           : 0x05aed3
# Starting controller cpu_gate_tessent_lbist_i in Normal mode, patterns
25 to 27
#   Checking that the controller cpu_gate_tessent_lbist_i DONE signal is
NO at the beginning of the test
# Expected value 0x2070a3dc after initialization for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lfsm_vec
# Expected value 0x11e23c4d after initialization for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lbist_lp_mask_shift_reg
# Expected value 0x05aed3 after initialization for
cpu_inst.cpu_gate_tessent_edt_lbist_i.misr
# Expected value 0x4dd3bf15 at pattern 25 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lfsm_vec
# Expected value 0x67bb5c4f at pattern 25 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lbist_lp_mask_shift_reg
# Expected value 0x05aed3 at pattern 25 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.misr
# Expected value 0x04b71551 at pattern 26 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lfsm_vec
# Expected value 0x434886a2 at pattern 26 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lbist_lp_mask_shift_reg
# Expected value 0x3a2db4 at pattern 26 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.misr
# Expected value 0x7a9fb426 at pattern 27 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lfsm_vec
# Expected value 0x3f32bc25 at pattern 27 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.lbist_lp_mask_shift_reg
# Expected value 0xf4968c at pattern 27 for
cpu_inst.cpu_gate_tessent_edt_lbist_i.misr
# Test Complete for controller cpu_gate_tessent_lbist_i
#   Checking that signal DONE is YES for controller
cpu_gate_tessent_lbist_i
#   Checking results of controller cpu_gate_tessent_lbist_i
#   Expected Signature for controller cpu_gate_tessent_lbist_i:
0x268b4d# Turning off LogicBist controller cpu_gate_tessent_lbist_i
```

To show the passing clock comparisons, use the +show_passing_clocks plurals. As each pattern runs, the expected number of pulses for each clock displays, including the currently active NCP name. For example:

```
# Pattern_set lbist_normal
# Setting up controller cpu_gate_tessent_lbist_i
#   Number of patterns   : 3 (3 + 0 warm-up patterns)
#   Pattern Length      : 72
#   Shift Clk Select     : 0b01
#   Capture Phase Width : 0x2 Shift Clock Cycles
#   PRPG Seed           : 0x2070a3dc
#   MISR Seed           : 0x05aed3
# Starting controller cpu_gate_tessent_lbist_i in Normal mode, patterns
25 to 27#   Checking that the controller cpu_gate_tessent_lbist_i DONE
signal is NO at the beginning of the test
# 2 expected pulses at pattern 25 (NCP 'CLK1') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 25 (NCP 'CLK1') for clock
'cpu_inst.cpu_gates_tessent_occ_NX2_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 25 (NCP 'CLK1') for clock
'cpu_inst.cpu_gates_tessent_occ_NX3_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 26 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_clock_out_mux.y'
# 2 expected pulses at pattern 26 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX2_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 26 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX3_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 27 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_clock_out_mux.y'
# 2 expected pulses at pattern 27 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX2_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 27 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX3_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 28 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX1_inst.tessent_persistent_cell_clock_out_mux.y'
# 2 expected pulses at pattern 28 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX2_inst.tessent_persistent_cell_clock_out_mux.y'
# 0 expected pulses at pattern 28 (NCP 'CLK2') for clock
'cpu_inst.cpu_gates_tessent_occ_NX3_inst.tessent_persistent_cell_clock_out_mux.y'
# Test Complete for controller cpu_gate_tessent_lbist_i
#   Checking that signal DONE is YES for controller
cpu_gate_tessent_lbist_i
#   Checking results of controller cpu_gate_tessent_lbist_i
#   Expected Signature for controller cpu_gate_tessent_lbist_i: 0x268b4d
#   Turning off LogicBist controller cpu_gate_tessent_lbist_i
```

You can specify `+show_passing_clocks` and `+show_passing_regs` at the same time.

Chapter 7

Top-Level ICL Network Integration

In the sixth step of the bottom-up flow, you use the top-level netlist that instantiates all of the LogicBIST implemented cores.

Note



You perform this step only when using the [Bottom-Up Flow](#).

Chapter topics follow this sequence:

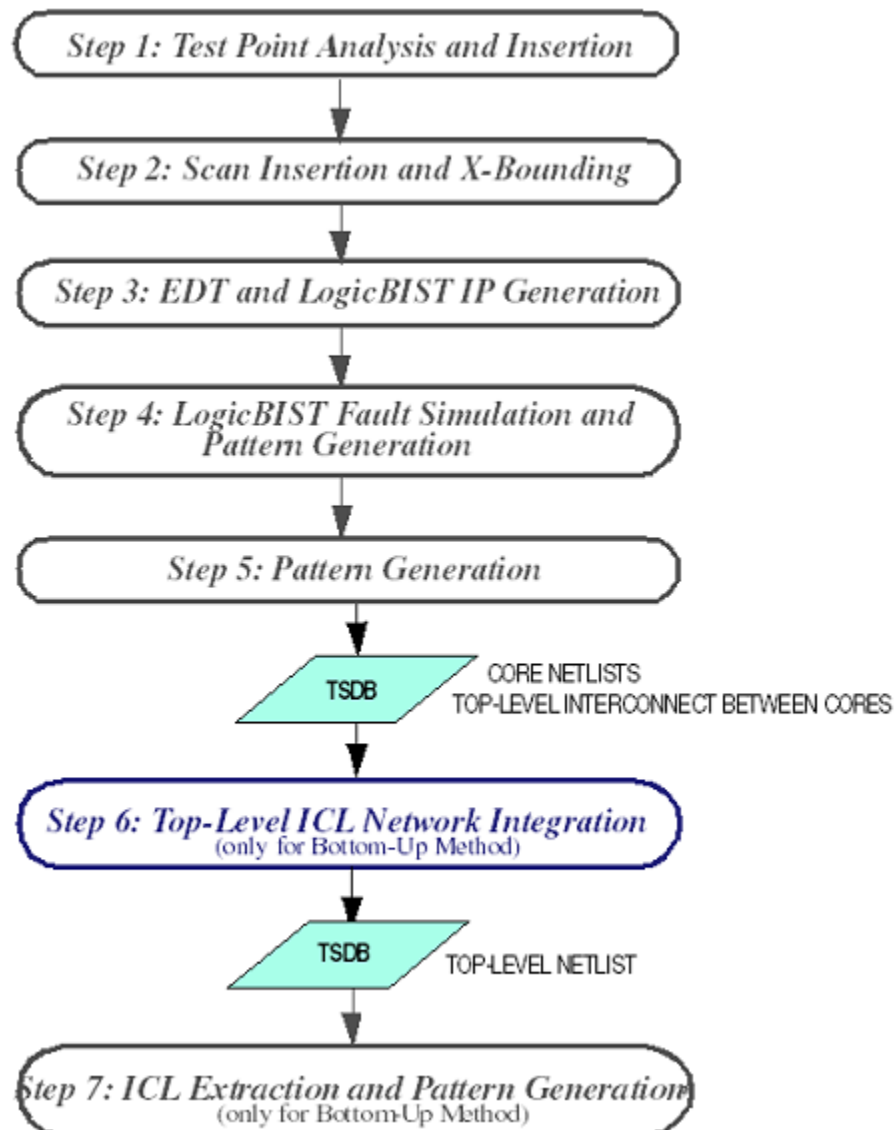
Top-Level ICL Network Integration Overview	141
Performing Top-Level ICL Network Integration.....	142
Top-Level ICL Network Integration Command Summary.....	146

Top-Level ICL Network Integration Overview

For top-level ICL network integration, you use the top-level netlist that instantiates all the LogicBIST implemented cores. You insert IJTAG compliant SIBs to provide access to each of the LogicBIST cores as well as connect these SIBs and cores to the top-level TAP controller.

The following figure illustrates this step within Hybrid TK/LBIST flow.

Figure 7-1. Top-Level SIB Netlist Insertion



Performing Top-Level ICL Network Integration

It is recommended to shadow each core by a separate SIB to provide maximum flexibility for test scheduling. You can connect EDT control signals from the core to the top level as well at this time.

You can create a [DftSpecification](#) and use the Tessent Shell [process_dft_specification](#) functionality to automate this task.

Prerequisites

The following input is required for this step of the flow:

- The netlists for all your cores created in [EDT and LogicBIST IP Generation](#).
- Top-level netlist with instantiation of cores and interconnect between them.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

After invocation, the tool is in unspecified setup mode. You must set the context before you can invoke the top-level SIB network Insertion commands.

2. Set the tool context to dft mode using the [set_context](#) command as follows:

```
SETUP> set_context dft -no_rtl
```

3. Load the LogicBIST-ready design netlists using the [read_verilog](#) command. For example:

```
SETUP> read_verilog top.v <tsdb_dft_inserted_designs_directory>/<core_name_1>.v  
<tsdb_dft_inserted_designs_directory>/<core_name_N>.v
```

4. Open the TSDB if it is not already open. For example:

```
SETUP> open_tsdb tsdb_outdir
```

5. Load one or more cell libraries into the tool using the [read_cell_library](#) command. For example:

```
SETUP> read_cell_library atpg.lib
```

6. Set the top design using the [set_current_design](#) command. For example:

```
SETUP> set_current_design top
```

7. Implement an optional TCL proc named “process_dft_specification.post_insertion” that will be executed after processing the DftSpecification and before writing out the top-level IJTAG network-inserted design. This proc can be used to connect the EDT signals from the cores to top-level design pins. For example:

```
SETUP> proc process_dft_specification.post_insertion  
{root wrapper} {  
    create_port  
    edt_clock  
    create_connection  
    edt_clock [get_pins *_edt_i/edt_clock] ...  
}
```

8. Load the top-level IJTAG network description in DftSpecification format. For example:

```
SETUP> read_config_data top.dft_spec
```

9. Validate and implement the IJTAG network using the [process_dft_specification](#) command. This command generates the RTL description for the IJTAG network components and the top-level IJTAG network-inserted design.

SETUP> process_dft_specification

Results

Now you are ready to perform [ICL Extraction and Pattern Retargeting](#).

Examples

The example in the following figure describes a design that has two cores, *alu* and *cpu*. The *alu* core has two EDT blocks named *B1* and *B2*. Two instances of the *alu* core are in the final top-level design (*/w2/A* and */w2/B*) and a single instance of the *cpu* core (*/c1*).

The example shows the Tessent Shell integration dofile for generating the IJTAG network and connecting the core-level EDT signals to the top level. The *process_dft_specification.post_insertion* TCL procedure connects the core-level EDT pins to the top level of the design. The example demonstrates the creation of shared top-level pins for all of the cores corresponding to the EDT control signals, such as *edt_clock*, *edt_update*, and *edt_bypass*. The example also shows the creation of dedicated top-level pins for channel inputs and outputs for each of the cores.

Figure 7-2. Top-Level ICL Network Integration Dofile Example

```
set_context dft -no_rtl

read_verilog top.v tsdb_outdir/dft_inserted_designs/alu_gate.dft inserted_deisgn/alu.v
tsdb_outdir/dft_inserted_designs/cpu_gate.dft inserted_deisgn/cpu.v

read_cell_library atpg.lib

read_icl {tsdb_outdir/dft_inserted_designs/alu_gate.dft inserted_deisgn/alu.icl
tsdb_outdir/dft_inserted_designs/cpu_gate.dft inserted_deisgn/cpu.icl}

set_current_design top

//TCL proc to connect EDT signals from block to top pins
proc process_dft_specification.post_insertion {root args} {
    foreach i {alu1_edt_channels_in1 alu1_edt_channels_in2
                alu2_edt_channels_in1 alu2_edt_channels_in2
                cpu_edt_channels_in1
                edt_clock edt_reset edt_update edt_bypass
                edt_single_bypass_chain} {
        create_port -direction input $i
    }
    foreach o {alu1_edt_channels_out1 alu1_edt_channels_out2
                alu2_edt_channels_out1 alu2_edt_channels_out2
                cpu_out1} {
        create_port -direction output $o
    }
}
```



```

    }
    create_connection alu1_edt_channels_in1 w2/A/B1_edt_channels_in1
    create_connection alu1_edt_channels_in2 w2/A/B2_edt_channels_in1
    create_connection alu2_edt_channels_in1 w2/B/B1_edt_channels_in1
    create_connection alu2_edt_channels_in2 w2/B/B2_edt_channels_in1
    create_connection cpu_edt_channels_in1 c1/edt_channels_in1
    create_connection alu1_edt_channels_out1 w2/A/B1_edt_channels_out1
    create_connection alu1_edt_channels_out2 w2/A/B2_edt_channels_out1
    create_connection alu2_edt_channels_out1 w2/B/B1_edt_channels_out1
    create_connection alu2_edt_channels_out2 w2/B/B2_edt_channels_out1
    create_connection cpu_edt_channels_out1 c1/edt_channels_out1
    foreach i {edt_clock edt_reset edt_update edt_bypass
        edt_single_bypass_chain} {
        create_connection $i w2/A/$i
        create_connection $i w2/B/$i
        create_connection $i c1/$i
    }
}

//Insert IJTAG network using DFT specification
read_config_data top.dft_spec
process_dft_specification

```

The DftSpecification that is referenced in the dofile follows. In it, the HostScanInterface/Interface wrapper specifies the top-level TAP controller design pins for the IJTAG interface signals. Three SIBs are to be inserted, each of which controls a core whose instance path name is specified in the DesignInstance wrapper. The SIBs are inserted one level above the core instance as specified in the parent_instance property for SIBs w2_A and w2_B. The SIB for core *cpu* (/c1) is inserted at the top level. The naming of the SIB instances is controlled using the leaf_instance_name property. The enable, control signal, and scan IO pin names (the IJTAG network interface at the core boundary) are taken from the ICL description of the cores in the *alu.icl* and *cpu.icl* files.

Figure 7-3. DftSpecification Example

```
DftSpecification(top, 3sibs_tap) {
  IjtagNetwork {
    HostScanInterface(3sibs_tap) {
      Interface {
        reset_polarity: active_high;
        tck: tck;
        reset: jtag/tlr;
        select: jtag/lbist_inst;
        capture_en: jtag/capture_dr;
        shift_en: jtag/shift_dr;
        update_en: jtag/update_dr;
        scan_in: tdi;
        scan_out: jtag/lbist_reg_out;
      }
      Sib(c1) {
        leaf_instance_name: piccpu_access_sib;
        DesignInstance(/c1) {}
      }
      Sib(w2_B) {
        parent_instance: /w2;
        leaf_instance_name: m8051_B_access_sib;
        DesignInstance(/w2/B) {}
      }
      Sib(w2_A) {
        parent_instance: /w2;
        leaf_instance_name: m8051_A_access_sib;
        DesignInstance(/w2/A) {}
      }
    }
  }
}
```

The RTL and ICL description of the generated IJTAG instruments is written out in the *tsdb_outdir/instruments/top_3sibs_tap_ijtag.instrument* directory. The top-level IJTAG network inserted design is written out as *tessent_outdir/dft_inserted_designs/top_3sibs_tap.dft_inserted_design/top.vg*. The final top-level netlist can be obtained by combining the *top.vg* file along with the gate-level synthesized netlists of the IJTAG instruments. The ICL files generated by this example can be used for downstream steps that use IJTAG, such as top-level LBIST pattern retargeting.

Note



For more information about ICL insertion using the DftSpecification, refer to the “[IJTAG Network Insertion](#)” chapter of the *TessentIJTAG User’s Manual*.

Top-Level ICL Network Integration Command Summary

Tessent Shell provides a variety of commands that are used for top-level SIB network insertion.

Table 7-1. Top-Level ICL Network Integration Commands

Command	Description
create_connections	Connects, ports, pins, port objects.
create_instance	Instantiates an instance of a module mod_spec inside a design module of a current design.
create_port	Creates a port on a specified design module.
process_dft_specification	Validates and processes the content contained in a DftSpecification wrapper.
read_cell_library	Loads one or more cell libraries into the tool.
read_config_data	Loads a configuration data file into the Tessent Shell environment.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.
set_current_design	Specifies the top level of the design for all subsequent command until reset of another execution of this command.
write_design	Writes the current design to the specified file in Verilog netlist format.

Chapter 8

ICL Extraction and Pattern Retargeting

In the seventh and final step of the bottom-up flow, you perform ICL extraction and pattern retargeting.

Note



You perform this step only when using the [Bottom-Up Flow](#).

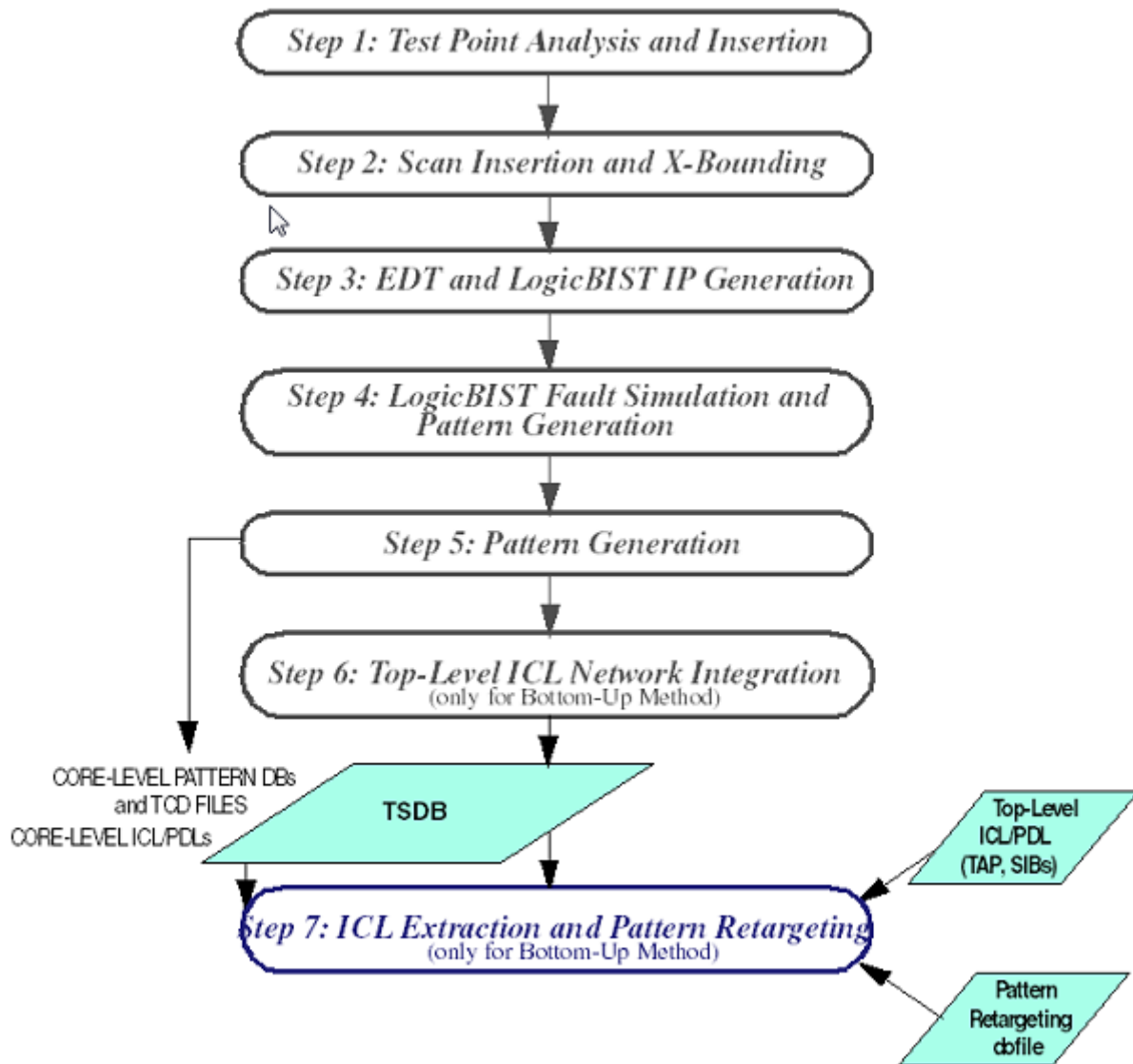
ICL Extraction and Pattern Retargeting Overview	149
Performing ICL Extraction and Pattern Retargeting	150
Usage Examples for ICL Extraction and Pattern Retargeting	151
ICL Extraction and Pattern Retargeting Command Summary	154

ICL Extraction and Pattern Retargeting Overview

You use the fully integrated top-level netlist, top-level ICL/PDL files for your IJTAG instruments such as the TAP controller and clock controller, and the per-core LogicBIST files generated at the core level. The final tester patterns can be written out in any of the supported formats.

The following figure illustrates the ICL extraction and pattern retargeting step.

Figure 8-1. ICL Extraction and Pattern Retargeting



Performing ICL Extraction and Pattern Retargeting

ICL extraction can be used here instead of manually creating the top-level ICL describing the SIB access network and connectivity between your instruments and the LogicBIST cores.

Prerequisites

The required inputs for this step of the flow are as follows:

- PDL and ICL files for each core in your design created during [EDT and LogicBIST IP Generation](#).

- PatternDB and TCD files for each core in your design generated during [Pattern Generation](#).
- Top-level PDL and ICL files that include information about the TAP and SIBs.
- Pattern retargeting dofile.

Procedure

1. From a shell, invoke Tessent Shell using the following syntax:

```
% tessent -shell
```

After invocation, the tool is in unspecified setup mode. You must set the context before you can invoke the pattern retargeting commands.

2. Set the tool context to IJTAG mode using the [set_context](#) command as follows:

```
SETUP> set_context patterns -ijtag
```

3. Execute the pattern retargeting dofile that reads all the ICL and PDL files and specifies test scheduling. For example:

```
SETUP> dofile ./test_retarget.dofile
```

4. Write a current pattern set to a file in a specified format using the [write_patterns](#) command as follows:

```
SETUP> write_patterns my_patterns -verilog -parallel -mode_internal -replace
```

Usage Examples for ICL Extraction and Pattern Retargeting

You can use pattern merging commands to process your core patterns in parallel or sequentially.

Example 1

This example merges all the core patterns to be run in parallel.

```
// Set context for pattern retargeting
set_context patterns -ijtag

// Read output design from top-level ICL network integration step
read_verilog top_lbist_integrated.v
read_cell_library atpg.lib

// Read ICL for cores extracted during block level LogicBIST pattern generation
read_icl {alu.icl cpu.icl}

// Read user provided ICL for top-level components
```

```
read_icl {top_sib.icl jtag_controller.icl}

// Set current design and report ICL matched modules
set_current_design top
report_module_matching -icl

// Define top-level clocks and pin constraints
add_clocks 0 refclk -pulse_always
add_clocks 0 tck

add_input_constraints RST -c0
add_input_constraints edt_reset -c0

// Change to analysis mode and perform ICL extraction
set_system_mode analysis

// Report user settings
report_clocks
report_input_constraints

// Create pattern specification, or you can read in a previously created patterns specification
// file by using the read_config_data command
create_patterns_specification
process_patterns_specification
```

Example 2

This example shows the pattern merging commands required for running 100 patterns of all cores sequentially. The initial setup is the same as the previous full example.


```

PatternsSpecification(top,gate,signoff) {
  Patterns(LogicBist_Top) {
    ...
    TestStep(cpu_serial_load) {
      LogicBist {
        CoreInstance(cpu) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 99;
        }
      }
    }
    TestStep(alu1_serial_load) {
      LogicBist {
        CoreInstance(alu1) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 99;
        }
      }
    }
    TestStep(alu2_serial_load) {
      LogicBist {
        CoreInstance(alu2) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 99;
        }
      }
    }
  }
}

```

Example 3

The following example shows a pattern specification that you can use to run 100 patterns of cpu in parallel with 50 patterns of an alu core, followed by another alu core running 50 patterns by itself.

```

PatternsSpecification(top,gate,signoff) {
  Patterns(LogicBist_Top) {
    ...
    TestStep(cpu_and_alu1_serial_load) {
      LogicBist {
        CoreInstance(cpu) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 99;
        }
        CoreInstance(alu1) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 49;
        }
      }
    }
    TestStep(alu2_serial_load) {
      LogicBist {
        CoreInstance(alu2) {
          run_mode : run_time_prog;
          begin_pattern : 0;
          end_pattern : 49;
        }
      }
    }
  }
}

```

ICL Extraction and Pattern Retargeting Command Summary

Tessent Shell provides a variety of commands for ICL extraction and pattern retargeting.

Table 8-1. ICL Extraction and Pattern Retargeting Commands

Command	Description
create_patterns_specification	Generates a pattern specification for the specified usage. The usage is either signoff or manufacturing.
process_patterns_specification	Validates and processes the content of the PatternsSpecification wrapper.
read_cell_library	Loads one or more cell libraries into the tool.
read_icl	Loads one or more ICL files into the tool.
read_verilog	Reads one or more Verilog files into the specified or default logical library.
set_context	Specifies the current usage context of Tessent Shell. You must set the context before you can enter any other commands in Tessent Shell.

Chapter 9

Hybrid TK/LBIST Embedded Structures

The EDT/LogicBIST hybrid IP reduces the area overhead compared to a separate implementation of EDT and LogicBIST IP. This is done by re-using parts of the IP for both EDT and LogicBIST modes. This hybrid logic controls input stimuli generation and output response comparison and is implemented separately for each EDT block. The top-level controller including the LogicBIST FSM is then connected to these hybrid IP inserted EDT blocks.

The hybrid TK/LBIST flow utilizes the following during generating and embedding the EDT/LogicBIST IP into your design.

- EDT and LogicBIST blocks (blocks and ELT cores)
- A decompressor
- Low-Power Shift controller
- LogicBIST controller
- EDT controller
- EDT compactor
- MISR
- Bypass logic

Shared Logic	155
Inserted Hybrid TK/LBIST IP	156
Scan Chain Masking	160
New LogicBIST Control Signals	161
Clocking	161
Programmable Registers Inside Hybrid IP	163
Low-Power Shift Controller	164

Shared Logic

The EDT/LogicBIST hybrid IP is shared in a number of ways.

The sharing is accomplished as follows:

- The EDT decompressor is re-configured as the PRPG by blocking the channel inputs during LogicBIST mode.
- Lockup cells (those placed in between the decompressor and the phase shifter) are re-used as hold cells when low-power LogicBIST is implemented.
- Biasing gates used when synthesizing EDT low-power hardware is shared with chain masking.
- The phase shifter network is used as is for driving scan chains from the PRPG.
- The spatial compactor XOR network is re-used for compacting scan chain outputs into MISR inputs.
- Lockup cells required between the EDT/LogicBIST IP and the design scan cells are shared between both modes.

Inserted Hybrid TK/LBIST IP

The hybrid TK/LBIST IP is inserted during the IP generation step. The same DFT architecture serves two functions: provide a mechanism for compression-based ATPG through EDT (decompressor, compactor, and bypass logic) and provide LogicBIST test capability.

For LogicBIST test, the inserted IP includes a:

- Pseudo Random Pattern Generator (PRPG)

LogicBIST generates patterns internally using a PRPG. In the hybrid TK/LBIST architecture, the PRPG is shared with the decompressor, which internally contains a linear-feedback shift register (LFSR). When a pattern runs, the PRPG seed determines the value to be expected on the MISR signature. In LogicBIST mode, the PRPG (decompressor) is controlled by the `lbist_en` and the `lbist_prpg_en` signals, which trigger test input generation to the design under test (DUT).

- Multiple Input Signature Register (MISR)

The MISR is a programmable register that connects to the output of the compactor; it is part of the shared EDT IP. During LogicBIST mode, the accumulate input (`mISR_accumulate_en`) tells the MISR when to accumulate/capture the output of the compactor during the shift state. The size of the MISR varies depending on the DUT and the Tessent command options specified, but it will be comprised of 24-bit and 32-bit segments.

The hybrid TK/LBIST flow generates hybrid IP that consists of the following three blocks:

- Hybrid EDT/LogicBIST controller

The hardware shared between EDT and LogicBIST consists of the decompressor/PRPG, the compactor, and the bypass. You can identify this block by its module name or instance name with the prefix *designname_designID_edt_lbist*. This module is described in the instruments directory of the TSDB.

- LogicBIST controller

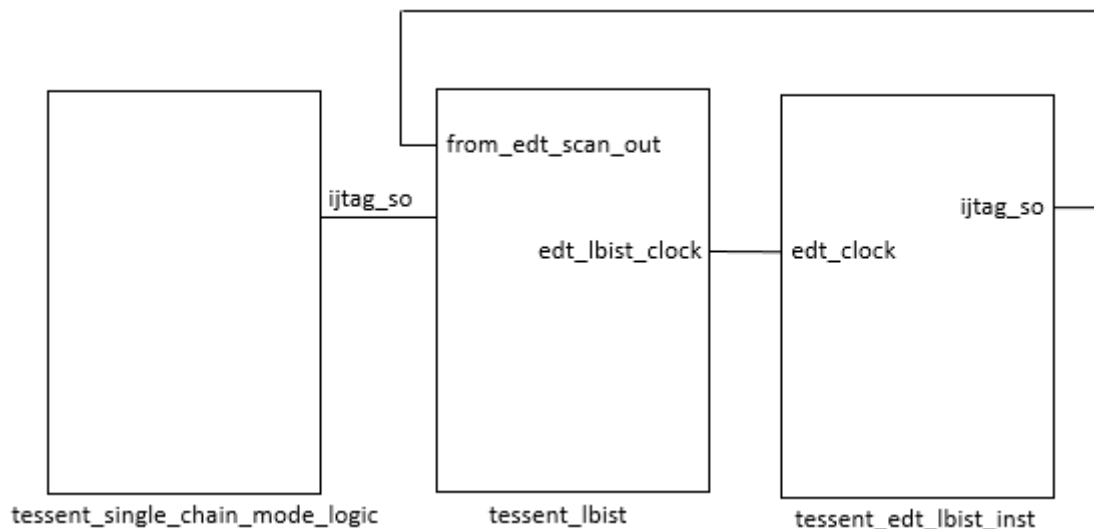
This block contains hardware that is only used during LogicBIST mode and does not play a significant role during ATPG, except for some clock paths and multiplexed control signal lines that travel through this block. You can identify the controller in the netlist with the prefix *designname_designID_lbist*. This module is described in the instruments directory of the TSDB.

The LogicBIST controller drives the shared hybrid EDT/LBIST block during LogicBIST mode and functions as a propagation path for the scan chain in the single chain mode.

- Single chain mode controller

This module is responsible for setting the design mode to single chain mode for LogicBIST diagnosis. This block is optional.

The LogicBIST controller connects to the hybrid EDT/LBIST controller and the single chain mode controller as follows:



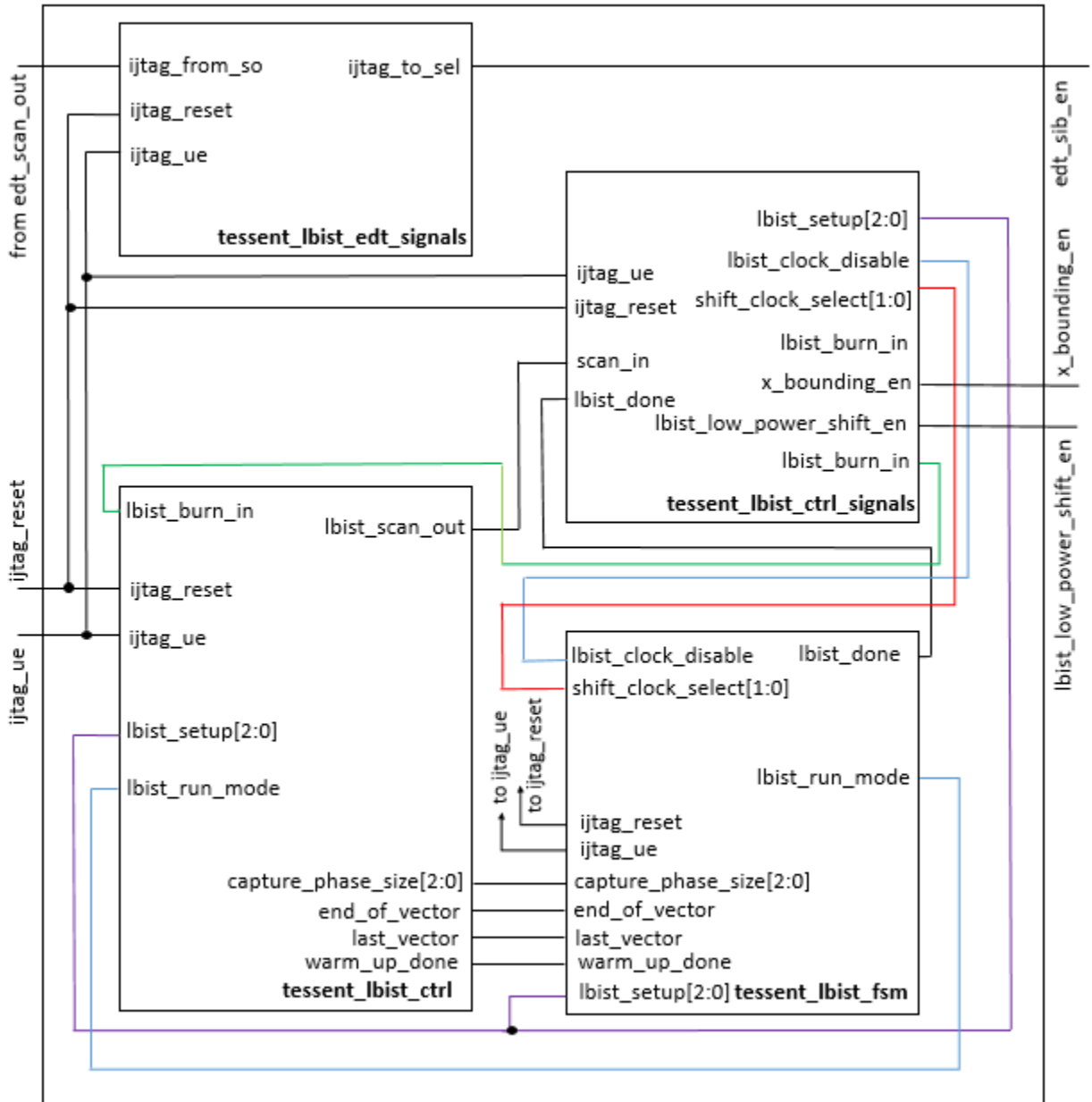
The LogicBIST Controller

Internally, the LogicBIST controller block (*tessent_lbist*) contains the following blocks:

- Control signals: *tessent_lbist_ctrl_signals*
- Finite state machine (FSM): *tessent_lbist_fsm*

- LogicBIST control: tessent_lbist_ctrl
- SIB that enables the shared EDT/LogicBIST logic: tessent_lbist_edt_signals
- Combinational cloud and some persistent cells feeding into the outputs. (Not discussed.)

The modules are connected as follows. Colored connections are for clarity only.



Control Signals Block

The control signals block generates all necessary control signals to the LogicBIST controller, the hybrid EDT/LBIST controller, and the NCP decoder block, and it generates the `lbist_clock_disable` signal.

The tool asserts the `lbist_clock_disable` signal during setup so that the values being shifted are not disturbed. This signal also suspends the clock to the MISR, PRPG, counters, and so on, when switching the clock from TCK to the chosen shift clock (`shift_clock_src`, by default), and vice versa. This prevents clock glitches stemming from the 3:1 mux from disturbing flops that were just loaded (and scan flops during a LogicBIST diagnostic scan out). The signal is de-asserted during shift.

The outputs primarily fan out and drive the FSM and the LogicBIST controller. The LogicBIST enable, burn-in controls, clock selection signal, and low-power shift enable are outputs of this block that control LogicBIST operation.

This block connects to the JTAG network and is part of the LogicBIST scan chain.

FSM Block

The seven-state FSM ensures that signals generated by the control signals block reach the LogicBIST control in the desired sequence. That is, the FSM shapes the control signals such as `lbist_reset`, `lbist_run_mode`, and `lbist_enable` so that they toggle correctly. The FSM also ensures that the `prpg_en` signal reaches the PRPG, and the `misr_accumulate_enable` signal reaches the MISR.

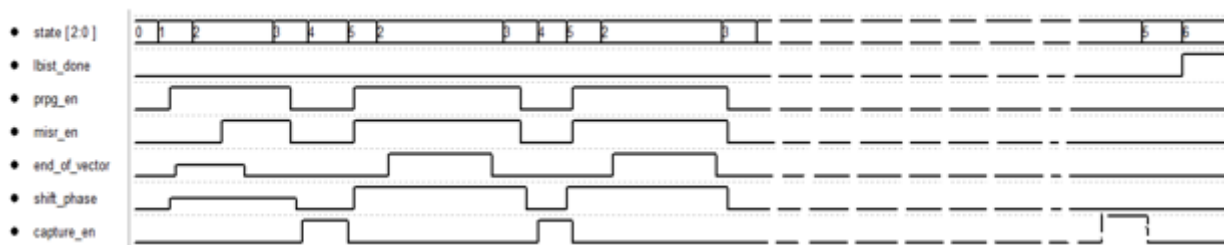
The state register in the FSM determines the state of the FSM. During LogicBIST, the FSM begins in the IDLE state and changes states from IDLE through CAPTURE_PAUSE. If there are pending patterns in the CAPTURE phase, the state returns to SHIFT. These iterations continue until the LogicBIST test is done.

Value of State Register	State Name	Description
0	IDLE	LogicBIST tests are not run in this state. The controller is at rest.
1	INIT	The FSM prepares the LogicBIST controller to start shift procedure.
2	SHIFT	Start shift procedure as part of LogicBIST. Test vectors are applied.
3	SHIFT_PAUSE	Shift procedure comes to a stop to prepare for capturing the responses.
4	CAPTURE	Once the test inputs are applied, the system captures the response stimulus.

Value of State Register	State Name	Description
5	CAPTURE_PAUSE	The captured response and the MISR are triggered to enable signature generation for comparison. If there are more patterns to be applied, the PRPG is enabled and the state returns to SHIFT. When all patterns are generated and responses captured, the state changes to DONE.
6	DONE	Marks the end of the LogicBIST run and what follows is signature recognition and whether it is as expected or not.

The following timing diagram applies to the seven values of the state register:

Figure 9-1. Timing Diagram for the FSM



By default, scan chain masking occurs on a per-chain basis with a 1:1 ratio of chain mask register bits to number of scan chains. Specify per-chain scan masking by loading the chain masking register with the [add_chain_masks](#) command. Per-chain masking is applicable to all patterns in the test set.

For large designs, the area impact of the chain mask register can be more than the controllers themselves. It can also significantly increase the number of setup cycles to start BIST test. Optionally, you can use a single chain mask register bit to mask a group of chains rather than only one chain by specifying the [set_edt_options](#) -chain_mask_register_ratio option or the EDT/Controller/[LogicBistOptions](#)/chain_mask_register_ratio wrapper property.

You can specify the ratio on a per EDT block basis, and the ratios can be different, with some blocks using single scan chain masking and others using shared masking.

When you specify a chain mask register ratio greater than 1, the EDT logic changes so that you have a smaller chain mask register. The ICL, PDL, TCD and patdb files reflect the smaller register size. Converting masking information from individual scan chains to masking bit groups occurs automatically during LogicBIST fault simulation and EDT pattern generation.

New LogicBIST Control Signals

The new LogicBIST mode control signals are added to the EDT IP when using the hybrid hardware.

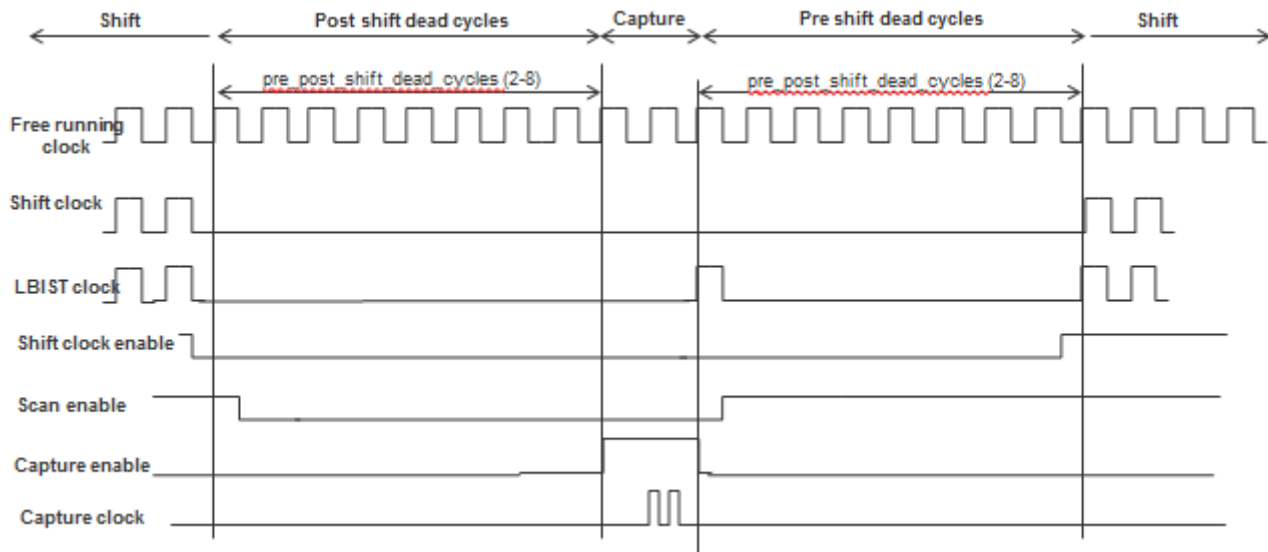
The *lbist_en* control signal determines whether the hybrid EDT operates in EDT mode (*lbist_en*=0) or LogicBIST mode (*lbist_en*=1). There are several other LogicBIST control signals like *lbist_reset*, *lbist_prpg_en*, and *lbist_misr_en* added to the Hybrid IP, which are driven by the common LogicBIST controller. These control signals have no impact during EDT mode of operation, and they are driven appropriately during LogicBIST mode by the LogicBIST controller.

Clocking

The input clock to the LogicBIST controller is a free-running clock, which could be a fast PLL output clock.

The following figure presents a timing diagram for EDT/LogicBIST IP.

Figure 9-2. Timing Diagram for LogicBIST



To allow for the control signals to change between shift and capture, 8 empty (pause) cycles are introduced in between transitions, which helps with timing closure of the test logic.

- *SE* is a scan enable signal that goes to all the scan flops in the design. It transitions half-way through the pause states.
- *shift_clock_en* is a gating signal that could be gated with the free-running input clock to generate the shift-clock for the scan cells. This gating logic can either be implemented by the tool (when using [set_clock_controller_pins](#) *shift_clock*), or you can generate it as part of your clocking logic (when using [set_clock_controller_pins](#) *shift_clock_en*).
- The capture enable signal indicates the start of the capture cycle. This is intended as a trigger for the logic that generates programmable capture sequences. This signal is connected to the scan enable pin of your clock controller.
- The capture clock signal is shown here for illustration purposes only; this signal is generated inside your clock controller.
- The *BIST* clock signal is supplied to all hybrid EDT/LogicBIST blocks as well as used internally by the BIST controller. This clock is pulsed during shift and OFF during capture. It also has a pulse during capture pause state to operate the low-power BIST logic in the hybrid IP as well as to reset certain registers for each pattern in the BIST controller.

You specify the clock controller signals using the [set_clock_controller_pins](#) command.

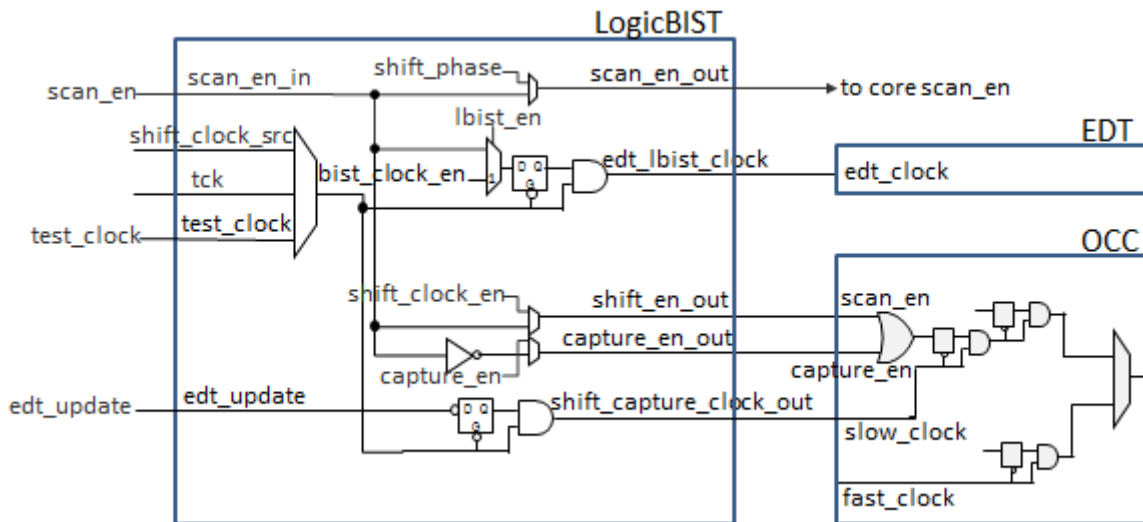
As stated previously, the LogicBIST clock is typically a free-running clock, but the EDT clock should be controllable during test as it is pulsed during **load_unload** but held at off-state during capture. When the test clock is a top-level clock, it can be shared for both EDT and LogicBIST modes. When an internal clock, such as a free-running output of a PLL, is to be used for

LogicBIST, then separate clock sources are required for EDT and LogicBIST modes, where the EDT mode clock is still controllable during test. An example of such a configuration is when shifting during LogicBIST is to be done at a higher speed than during EDT mode. *TCK* is used for seeding of specific values in the LogicBIST registers.

When internally generated functional clocks are used in the design, a top-level shift clock is required for shifting in EDT mode. Typically, a clock controller is used to generate the exact sequence of capture clocks and also to switch between shift-mode and capture-mode clocks. The clock controller takes a free-running clock, shift clock, and scan enable as inputs.

When you are using Tessent OCCs and have specified the `add_dft_signals edt_clock -create_from_other_signals` command, the clocking hardware after LogicBIST and EDT insertion looks as follows.

Figure 9-3. Hybrid TK/LBIST Clocking



Programmable Registers Inside Hybrid IP

Some of the registers inside the per-block hybrid IP are programmable.

The following registers are programmable:

- PRPG
- Low-power control registers (toggle, hold, switching, and mask shift)
- Chain mask register
- MISR

The following registers inside the top-level BIST controller are programmable.

- Capture phase size

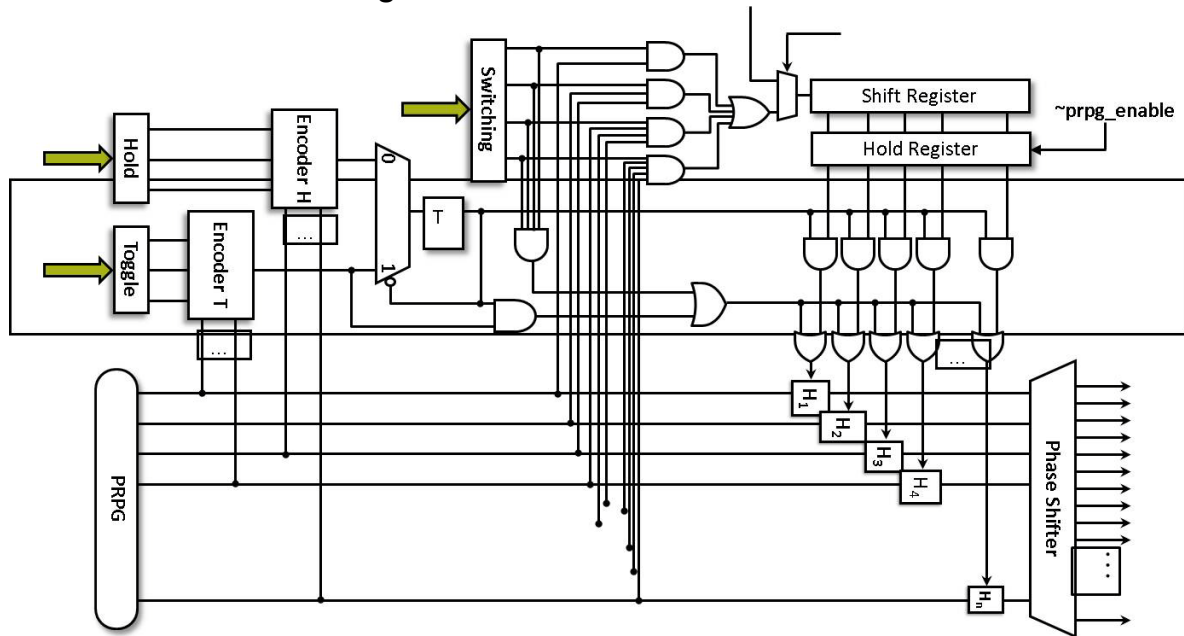
- Shift clock select
- Scan length counter (bit or byte counter depending on how you specify the `set_lbist_controller_options -shift_counter_resolution` command)
- Pattern counter (vector counter)
- BIST enable signal registers

Low-Power Shift Controller

The low-power scheme controls the switching activity during “shift” to reduce power consumption.

The following figure shows the overall architecture of the low-power shift controller. The existing Linear Feedback Shift Machine (LFSM) lockup cells are replaced by a hold register (denoted by **Hi**) which load select signal is controlled by the low-power control logic. By controlling when these hold registers update versus when they hold previous cycle values, the overall switching at the scan chain inputs can be controlled.

Figure 9-4. Low-Power Controller



The performance of the Low-Power BIST controller depends on the following factors:

- The switching code (SC)
- The Hold value (HV)
- The Toggle value (TV)

The SC, HV, and TV values are automatically calculated by the tool based on the switching threshold you set using the [set_power_control](#) command during fault simulation.

The 4-bit switching code might assume one of 15 different binary values ranging from *0001* up to *1111*. The last value can be alternatively used to disable the control register and enter the poorly pseudo-random test pattern generation (unless you activate the Hold mode). All codes are used to enable certain combinations of AND gates forming biasing logic, and, hence, to produce *1*s with probabilities *0.5 (0001)*, *0.25 (0010)*, *0.125 (0100)*, *0.0625 (1000)*, plus their combinations obtained due to an additional OR gate. The resultant *0*s and *1*s are shifted into the mask shift register, and, subsequently, they are reloaded to the mask hold register at the beginning of a test pattern to enable/disable the hold latches placed between the ring generator and its phase shifter.

The duration of how long the entire generator remains in the Hold mode with all latches temporarily disabled regardless of the hold register content.

In the Toggle mode (its duration is determined by TV), the latches enabled through the control register can pass test data moving from the ring generator to the scan chains. In order to switch between these two modes, a weighted pseudo-random signal is produced by a module Encoder H/T based on the content of different stages of the ring generator.

Chapter 10

Tessent OCC for Hybrid TK/LBIST

The Tessent On-Chip Clock Controller (OCC) can be used in the Hybrid TK/LBIST flow. Tessent Shell can generate and insert the Tessent OCCs with programmable capture clock sequences for use with Hybrid TK/LBIST applications.

During LBIST mode, the actual clock sequence is parallel loaded into the Tessent OCC. You can configure the Tessent OCC so that the values can be loaded through OCC module input ports or through a TDR inside the OCC. When an LBIST test uses only one NCP at a time, this value can be loaded through the TDR or be available as constants at the module inputs. If the LBIST test uses multiple NCPs, then the parallel load clock sequence has to be generated for the currently-active NCP, the index for which is provided by the LBIST controller.

Refer also to “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual* for additional Tessent OCC-specific information.

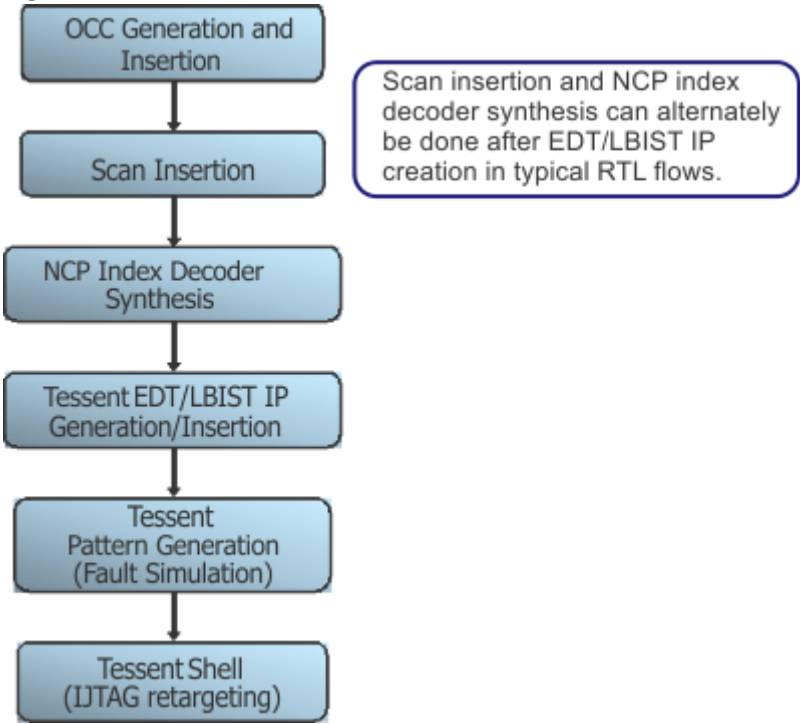
Tessent OCC TK/LBIST Flow	168
Tessent OCC for TK/LBIST Flow Configuration	169
NCP Index Decoder	169
OCC Generation and Insertion	171
Scan Insertion	173
OCC EDT/LBIST IP Creation	173
NCP Index Decoder Synthesis	175
Fault Simulation with a Tessent OCC	175
Pattern Generation with a Tessent OCC	176
Example Tessent OCC TK/LBIST Flow	177
Generating and Inserting the Tessent OCC	177
Inserting the Scan Chains	178
Generating the Hybrid TK/LBIST IP	179
Synthesizing and Inserting the LBIST NCP Index Decoder	181
Generating the EDT Patterns	182
Performing the LBIST Fault Simulation	183
Tessent OCC Examples	185

Tessent OCC TK/LBIST Flow

The Tessent OCC TK/LBIST flow varies from the standard TK/LBIST flow. Use the information in this section as a guide to configure and insert the Tessent OCC, and interface the Tessent OCC with the hybrid TK/LBIST flow controller.

In general, you use the standard TK/LBIST [Bottom-Up Flow](#) with the modifications specific to the TK/LBIST flow as illustrated in [Figure 10-1](#). See also [Example Tessent OCC TK/LBIST Flow](#) for a step-by-step illustration of the Tessent OCC TK/LBIST flow.

Figure 10-1. Modified TK/LBIST Flow for Tessent OCC



Tessent OCC for TK/LBIST Flow Configuration	169
NCP Index Decoder	169
OCC Generation and Insertion	171
Scan Insertion	173
OCC EDT/LBIST IP Creation	173
NCP Index Decoder Synthesis	175
Fault Simulation with a Tessent OCC	175
Pattern Generation with a Tessent OCC	176

Tessent OCC for TK/LBIST Flow Configuration

The clock controller can be used in two modes for LBIST: using a static sequence loaded through an ICL network for all patterns in a session; or, using a set of sequences that are cycled through every 256 patterns.

A Tessent OCC configured with *capture_en* as the capture trigger is used for hybrid TK/LBIST flow. The slow clock input of the Tessent OCC is connected to TK/LBIST reference input clock in LBIST mode. The scan enable input of the Tessent OCC is connected to LBIST shift enable output in LBIST mode. The capture enable input of the Tessent OCC is connected to the LBIST capture enable output in LBIST mode. The tool adds muxes inside the LBIST controller to choose between LBIST and ATPG mode signals.

TCD for the Clock Controller

The tool writes out a Tessent Core Description file after you execute the `process_dft_specification` command when processing the clock controller wrapper. This file is written out in the instrument directory for the clock controller with *.tcd_occ* filename extension. This file is for input into the Tessent Scan insertion tool.

NCP Index Decoder

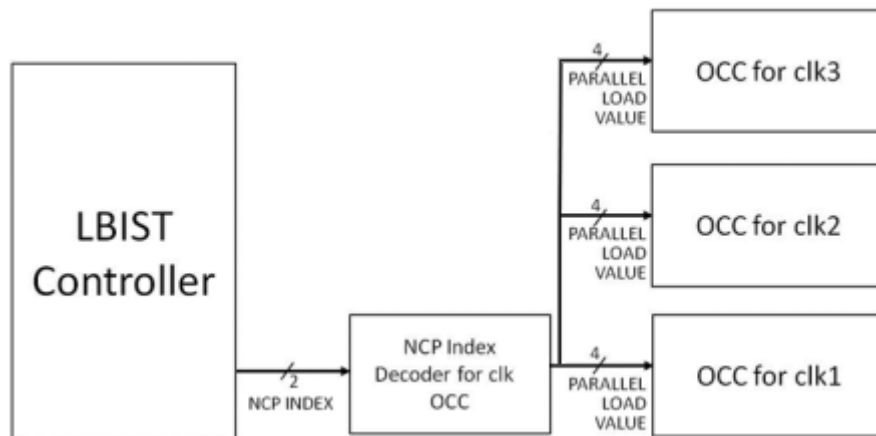
The NCP index decoder is a simple combinational logic block that decodes the NCP index output of the Hybrid TK/LBIST controller into clock sequences to be generated by the Tessent OCCs across all capture procedures.

The LBIST controller generates an NCP index that cycles through the NCPs based on the active percentage for each NCP. This NCP index is decoded to provide the actual clock sequence that is parallel loaded to the OCC.

You must specify the clocking combinations to be used during TK/LBIST test. The tool synthesizes the NCP index decoder and generates named capture procedures based on this description.

One NCP index decoder is synthesized for each LogicBIST controller and can be used for controlling all the OCCs involved in LogicBIST. In the following example, there are three OCCs configured for four cycles each. Since there are three OCCs, you can expect two input binary values to the NCP index decoder, which will be encoded as a single control signal per OCC per cycle.

Figure 10-2. NCP Index Decoder Connections



Only one index decoder is generated for all OCCs. The NCP index decoder is instantiated by default at the top-level, or as controlled with `parent_instance` property of the [NcpIndexDecoder](#) specification—see “[NCP Index Decoder Creation](#)” on page 170.

The NCPs loaded and enabled in memory when you issue [write_edt_files](#) command in Tessent Shell. The [create_capture_procedures](#) command can be used to create new NCPs; this can be used in addition to NCPs present in the input test procedure files.

When using the NCP decoder, you specify this using the “-capture_procedure count” switch to the [set_lbist_controller_options](#) command.

NCP Index Decoder Creation

Specify the NCP index decoder using the `NcpIndexDecoder` wrapper `DftSpecification/LogicBist` wrapper. See the [NcpIndexDecoder](#) wrapper in the *Tessent Shell Reference Manual* for a complete wrapper description and usage.

When you are using only one NCP, you cannot use the `NcpIndexDecoder` wrapper because it is supported only for external static clock controls and two or more NCPs. Refer to “[Considerations When Only Using One NCP](#)” on page 175 for fault simulation considerations.

Note

 NCP index decoder generation requires an elaborated design and Tessent OCC instances you have added with the [add_core_instances](#) command.

Examples

In the following example, assume the design has two top-level Tessent OCC instances named `m8051_gate_tessent_occ_clk1_inst` and `m8051_gate_tessent_occ_clk2_inst` of the same Tessent OCC module `m8051_gate_tessent_occ`.

```

LogicBist
  NcpIndexDecoder {
    Connections {
      NcpIndex: m8051_lbist_i/ncp;
    }
    Ncp(stuck) {
      cycle(0): m8051_gate_tessent_occ;
      //Specified using module name, refers to both the OCC instances.
    }
    Ncp(clk1_double_pulse) {
      cycle(0): m8051_gate_tessent_occ_clk1_inst;
      cycle(1): m8051_gate_tessent_occ_clk1_inst;
    }
    Ncp(clk2_double_pulse) {
      cycle(0): m8051_gate_tessent_occ_clk2_inst;
      //Note - cycle(1) is omitted, so no clock activity
      cycle(2): m8051_gate_tessent_occ_clk2_inst;
    }
  }
}

```

Assuming the above specification is in a file named *ncp.dft_spec*, the following Tessent Shell dofile synthesizes and inserts the NCP index decoder.

```

set_context dft -no_rtl
read_verilog \
  tsdb_outdir/dft_inserted_designs/m8051_gates.dft_inserted_design/
m8051.vg
read_verilog tsdb_outdir/instruments/m8051_gates_occ.instrument/
m8051_gate_tessent_occ.vg
read_cell_library atpg.lib
set_current_design
read_core_description \
  tsdb_outdir/instruments/m8051_gates_occ.instrument/
m8051_gate_tessent_occ.tcd
add_core_instances -module m8051_gates_tessent_occ
report_core_instances
read_config_data ncp.dft_spec
set_design_level physical_block
process_dft_specification

```

OCC Generation and Insertion

You create an OCC-specific DftSpecification to interface the hybrid controller to the OCC. Then you use Tessent Shell to process this specification.

You define the [OCC DftSpecification](#) using [DftSpecification configuration syntax](#). A separate OCC DftSpecification with controller wrappers should be added for each different clock that needs to be programmable during capture. Additionally, OCCs may be added for asynchronous reset signals declared as a clock.

Tessent Shell creates Verilog RTL, ICL, PDL, TCD files, and TCD scan describing the Tessent OCC instrument, as well as a Verilog netlist that instantiates the OCC in the user design. Many generation and insertion options are available in the Dft Specification to control this process. The ICL and TCD outputs are used in later steps like EDT/LBIST IP creation to describe the

configuration of the generated OCCs as well identifying the port functions. The Tessent OCC RTL should be synthesized to a gate level design before it is used for downstream steps that require a gate level netlist.

The Tessent OCC can be inserted in a design either at RTL or gate level. When inserted at RTL level or before EDT IP, the Tessent OCC shift registers can either be merged with design scan cells or stitched up into dedicated Tessent OCC scan chains. When using stitched into dedicated chains, these can be either compressed or uncompressed. When OCC is inserted after EDT IP, the OCC shift registers have to be stitched into dedicated OCC scan chains and handled as uncompressed chains driven directly by the tester.

Static Clock Control

By default, the Tessent OCC generated by Tessent Shell provides programmability only through the clock control shift register suitable for ATPG. To use the OCC on a LBIST design, you should add static clock control. Static clock control, which refers to clock sequence not decided by ATPG, can be one of the following options:

- **Internal** — The Tessent OCC is statically programmable using an internal TDR for both LBIST and ATPG modes. When using this option, the LBIST test can use only one NCP at a time. When multiple NCPs are to be used, it needs to be done in multiple pattern sets.
- **External** — The Tessent OCC is statically programmable through OCC module ports for the LBIST mode. This allows use of multiple NCPs for LBIST test in a single pattern set. An NCP index decoder is synthesized to provide the clock sequence for the different NCPs based on the `ncp_index` output from the LBIST controller. The Tessent OCC external clock control module port is available only for the LBIST mode and unavailable for ATPG.
- **Both** — This combines both the internal and external options described above. ATPG can use the TDR for static clock control. LBIST can use either the TDR or the OCC module ports.

Capture Trigger

To use the Tessent OCC for TK/LBIST operation, you should set the capture trigger to capture enable. In this case, scan enable is replaced by the LBIST capture enable signal as the trigger. To allow either fast capture or slow capture to be used during LBIST, the slow clock signal is connected to the free running LBIST controller input clock so that it pulses on all capture cycles.

The capture enable signal should to be tied to constant-0 or connected to inverted scan enable during OCC insertion.

Connection to OCC External Clock Ports

By default, when configuring the Tessent OCC with `static_clock_control` as either “external” or “both”, the tool ties the OCC external clock control module ports to constant-0. These ports are not described in the OCC ICL and are not referenced in the OCC setup iProc. In an LBIST application with multiple NCPs, these OCC pins would be eventually connected to the NCP index decoder.

Scan Insertion

During Tessent OCC insertion, the clock control shift register IO of the Tessent OCCs are left unconnected. Integrating these Tessent OCC shift register sub-chains into the design is performed outside of the Dft Specification. Scan insertion can be performed using third-party tools.

You can insert the Tessent OCC into either a non-scan design or a scan design.

Non-Scan Design Scan Insertion

When the Tessent OCC is inserted in a non-scan design, the OCC shift registers should be declared as sub chains to the scan insertion tool. The scan insertion tool will merge these sub-chains with other scan cells taking care of scan chain length balancing.

Scan Design Insertion

When the Tessent OCC is inserted in a scan design, the Tessent OCC shift registers should be connected into one or more dedicated scan chains by the user, based on the target scan chain length. The Tessent OCC scan enable pin is connected to the design scan enable during scan insertion. The length of the sub chains should be the length of the Tessent OCC shift register. The OCC sub chains are always made part of scan chains considering the EDT mode in contrast to LBIST mode where the OCC shift register is bypassed by a single flop. The sub chains are internally bypassed within the OCC during LBIST.

OCC EDT/LBIST IP Creation

The method you use for Tessent OCC EDT/LBIST creation depends on which flow you are using: pre-synthesis or gate level.

Pre-Synthesis Flow

In pre-synthesis flow, the Tessent OCC is not present in the input skeleton design and the ICL/TCD files for the OCC cannot be read during IP creation. You should instruct the tool to generate LBIST controller compatible with Tessent OCCs. When using this flow, you are responsible for making all connections between the LBIST controller, EDT blocks, NCP index decoder and OCCs. You do this by using the “[set_lbist_controller_options](#) -tessent_occ on” command and options.

Gate-Level Flow

During EDT IP creation, you should input into Tessent Shell the Tessent OCC inserted design with OCC shift registers included in scan chains should be read. The OCC scan chains can be either part of compressed or uncompressed chains. Do not add Tessent OCC uncompressed scan chains during IP creation; only add them during pattern generation. You must configure the Tessent OCC correctly to pass the IP creation DRC checks, specifically the shift clock, scan enable, and capture enable signals of the Tessent OCC are properly connected and operated in the incoming test procedures.

During IP creation, you read in the ICL, PDL and Tessent Core Description (TCD) for the OCC. This is required to properly setup the Tessent OCC during IP creation. The TCD description is bound to a netlist instance by treating it as a core instance, similar to how scan pattern retargeting uses TCD. The tool identifies the Tessent OCCs when the `tessent_instrument_type` ICL attribute is set to “mentor::occ”. This attribute value is considered when generating the ICL signature, so it cannot be added to user OCCs. When Tessent OCCs are present in the design, the LBIST controller is modified to correctly interface with the OCC.

The TK/LBIST compatible Tessent OCCs are required to have the following two features: capture trigger using capture enable and static clock control either external or both when using multiple NCPs. See “[Static Clock Control](#)” on page 172 and “[Capture Trigger](#)” on page 172.

Note



Do not mix Tessent OCCs and custom OCCs (defined using [set_clock_controller_pins](#) command) in the same LBIST controller. The tool performs rule checks to validate this requirement.

During EDT IP creation, the Tessent OCC capture enable pins that are not functionally driven are driven by the inverted OCC scan enable. The Tessent OCC capture enable pins that are functionally driven (that is, by inverted scan enable) are multiplexed between existing functional connection and LBIST capture enable.

Capture Procedures

During IP creation, you specify the total number of NCPs used for LBIST.

This is required to synthesize the NCP index output and NCP activity percentage registers. If the exact number of NCPs is not known during IP creation, an upper bound can be used. During fault simulation, unused NCP indices can be specified as 0%. If the names and activity percentage of the NCPs is specified during IP creation, this will be used for the hardware default mode. When not specified, the tool defaults to equal activity for all the NCPs for the hardware default mode.

NCP Index Decoder Synthesis

The NCP index decoder is synthesized is normally done with a third-party synthesis tool.

The number of NCPs generated should be equal to or less than the number specified during hybrid TK/LBIST IP generation.

Fault Simulation with a Tessent OCC

Fault simulation with the Tessent OCCs is similar to a flow that uses custom OCCs. Tessent Shell will read in the complete gate level Verilog netlist with EDT and OCC, ICL/PDL, and / TCD files for EDT and OCC instruments.

Tessent OCC core instances can be added during fault simulation.

Tessent Shell automatically adds two internal clocks for each OCC instance:

- A pulse in capture clock at the output of shift register clock gater (cgc_SHIFT_REG_CLK/clkg).
- A programmable capture clock at the output of the programmable clock gater (cgc_CLK_OUT/clkg) at the output of the mux [tessent_persistent_cell_clock_out_mux/Z].

Provide NCPs that refer to these internal clocks. If you have already added internal clocks in the same location as the tool would, then Tessent Shell tool recognizes this and does not add duplicate clocks. The Design Rule Check [R18](#), which validates whether OCC control register is part of scan chains, is bypassed in LBIST mode. Clock control definitions are not generated in the LBIST mode. The output core description TCD file will include the parameters that were specified for the OCC instruments used in this run.

In the LBIST mode dofile, an internal user-PI is added for the LBIST capture enable signal and constrained to 1. For fault simulating chain test patterns, you manually change this constraint to 0.

Considerations When Only Using One NCP

When you are using only one NCP, you must manually create the NCP description because the NcpIndexDecoder, which usually creates the named capture procedure description, cannot be used for a single NCP. the OCC can have either an external or internal static clock control. During fault simulation, specify the name of the NCP as you would when you have more than one NCP.

For the internal static clock control, load the clock sequence corresponding to the user-created NCP through the ICL network. For Tessent OCCs, you can do this by using the clock_sequence core instance parameter. For the external static clock control, connect the Tessent OCC's clock sequence pins to constant values that generate the desired NCP clock waveforms.

Pattern Generation with a Tessent OCC

Pattern generation is performed through pattern specification.

The presence of Tessent OCC is identified through the ICL attribute on the OCC modules and is matched with TCD description for the OCC. When using Tessent OCCs, the pattern specification processing will automatically call the OCC setup iProc with the parameter values that were used during fault simulation.

Example Tessent OCC TK/LBIST Flow

In the OCC flow, you insert Tessent OCC and scan chains before generating the hybrid IP. Unlike the non-OCC hybrid flow, you also need to insert the LogicBIST NCP index decoder before generating your patterns and performing fault simulation.

This example demonstrates a gate-level flow using DFTSpecification for the OCC instertion. The flow uses the TSDB (Tessent Shell Database).

Generating and Inserting the Tessent OCC	177
Inserting the Scan Chains	178
Generating the Hybrid TK/LBIST IP	179
Synthesizing and Inserting the LBIST NCP Index Decoder	181
Generating the EDT Patterns	182
Performing the LBIST Fault Simulation.....	183

Generating and Inserting the Tessent OCC

The first step to using a Tessent OCC in the TK/LBIST flow is creating the OCC, then subsequently insert the Tessent OCC into your design.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to “dft” and specify a design identifier (gate1) for the current design.

```
SETUP> set_context dft -no_rtl -design_identifier gate1
```

3. Read in the design netlist. For example:

```
SETUP> read_verilog m8051_nonscan.v
```

4. Read the cell library. For example:

```
SETUP> read_cell_library atpg.lib
```

5. Specify the top-level module of the current design. For example:

```
SETUP> set_current_design
```

6. Set the design level. The physical_block level indicates the design is a block that is synthesized and laid out as an independent block. For example:

```
SETUP> set_design_level physical_block
```

7. Read the OCC specific configuration data. For example:.

```
ANALYSIS> read_config_data-from_string {
  DftSpecification(m8051, gate1) {
    reuse_modules_when_possible: on;
    occ {
      capture_trigger: capture_en;
      static_clock_control: external;
      Controller(clk) {
        clock_intercept_node: clk;
      }
    }
  }
}
```

8. Validate and process the content defined in the DftSpecification wrapper.

```
ANALYSIS> process_dft_specification
```

9. Synthesize the generated RTL.

```
ANALYSIS> run_synthesis
```

10. Exit the tool.

```
ANALYSIS> exit
```

Results

After OCC insertion, the created hardware is synthesized and replaces the inserted RTL modules with gate-level modules in the Tessent Shell Data Base (TSDB).

Inserting the Scan Chains

During scan insertion, the top level shift clock reaches all the design scan cells, hence the entire design is treated as a single clock domain design.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to 'dft' and the design identifier, in this case to gate2.

```
SETUP> set_context dft -scan -design_identifier gate2
```

3. Reload the design created in the OCC insertion. For example:

```
SETUP> read_design m8051 -design_identifier gate1
```

4. Read the cell library. For example:

```
SETUP> read_cell_library atpg.lib
```

5. Set the current design. For example:

```
SETUP> set_current_design
```

6. Add signal constraints. For example:

```
SETUP> add_input_constraint RST -c0
```

7. The OCCs are automatically marked as hard macros.

8. Go to analysis mode.

```
SETUP> set_system_mode analysis
```

9. Perform X-bounding analysis to identify memory elements that might capture an unknown during Logic BIST. For example:

```
ANALYSIS> analyze_xbounding
```

10. Specify the scan chain mode. In this case the scan chain mode is edt and the chain count is 16. For example:

```
ANALYSIS> add_scan_mode edt -chain_count 16
```

11. Run scan chain analysis to distribute the scan elements into new chains. For example:

```
ANALYSIS> analyze_scan_chains
```

12. Insert the test structures in to the netlist and stitch up the scan chains. For example:

```
ANALYSIS> insert_test_logic
```

13. Write the test procedure file and dofile that describe the chains created during scan insertion. For example

```
ANALYSIS> write_atpg_setup -replace
```

14. Exit the tool.

```
SETUP> exit
```

Generating the Hybrid TK/LBIST IP

To generate the hybrid TK/LBIST IP, you read in the scan-inserted design with the Tessent OCC chains integrated with design scan cells. The Tessent OCC is thus part of compressed chains driven by the EDT decompressor.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to 'dft -edt -logic_bist'. Specify the design identifier for, in this case it is gate3. For example:

```
SETUP> set_context dft -edt -logic_bist -design_identifier gate3
```

3. Reload the design created during scan insertion. For example:

```
SETUP> read_design m8051 -design_identifier gate2
```

4. Read the cell library. For example:

```
SETUP> read_cell_library atpg.lib
```

5. Set the current design. For example:

```
SETUP> set_current_design
```

6. Add the core instances. For example:

```
SETUP> add_core_instances -module m8051_gate1_tessent_occ
```

7. Setup up scan and constrain the inputs as required. The dofile and procedure were created by the write_atpg_setup command. For example:

```
SETUP> dofile scan_setup.dofile  
SETUP> tessent_scan_setup edt
```

8. Setup the EDT IP. For example:

```
SETUP> set_edt_options -location internal -channels 1
```

9. Setup the LBIST controller. For example:

```
SETUP> set_lbist_controller_options -max_shift 100 -max_capture 3 \  
-max_pattern 100000  
SETUP> set_lbist_controller_options -capture_procedures 2  
SETUP> set_lbist_pins clock REFCLK  
SETUP> set_lbist_pins scan_en scan_en  
SETUP> set_lbist_pins xbounding_en lbist_en  
SETUP> set_clock_controller_pins capture_procedure_index \  
-no_connection
```

10. Go to analysis mode.

```
SETUP> set_system_mode analysis
```

11. Write the IP into the Tessent Shell Data Base (TSDB). For example:

```
ANALYSIS> write_edt_files -tsdb -replace
```

12. Run synthesis. For example:

```
ANALYSIS> run_synthesis
```

13. Exit the tool.

```
SETUP> exit
```

Synthesizing and Inserting the LBIST NCP Index Decoder

You can use Tessent Shell to read the synthesized gate-level netlist and identify the clock usage in the design.

The “[report_clock_domains](#) –compatible_clocks –details” command provides information that can be used to design effective NCP sequences for LogicBIST.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to ‘dft’.

```
SETUP> set_context dft -no_rtl -design_identifier gate4
```

3. Read in the design. For example:

```
SETUP> read_design m8051 -design_identifier gate4
```

4. Read the cell library. For example:

```
SETUP> read_cell_library atpg.lib
```

5. Set the current design. For example:

```
SETUP> set_current_design
```

6. Set the design level. For example:

```
SETUP> set_design_level physical_block
```

7. Add the OCC instances. For example:

```
SETUP> add_core_instances -module m8051_gate1_tessent_occ
```

8. Read in the NCP index decoder specification. For example:

```
SETUP> read_config_data -from_string {
  DftSpecification(m8051, gate4) {
    LogicBist {
      NcpIndexDecoder {
        Ncp(pulse_once) {
          cycle(0): m8051_gate1_tessent_occ;
        }
        Ncp(pulse_twice) {
          cycle(0): m8051_gate1_tessent_occ;
          cycle(1): m8051_gate1_tessent_occ;
        }
      }
    }
  }
}
```

9. Process the NCP index decoder specification. For example:

```
SETUP> process_dft_specification
```

10. Setup and run synthesis. For example:

```
SETUP> run_synthesis
```

11. Exit the tool.

```
SETUP> exit
```

Generating the EDT Patterns

To switch between capture modes, you only need to change the OCC setup iProc parameters.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to 'patterns -scan'.

```
SETUP> set_context patterns -scan -design_identifier gate4
```

3. Reload the EDT-inserted design. For example:

```
SETUP> read_design m8051
```

4. Read the cell library. For example:

```
SETUP> read_cell_library atpg.lib
```

5. Set the current design. For example:

```
SETUP> set_current_design
```

6. Extract the ICL. For example:

```
SETUP> extract_icl
```

7. Add signal constraints. For example:

```
SETUP> add_input_constraint RST -c0
```

8. Add the core instances including the EDT core. For example:

```
SETUP> add_core_instances -module m8051_gate1_tessent_occ \  
-param {fast_capture_mode 1}  
SETUP> add_core_instances -module m8051_gate3_tessent_edt_lbist
```

9. Set the fault type. For example:

```
SETUP> set_fault_type transition
```

10. Set other parameters as required by your design style. For example:

```
SETUP> set_output_masks on  
SETUP> add_input_constraints -all -hold  
SETUP> set_pattern_type -sequential 2
```

11. Change the system mode to analysis to run DRCs.

```
SETUP> set_system_mode analysis
```

12. Read in the procedure file and setup the capture options. For example:

```
ANALYSIS> read_procf file external_capture_options \  
external_capture.testproc  
ANALYSIS> set_external_capture_options -capture_procedure \  
ext_fast_cap_proc
```

13. Create and write out the patterns. For example:

```
ANALYSIS> create_patterns  
ANALYSIS> write_patterns edt_patt_fast_capture.v -verilog \  
-serial -replace
```

14. Exit the tool.

```
ANALYSIS> exit
```

Performing the LBIST Fault Simulation

The final phase of the hybrid flow for OCC is to use Tessent Shell to perform the LBIST fault simulation.

Procedure

1. Invoke Tessent Shell from the shell prompt.

```
% tessent -shell
```

2. Set the Tessent Shell context to 'patterns -scan'.

```
SETUP> set_context patterns -scan -design_identifier gate4
```

3. Reload the design. For example:

```
SETUP> read_design m8051
```

4. Read cell library to allow creation of some instances such as muxes. For example:

```
SETUP> read_cell_library ../data/atpg.lib
```

5. Set the current design. For example:

```
SETUP> set_current_design
```

6. Extract the ICL. For example:

```
SETUP> extract_icl
```

7. Add signal constraints. For example:

```
SETUP> add_input_constraints RST -c0
```

8. Add the core instances. For example:

```
SETUP> add_core_instances -module m8051_gate1_tessent_occ  
add_core_instances -module m8051_gate3_tessent_edt_lbist  
add_core_instances -module m8051_gate3_tessent_lbist
```

9. Set the LBIST controller options. For example:

```
SETUP>dofile tsdb_outdir/instruments/  
m8051_gate4_lbist_ncp_index_decoder.instrument/  
m8051_gate4_tessent_lbist_ncp_index_decoder.dofile  
SETUP> set_lbist_controller_options -capture_procedure \  
{pulse_once 100 pulse_twice 0}
```

10. Change the system mode to analysis to run DRCs.

```
SETUP> set_system_mode analysis
```

11. Read in the NCP index decoder test procedure file for the TSDB instruments directory using the [read_procfile](#) command. For example:

```
read_procfile tsdb_outdir/instruments/  
m8051_gate4_lbist_ncp_index_decoder.instrument/  
m8051_gate4_tessent_lbist_ncp_index_decoder.testproc
```

12. Set capture options. For example:

```
ANALYSIS> set_external_capture_options -fixed 4
```

13. Add faults. For example:

```
ANALYSIS> add_faults
```


14. Specify the pattern count and add the faults. For example:

```
ANALYSIS> set_random_patterns 100
```

15. Perform the LBIST simulation.

```
ANALYSIS> simulate_patterns -source bist -store_patterns all
```

16. Exit the tool.

```
ANALYSIS> exit
```

Tessent OCC Examples

The examples in this section illustrate usage models for using a Tessent OCC in the hybrid TK/LBIST flow.

Clock Domain Analysis for NCP Decoder Generation

This example shows how to use Tessent Shell to perform clock domain analysis and identify NCPs required for LBIST test. It uses a small design that has the following clocks:

- NX1
- NX2
- NX3

The following dofile reports the clock domains and the percentage of faults in each of the domains. Since the netlist is non-scan, the dofile instructs Tessent Shell to treat the netlist as a full-scan design, using the “add scan groups dummy dummy” command. If the design were already scan-inserted, you would instead specify the actual test procedure file and scan chains.

```
set_context pattern -scan
read_verilog nonscan_netlist.v
read_cell_library atpg.lib
set_current_design
add_scan_groups dummy dummy
add_clock 0 NX1
add_clock 0 NX2
add_clock 0 NX3
set_system mode analysis
add_faults -all
report_statistics -clock_domains summary
report_clock_domains -compatible_clocks -details
```

The “Clock Domain Summary” section of report_statistics command’s output is shown below:

Clock Domain Summary	% faults (total)	Test Coverage (total relevant)
/NX1	22.38%	0.00%
/NX2	74.70%	0.00%
/NX3	0.33%	0.00%

The output of report_clock_domains command is shown below:

```
// No. | Clock Name Domain | Clock Compatibility
// -----+-----+-----
// 1 | '/NX1' (1) 1 | .
// 2 | '/NX2' (2) 2 | 437 .
// 3 | '/NX3' (56) 3 | 8 44 .
// -----+-----+-----
// No. | 1 2 3
// . = Compatible (non-interacting) clock pair
// <number> = Incompatible (interacting) clock pair
// Compatibility analysis is based on same-edge clock interaction.
```

From the above reports, there are small number of interacting flops (8) between NX1 and NX3. The paths between NX1 and NX3 should be bounded, which can be accomplished by an SDC file that describes all paths between these clock domains as false, as shown below (declared at the mux output):

```
create_clock occ_NX1/clock_out -period 40 -name NX1
create_clock occ_NX2/clock_out -period 40 -name NX2
create_clock occ_NX3/clock_out -period 40 -name NX3
set_false_path -from NX1 -to NX3
set_false_path -from NX3 -to NX1
```

The tool can now treat NX1 and NX3 as compatible clock domains and pulse them together, since all interactions between them are blocked during X-bounding. From the prior clock activity table, we can divide the design into two clock domains: NX2 and NX1_NX3. Consequently, the NX2 and NX1_NX3 domains can be tested for 75% and 25% of the test duration, respectively.

During fault simulation, the output of the report_clock_domains command shows that NX1 and NX3 are indeed compatible after X-bounding. The functional clocks referred earlier are numbered 7-9 in the output below.

//No.	Clock Name	Domain	Clock Compatibility								

// 1	'/RST' (1)	1	.								
// 2	'/refclk' (55)	1	.	.							
// 3	'/shift_clock' (56)	1	.	.	.						
// 4	'/tck' (57)	1	.	.	.	.					
// 5	'/edt_clock' (64)	1	.	.	.	.	.				
// 6	'/edt_lbist_int_clock' (73)	1	.	.	.	.	.	.			
// 7	'/NX3' (75)	1	.	.	.	.	.	.	.		
// 8	'/NX2' (76)	2	9	.	.	.	.	.	.	36	.
// 9	'/NX1' (77)	1	.	.	.	.	.	.	.	.	437

//		No.	1	2	3	4	5	6	7	8	9
//	. = Compatible (non-interacting) clock pair										
//	<number> = Incompatible (interacting) clock pair										

A sample DftSpecification for this design is as follows:

```

LbistNcpIndexDecoder {
  LbistNcpIndex: m8051_lbist_i/ncp;
  Ncp(NX1_NX3_single_pulse) {
    cycle(0): occ_NX1, occ_NX3;
  }
  Ncp(NX2_single_pulse) {
    cycle(0): occ_NX2;
  }
  Ncp(NX1_NX3_double_pulse) {
    cycle(0): occ_NX1, occ_NX3;
    cycle(1): occ_NX1, occ_NX3;
  }
  Ncp(NX2_double_pulse) {
    cycle(0): occ_NX2;
    cycle(1): occ_NX2;
  }
}

```

The following NCPs are generated for the above DftSpecification:

```

procedure capture NX1_NX3_single_pulse =
  timeplate gen_tp1;
  cycle =
    force_pi;
    force occ_NX2_clock_out 0;
    force shift_clock 0;
    pulse occ_NX1_clock_out;
    pulse occ_NX3_clock_out;
  end;
end;

```

```
procedure capture NX2_single_pulse =  
  timeplate gen_tpl;  
  cycle =  
    force_pi;  
    force occ_NX1_clock_out 0;  
    force occ_NX3_clock_out 0;  
    force shift_clock 0;  
    pulse occ_NX2_clock_out;  
  end;  
end;  
  
procedure capture NX1_NX3_double_pulse =  
  timeplate gen_tpl;  
  cycle =  
    force_pi;  
    force occ_NX2_clock_out 0;  
    force shift_clock 0;  
    pulse occ_NX1_clock_out;  
    pulse occ_NX3_clock_out;  
  end;  
  cycle =  
    pulse occ_NX1_clock_out;  
    pulse occ_NX3_clock_out;  
  end;  
end;  
  
procedure capture NX2_double_pulse =  
  timeplate gen_tpl;  
  cycle =  
    force_pi;  
    force occ_NX1_clock_out 0;  
    force occ_NX3_clock_out 0;  
    force shift_clock 0;  
    pulse occ_NX2_clock_out;  
  end;  
  cycle =  
    pulse occ_NX2_clock_out;  
  end;  
end;
```

Hybrid Clock Gating When Inserting OCC and DFT Signals in the Same DFT Insertion Pass

This example illustrates clock-gating with the hybrid TK/LBIST DFT insertion flow with OCC. This usage uses the TSDB flow to insert OCC with the `edt_clock` and `shift_capture_clock` DFT signals in the first insertion pass, followed by inserting EDT and LogicBIST in the second insertion pass.

Generate the `edt_clock` and `shift_capture_clock` signals by using the `add_dft_signals -create_from_other_signals` command as shown in the following dofile example.

```
set_context dft -no_rtl -design_identifier dft_signals
set_tsdb_output_directory tsdb_outdir
read_core_descriptions [lsort [glob design/mem/*.lib]]
read_cell_library ../tessent/adk.tcelllib
read_cell_library design/mem/mems.atpglib

read_verilog design/gate/elt1.v
set_current_design elt1
set_design_level physical_block
set_dft_specification_requirements -memory_test off -logic_test on

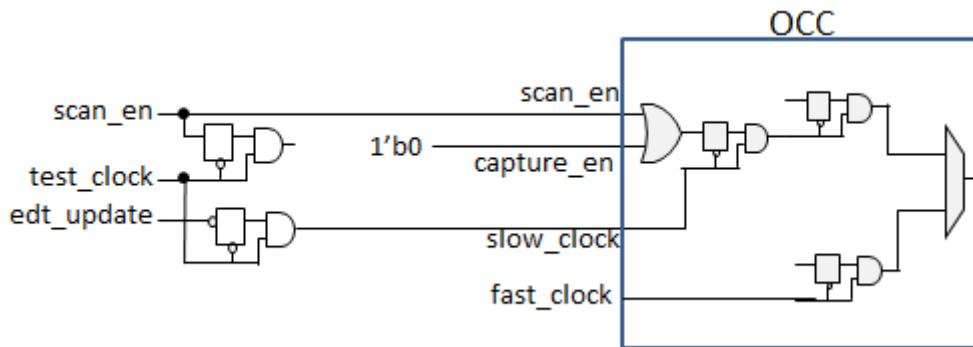
add_dft_signals scan_en test_clock edt_update \
    -source_node {scan_enable test_clock edt_update}
add_dft_signals edt_clock -create_from_other_signals
add_dft_signals shift_capture_clock -create_from_other_signals
report_dft_signals

add_clocks CLK_F300 -period [expr {1000.0/300.0}]

check_design_rules
set_system_mode analysis
set_spec [create_dft_specification -sri_sib_list {occ}]
report_config_data $spec
set_config_value use_rtl_cells on -in_wrapper $spec
read_config_data -in_wrapper $spec -from_string {
    OCC {
        ijttag_host_interface : Sib(occ);
        static_clock_control : external;
        capture_trigger       : capture_en;
        Controller(clk_controller) {
            clock_intercept_node : CLK_F300;
            parent_instance : dft_inst;
        }
    }
}
report_config_data $spec
process_dft_specification
extract_icl
run_synthesis -startup_file ../prerequisites/techlib_adk.tnt/current/synopsys/
synopsys_dc.setup
```

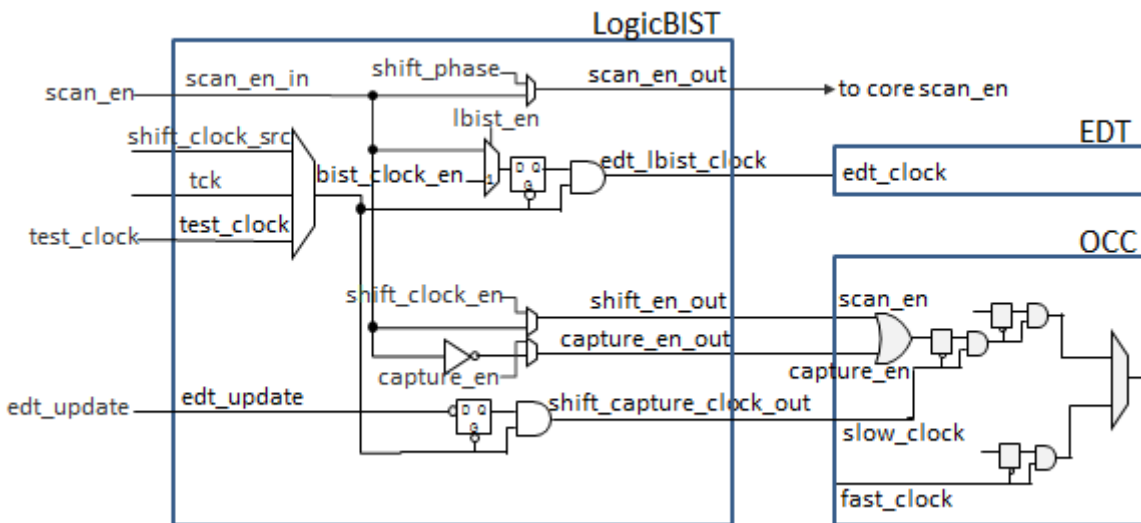
After the first pass, the OCC `slow_clock` is driven by the `shift_capture_clock` gater and the `edt_clock` gater has no fanout.

Figure 10-3. Clock Gating With DFT Signals and OCC in the First Pass



After inserting EDT and LogicBIST in the second hybrid DFT insertion pass, the tool creates the circuit shown below. The edt_clock gater and shift_capture_clock gaters have been removed and their previous connections are now driven by their respective ports on the LogicBIST controller.

Figure 10-4. Clock Gating With EDT and LogicBIST in the Second Pass



Note

This is also the resulting circuit when you first run the TSDB flow to insert the DFT signals and then the dofile flow to insert the LogicBIST controller. In the dofile flow for LogicBIST insertion, the source clock of the DFT signal gaters supply the test_clock to the LogicBIST controller.

Chapter 11

Third-Party OCC for Hybrid TK/LBIST

Within the hybrid TK/LBIST flow, you can use third-party OCCs that have already been inserted into the design. To do so, you must first generate Tessent Core Description (TCD) files for each third-party OCC instrument.

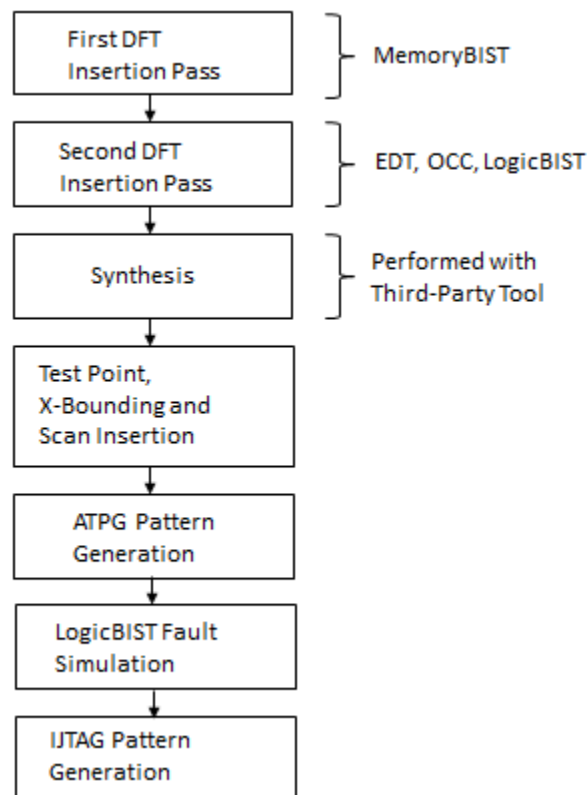
Overview of the Third-Party OCC Flow	191
ThirdPartyOCC TCD File Syntax	193
Usage Example for the ThirdPartyOCC TCD	194

Overview of the Third-Party OCC Flow

After generating OCC TCD files and during hybrid IP insertion, read in the third-party OCCs explicitly using the `read_core_descriptions` command.

As described in “[RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST](#)” in the *Tessent Shell User’s Manual*, the high-level Tessent Shell flow with hybrid TK/LBIST is:

Figure 11-1. Hybrid TK/LBIST Flow With Tessent Shell



During the DFT insertion pass in which you are inserting EDT, OCC, and LogicBIST, specify the third-party OCCs as follows:

```
read_core_descriptions TCD_file_name ...
add_core_instances -module {OCC_module ...}
```

When using third-party OCCs rather than Tessent OCCs, you must:

- Declare how many NCPs are required. In the DftSpecification, include a LogicBist/Controller/NcpOptions wrapper with either the count or percentage_of_patterns_per_ncp property; these properties are mutually exclusive.

```
DftSpecification (module_name,id) {
  LogicBist {
    Controller(id) {
      NcpOptions {
        count : int ;                                // default: 1
        percentage_of_patterns_per_ncp : integer, ... ;
      }
    }
  }
}
```

- Create the decoding logic. This is the responsibility of the user.
- Hook the decoding logic to the NCP output of the LogicBIST controller. This is the responsibility of the user.
- Ensure that the OCC uses the decoded logic to control the clock sequence when the OCC is active and creates NCPs for various clocking sequences.

After inserting the hybrid IP, proceed with the rest of the flow as normal. A mix of Tessent OCC and third-party OCCs is not supported.

ThirdPartyOCC TCD File Syntax

Use the ThirdPartyOCC TCD to define third-party OCCs already inserted in the design.

Usage

```
Core(module_name) {  
  ThirdPartyOcc {  
    Interface {  
      ShiftClock(port_name)      {}  
      ShiftClockEn(port_name)    {}  
      ShiftCaptureClock(port_name) {}  
      ScanEn(port_name)          {}  
      CaptureEn(port_name)       {}  
      LbistEn(port_name)         {}  
      DiagClockEn(port_name)     {}  
      ClockOut(port_name)        {}  
      InjectTck(port_name)       {}  
    }  
  }  
}
```

Description

Describes the interface for one third-party OCC already inserted in the design.

During hybrid IP insertion, the process_dft_specification command:

- Rule checks that no Tessent OCC wrappers are specified.
- Rule checks that Tessent OCCs were not inserted in a previous insertion pass. You may have Tessent OCCs present, but they cannot be configured for use with LogicBIST.
- Rule checks that the NcpIndexDecoder wrapper is not specified.
- Validates that the ThirdPartyOCC TCD includes all required interface pins and that mutually exclusive pins are not specified. For example, if one OCC uses a shift clock and another uses a shift clock enable, the tool issues an error.
- Traces each intercepted OCC pin—that is, ShiftClock and ScanEn—to find a common source. This helps prevent redundant muxes from being inserted in the LogicBIST controller.
- Disconnects non-intercept OCC pins (except for ClockOut and InjectTck) and re-connects them to the LogicBIST controller source pin.

In Tessent Shell, all TCDs are automatically organized under the Core wrapper, which is unique for a given module name. Depending on the design, the Core wrapper may include descriptions for other core elements in addition to ThirdPartyOCC descriptions. Refer to “[Tessent Core Description](#)” in the *Tessent Shell Reference Manual* for more information.

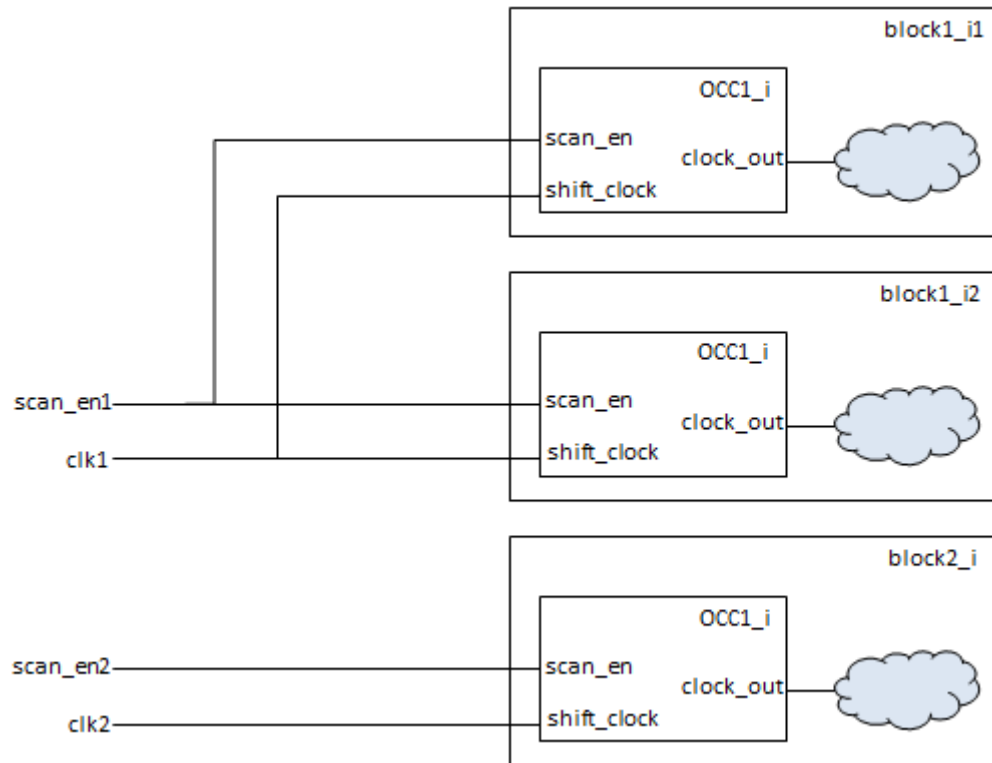
Arguments

- `Interface/ScanEn(port_name)`
A wrapper that specifies the clock controller scan enable input. The inverted LogicBIST capture enable output from the controller in LogicBIST mode drives this port.
- `Interface/CaptureEn(port_name)`
A wrapper that specifies the capture enable trigger LogicBIST clock enable input and is typically connected to the LogicBIST capture enable output.
- `Interface/ShiftClock(port_name)`
A wrapper that specifies the clock controller shift clock input. The LogicBIST shift clock output from the controller in LogicBIST mode drives this pin. This property is mutually exclusive with `ShiftClockEn`.
- `Interface/ShiftClockEn(port_name)`
A wrapper that specifies the clock controller shift clock enable input. The LogicBIST shift clock enable output from the controller in LogicBIST mode drives this pin. This property is mutually exclusive with `ShiftClock`.
- `Interface/ShiftCaptureClock(port_name)`
A wrapper property that specifies an input on the clock controller that needs to receive a clock that pulses during shift and capture. This behavior differs from the shift clock, which only pulses during the shift mode operation. The LogicBIST shift capture clock output from the controller in LogicBIST mode drives the `ShiftCaptureClock` pin. If this pin is not specified, the output port will not be created.
- `Interface/LbistEn(port_name)`
A wrapper that specifies the LogicBIST enable pins.
- `Interface/DiagClockEn(port_name)`
A wrapper that specifies the clock controller diagnosis clock enable input. The LogicBIST controller `DiagClockEn` output drives this pin. If you are using the `ShiftClockEn` clock controller pin, you need a `DiagClockEn` connection to perform LogicBIST diagnosis. The `DiagClockEn` signal is only allowed when `DftSpecification/LogicBist/Controller/SingleChainForDiagnosis/present` is on.
- `Interface/InjectTck(port_name)`
A wrapper that specifies the clock controller pin that injects TCK onto the clock. When MemoryBIST is present, the tool connects this port to the `tck_select` DFT signal, if present.

Usage Example for the ThirdPartyOCC TCD

The following example shows that there are three instances of one OCC module. The shift clock and scan enable on two of the instances share a common source.

Figure 11-2. Usage Example for Third-Party OCC



The TCD file is:

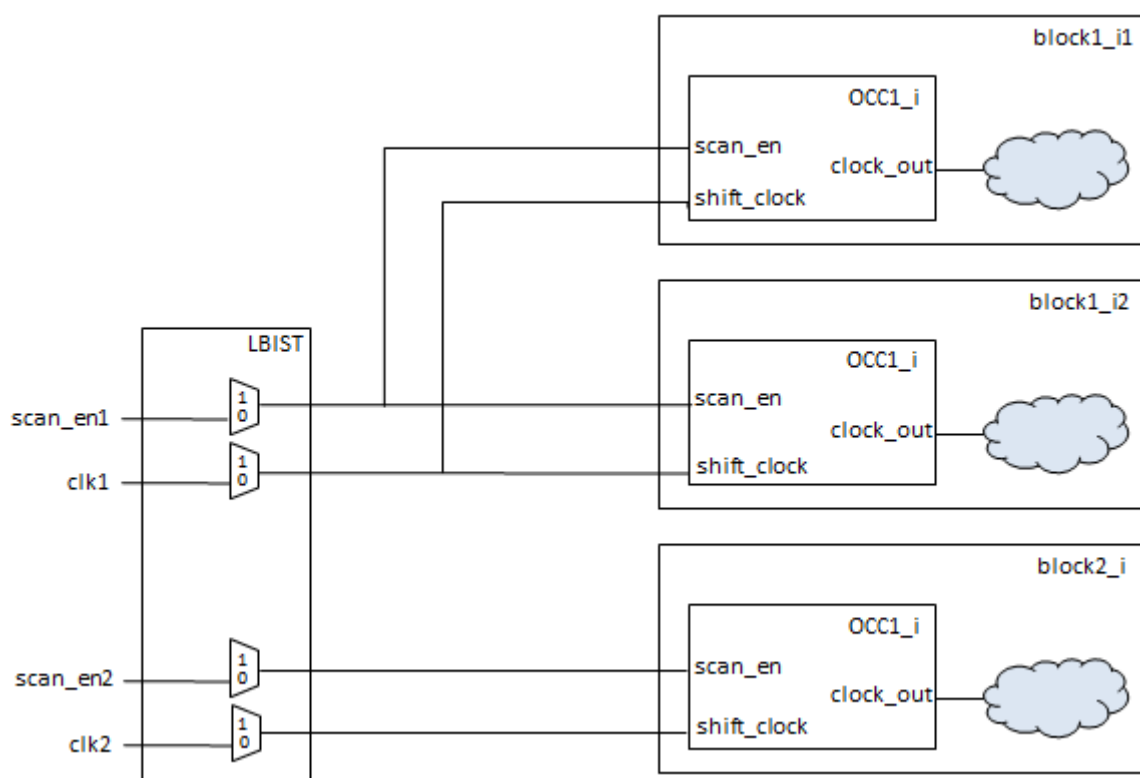
```
Core(OCC1) {
  ThirdPartyOCC {
    Interface {
      ScanEn(scan_en) {}
      ShiftClock(shift_clock) {}
      ClockOut(clock_out) {}
    }
  }
}
```

When running Tessent Shell, the relevant commands are:

```
...
SETUP> read_core_descriptions OCC1.tcd
SETUP> add_core_instances -module OCC1
...
SETUP> set_system_mode analysis
ANALYSIS> create_patterns_specification -sri_sib_list {edt lbist}
...
ANALYSIS> process_dft_specification
```

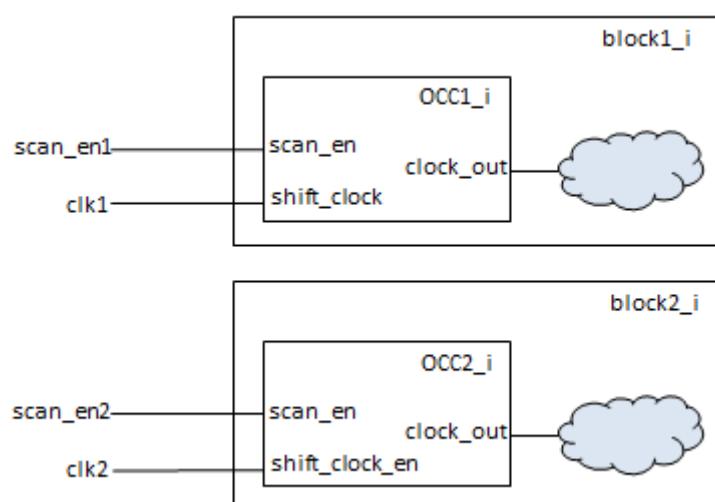
When Tessent Shell inserts the LogicBIST IP, it sees that block1_i1/OCC1_i and block1_i2/OCC1_i share the same shift clock and scan enable sources, and it only inserts one shift clock and scan enable mux, as shown below.

Figure 11-3. Usage Example for Third-Party OCC



In the following example, there are two OCC modules and one instance of each. One OCC TCD specifies the shift clock input while the other OCC TCD specifies the shift clock enable input.

Figure 11-4. Usage Example for Third-Party OCC



The TCD file is:

```
Core(OCC1) {
  ThirdPartyOCC {
    Interface {
      ScanEn(scan_en) {}
      ShiftClock(shift_clock) {}
      ClockOut(clock_out) {}
    }
  }
}
Core(OCC2) {
  ThirdPartyOCC {
    Interface {
      ScanEn(scan_en) {}
      ShiftClockEn(shift_clock_en) {}
      ClockOut(clock_out) {}
    }
  }
}
```

In this case, the tool issues an error because the TCD files have mutually exclusive ports.

```
...
SETUP> read_core_descriptions OCC1.tcd
SETUP> add_core_instances -module {OCC1 OCC2}
...
ANALYSIS> process_dft_specification
...
// Error: /DftSpecification(xxx,gate)/LogicBist
// The 'ShiftClock' and 'ShiftClockEn' ports on third party OCC
modules are mutually exclusive.
// OCC modules with 'ShiftClock' port:
// OCC1
// OCC modules with 'ShiftClockEn' port:
// OCC2
```


Appendix A

EDT Pattern Generation for the Hybrid IP

The EDT technology within the hybrid IP functions as described in the *Tessent TestKompress Users Manual*. In addition, EDT mode contains functionality specific to the hybrid IP.

EDT Mode Initialization with IJTAG	199
The EDT Setup iProc.....	199
Usage Examples	200

EDT Mode Initialization with IJTAG

IJTAG designs contain hardware components such as TDRs and SIBs, and an ICL network to provide access to IJTAG compatible instruments. The design should have either a TAP controller or top-level pins to operate the IJTAG circuitry. ICL describes the instruments and access network, and PDL describes the procedures for operating the instruments. The hybrid IP always includes IJTAG hardware and generates ICL and PDL (iProcs) for hybrid LBIST and EDT instruments.

Using iProcs is required for LBIST mode. Different LBIST pattern sets such as runtime programmable, hardware default, and diagnostics use different LBIST mode iProcs.

Using iProcs is optional for EDT mode with one exception. When EDT reset is not synthesized, the tool must use the EDT setup iProc to generate the initialization sequence for the chain masking register. In this case, the tool automatically provides the values for the EDT static control signals based on the iProc parameters. When iProcs are not used, the test procedures should provide the values required for EDT static control signals.

You can initialize the chain masking register either by adding an EDT reset signal to the hardware or by using IJTAG for seeding values into this register. By default, the tool initializes the chain masking register by using IJTAG. This reduces the need for extra hardware.

You can change the default behavior by synthesizing the EDT reset signal. Specify the following command during IP creation:

```
set_edt_options -reset asynchronous
```

The EDT Setup iProc

During the EDT and logicBIST generation step, you use the `write_edt_files` command to generate several EDT files needed for subsequent flow steps, such as test pattern generation.

Among these files, the tool creates a PDL file that is linked to a generated ICL module. The PDL file includes one EDT iProc named “setup” in addition to several LogicBIST iProcs for LBIST mode.

The tool-generated EDT setup iProc contains parameters for EDT static signals (EDT bypass, EDT single-chain bypass, EDT low power and EDT dual configuration), the EDT reset signal, and serial-load initializing of chain mask registers. The parameters for the EDT static signals and EDT reset are present in the iProc only when the corresponding hardware is synthesized. You use the static EDT control signals as you would for non-hybrid EDT.

The default values for the iProc parameters are:

- `edt_reset` = on
- `edt_bypass` = off
- `edt_single_bypass_chain` = off
- `edt_configuration` = low compression configuration
- `edt_low_power_shift_en` = on when the EDT low-power hardware is synthesized and enabled during IP creation and off when it is disabled

Refer to “[EDT IP Setup for IJTAG Integration](#)” for more information.

Usage Examples

To add parameters to the setup iProc, specify iCalls with the `set_test_setup_icall` command in the EDT pattern generation dofile.

For the following usage examples, suppose you have a tool-generated setup iProc as shown in [Example A-1](#):

Figure A-1. Example of Tool-Generated Setup iProc

```
iProcsForModule piccpu_edt
iProc setup {args} {
    if {[expr [llength $args]%2 != 0]} {
        display_message -error "Odd number of arguments. Expecting parameter
and value pairs."
        return -code error
    }

    set edt_reset 1
    set edt_configuration low_compression_cfg
    set edt_low_power_shift_en 1
    set edt_bypass 0
    set edt_single_bypass_chain 0
    set edt_chain_mask 1111111111111111
    set iwrite_chain_mask 0
    foreach {param value} $args {
        set param [string tolower $param]
        if {$param == "edt_reset"} {
            set edt_reset $value
        } elseif {$param == "edt_configuration"} {
            set edt_configuration $value
        } elseif {$param == "edt_low_power_shift_en"} {
            set edt_low_power_shift_en $value
        } elseif {$param == "edt_bypass"} {
            set edt_bypass $value
        } elseif {$param == "edt_single_bypass_chain"} {
            set edt_single_bypass_chain $value
        } elseif {$param == "tessent_chain_masking"} {
            foreach chain $value {
                if {$chain < 1 || $chain > 16} {
                    display_message -error "Invalid chain index '$chain'. Must
be 1 to 16."
                    return -code error
                }
                set pos [expr {[string length $edt_chain_mask]-$chain}]
                set edt_chain_mask [string replace $edt_chain_mask $pos $pos 0]
            }
            set iwrite_chain_mask 1
            continue
        } else {
            display_message -error "Invalid parameter '$param'. Valid
parameters are 'edt_reset', 'edt_configuration',
'edt_low_power_shift_en', 'edt_bypass', 'edt_single_bypass_chain' and
'tessent_chain_masking'."
            return -code error
        }
        if {$param == "edt_configuration"} {
            if {$value == "low_compression_cfg" || $value ==
"high_compression_cfg" || [string is boolean -strict $value]} {
                set edt_configuration $value
            } else {
                display_message -error "Invalid EDT configuration value
```

```
'$value'. Valid values are boolean, 'low_compression_cfg' and
'high_compression_cfg'."
    return -code error
}
} elseif {[string is boolean -strict $value]} {
    display_message -error "Invalid non-boolean value '$value' for
parameter '$param'."
    return -code error
}
}

if {$edt_reset} {
    iWrite edt_reset on
    iApply
    iWrite edt_reset off
    iApply
}
if {[string is boolean -strict $edt_configuration]} {
    iWrite edt_configuration $edt_configuration
} elseif {$edt_configuration} {
    iWrite edt_configuration high_compression_cfg
} else {
    iWrite edt_configuration low_compression_cfg
}
}
if {$edt_low_power_shift_en} {
    iWrite edt_low_power_shift_en on
} else {
    iWrite edt_low_power_shift_en off
}
}
if {$edt_bypass} {
    iWrite edt_bypass on
} else {
    iWrite edt_bypass off
}
}
if {$edt_single_bypass_chain} {
    iWrite edt_single_bypass_chain on
} else {
    iWrite edt_single_bypass_chain off
}
}
if {$iwrite_chain_mask} {
    iWrite bist_chain_mask 0b$edt_chain_mask
    iWrite bist_chain_mask_load_en 0b0
}
}
iApply
}
```

Example 1: EDT Reset

Suppose the EDT reset signal is asserted as part of test_setup—for example, through power-on-reset—and you do not want to pulse edt_reset in the EDT setup iProc. Specify the following command:

```
set_test_setup_icall {piccpu_edt_i.setup edt_reset off}
```

Example 2: Static Chain Masking

To support static chain masking, the tool uses the `tessent_chain_masking` parameter. The term “internally” distinguishes this masking, which is performed by the IP, from masking performed outside the IP by user-added chain masking.

The value of this parameter is a Tcl list of chain indexes, counted from 1 for the first chain connected to the specified EDT block.

The tool handles internally masked chains as follows depending on whether the `edt_reset` parameter is present in the setup iProc:

- Without `edt_reset`, always initialize the chain mask register.
- With `edt_reset`, initialize the chain mask register only when the parameter is explicitly specified. You can use an empty list (`{}`) to load the chain mask register to all 1's (unmasked).

Suppose the design has `edt_reset` and was pulsed in `test_setup`, but you want to serial load the mask chain register for EDT mode later in the setup iProc. Specify the following command:

```
set_test_setup_ical {piccpu_edt_i.setup edt_reset off tessent_chain_masking {}}
```

In the following example, assume that chains “chain1” and “chain2” are broken and need to be output masked for EDT. Also assume they are the first 2 scan chains in the design. Chain indices 1 and 2 corresponds to chains “chain1” and “chain2.”

```
add_chain_mask chain1 chain2 -unload 0
```

```
set_test_setup_ical {piccpu_edt_i.setup tessent_chain_masking {1 2}}
```

Example 3: EDT Static Control Signals

In the following example, the first iCall generates patterns with EDT low power disabled and with a high compression configuration. The second iCall generates uncompressed EDT bypass patterns.

```
set_test_setup_ical {piccpu_edt_i.edt_low_power_shift_en off edt_configuration  
high_compression_cfg}
```

```
set_test_setup_ical {piccpu_edt_i.edt_bypass on}
```


Appendix B Interface Pins

The interface pins on the various hybrid IP hardware components allow you to control various aspects of the hybrid IP.

LogicBIST Controller Pins	205
Clock Controller Pins	206
EDT/LogicBIST Wrapper Pins	207
Segment Insertion Bit Signals	208

LogicBIST Controller Pins

The following table lists the pin names on the LogicBIST controller.

Table B-1. LogicBIST Controller Pins

Pin Name	Type	Description
capture_dr: ijtag_se	Input	capture_dr output pin from the TAP controller.
clock: shift_clock_src	Input	LogicBIST clock pins.
edt_lbist_clock	Output	Drive the edt_clock port on the EDT block(s) and supply the lbist_clock during LBIST operation.
edt_update	Input	Used by the shift_capture_clock gater duplicated inside the LBIST controller.
control_point_en: control_point_en	Output	Control point enable pins.
x_bounding_en: x_bounding_en	Output	X-bounding enable pins.
mcp_bounding_en: mcp_bounding_en	Output	MCP bounding enable pins.
observe_point_en: observe_point_en	Output	Observe point enable pins.
scan_en: scan_en_in	Input	Scan enable pin and, optionally, the internal pin corresponding to the pad input.
scan_en: scan_en_out	Output	Scan enable pin and, optionally, the internal pin corresponding to the pad output.
setup_shift_scan_in: ijtag_si	Input	Seeding register input connection pin.

Table B-1. LogicBIST Controller Pins (cont.)

Pin Name	Type	Description
setup_shift_scan_out: ijtag_so	Output	Seeding register output connection pin.
shift_dr: ijtag_se	Input	shift_dr output from the TAP controller.
tap_instruction_decode: ijtag_sel	Input	Instruction decoder output connection.
tck: ijtag_tck	Input	JTAG TCK input.
test_en: test_en_in	Input	Test enable pins.
test_en: test_en_out	Output	Test enable pins.
test_logic_reset: ijtag_reset	Input	test_logic_reset output connection from the TAP controller.
update_dr: ijtag_ue	Input	update_dr output from the TAP controller.

Clock Controller Pins

The following table lists the pin names on the clock controller for connecting to your clock controller in the design.

Table B-2. Clock Controller Pins

Pin Name	Type	Description
capture_procedure_index: ncp	Output	NCP signal that identifies the NCP used for the current pattern.
capture_phase	Output	Indicates Capture BIST controller FSM is in capture state.
lbist_en: lbist_en	Output	LogicBIST enable pins.
scan_en: shift_phase	Output	Indicates BIST controller FSM is in shift state.
shift_clock_en: shift_clock_en	Output	Clock controller shift clock enable pins.
shift_clock: shift_clock_in	Input	Clock controller shift clock pins.
shift_clock: shift_clock_out	Output	Clock controller shift clock pins.
scan_en: shift_en_in	Input	Clock controller scan enable pins.
scan_en: shift_en_out	Output	Clock controller scan enable pins.
shift_capture_clock: shift_capture_clock_out	Output	Supply the Tessent OCC slow_clock pins with a free running clock during LBIST mode.

Table B-2. Clock Controller Pins (cont.)

Pin Name	Type	Description
shift_capture_clock: shift_capture_clock_in	Input	During ATPG mode, shift_capture_clock_out provides gated shift capture clock or PI clock source, via this port.
capture_en: capture_en_in	Input	OCC capture enable pins.
capture_en: capture_en_out	Output	OCC capture enable pins.
edt_clock_en	Input	LPCT is required this new input port will be used by the gater creating the edt_lbist_clock

EDT/LogicBIST Wrapper Pins

The following table lists the pin names for the LogicBIST-related pins on the EDT module.

Table B-3. EDT/LogicBIST Wrapper Pins

Pin Name	Type	Description
edt_clock	Input	Clock for shared EDT/BIST IP.
lbist_en	Input	Active high enable signal that indicates BIST mode of operation.
misr_accumulate_en	Input	Active high enable signal that allows MISR to compress input.
lbist_misr	Output	A parallel output bus that combines all the MISRs inside the EDT block. For example, if a design contains two 24-bit MISRs, then this pin is a 48-bit output. The tool does not automatically connect this signal to any design pins.
lbist_prpg_en	Input	Active high enable signal that allows decompressor to run as PRPG.
lbist_reset	Input	Synchronous reset signal from the LogicBIST controller.
ijtag_si	Input	Shift register input for LogicBIST register seeding.
ijtag_so	Output	Shift register output for LogicBIST register seeding.

Segment Insertion Bit Signals

The following table lists the IJTAG-style signals for controlling the SIBs inside the shared EDT/LogicBIST IP.

Table B-4. SIB Pins

Pin Name	Type	Description
sib_capture_en	Input	Active high signal that indicates TAP is in capture-DR state.
sib_en	Input	Active high enable SIB enable signal, connected to decoded LogicBIST instruction from TAP controller.
sib_reset	Input	Active low signal that indicates TAP is in test logic reset state.
sib_shift_en	Input	Active high signal that indicates TAP is in shift-DR state.
sib_update_en	Input	Active high signal that indicates TAP is in update-DR state.
tck	Input	1149.1 test clock input.

Appendix C

File Examples for the Dofile Flow

This appendix provides various examples for the Hybrid Tk/LBIST flow when you are using the dofile flow.

Synthesis Script Example	209
Timing Script Example	211
ICL Example	212
PDL Example	214

Synthesis Script Example

The following example excerpt shows the tool-produced script for synthesizing the EDT/LogicBIST logic.

```
#####
# Synopsys Design Compiler synthesis script for created_edt.v
# Tessent TestKompress version: 2013.1-snapshot_2013.01.29_06.01
# Date: Tue Jan 29 11:21:40 2013
#####

# Bus naming style for Verilog
set bus_naming_style {%s[%d]}
set hdlin_ff_always_async_set_reset true

# Read input design files
read_file -f verilog created_edt.v
read_file -f verilog created_lbist.v

# Synthesize EDT IP
current_design my_core_edt

# Check design for inconsistencies
check_design

# Timing specification
create_clock -period 10 -waveform {0 5} edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_clock_transition 0.0 edt_clock
set_dont_touch_network edt_clock

# Avoid reset signal buffering during synthesis. However, remember
# to perform reset tree synthesis later for edt_reset
set_drive 0 edt_reset
set_max_fanout 1000 edt_reset

# Avoid assign statements in the synthesized netlist.
set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants

# Compile design
uniquify
compile -map_effort medium

# Report design results for EDT IP
report_area > created_dc_script_report.out
report_constraint -all_violators -verbose >> created_dc_script_report.out
report_timing -path full -delay max >> created_dc_script_report.out
report_reference >> created_dc_script_report.out

write -f verilog -hierarchy -o created_my_core_edt_gate.v

# Synthesize single chain mode logic
current_design my_core_single_chain_mode_logic
...
```

Timing Script Example

The following example excerpt shows a tool-produced timing script.

```
*****
# Timing constraints for EDT/LBIST logic in module my_core during LBIST
# mode
# Tessent TestKompress version: 2013.1-snapshot_2013.01.29_06.01
# Date: Tue Jan 29 11:21:41 2013
*****

# The following variables can be set to customize this script before
# sourcing this file:
# lbist_clock_latency_min [default=0]
# lbist_clock_latency_max [default=0]
# lbist_clock_uncertainty_setup [default=0]
# lbist_clock_uncertainty_hold [default=0]
#
# tck_clock_latency_min [default=0]
# tck_clock_latency_max [default=0]
# tck_clock_uncertainty_setup [default=0]
# tck_clock_uncertainty_hold [default=0]
#
# lbist_pins_input_delay [default=0]
# lbist_pins_output_delay [default=0]
#
# reg_suffix [default=_reg]
#
# reg_output [default=Q]
#

# Create lbist clock
proc create_lbist_clock {} {
    create_clock -period 40 -waveform {20 30} refclk

    global lbist_clock_latency_min lbist_clock_latency_max
    lbist_clock_uncertainty_setup lbist_clock_uncertainty_hold
    set cmin [expr {[info exists lbist_clock_latency_min] ?
$lbist_clock_latency_min : 0}]
    set cmax [expr {[info exists lbist_clock_latency_max] ?
$lbist_clock_latency_max : 0}]
    set csetup [expr {[info exists lbist_clock_uncertainty_setup] ?
$lbist_clock_uncertainty_setup : 0}]
    set chold [expr {[info exists lbist_clock_uncertainty_hold] ?
$lbist_clock_uncertainty_hold : 0}]

    set all_lbist_clocks [list refclk]
    set_clock_latency -min $cmin $all_lbist_clocks
    set_clock_latency -max $cmax $all_lbist_clocks
    set_clock_uncertainty -setup $csetup $all_lbist_clocks
    set_clock_uncertainty -hold $chold $all_lbist_clocks
}
...
```

ICL Example

The following example shows an ICL output written by the tool.

```
//*****
// ICL script for module my_core
//
// Tessent TestKompress version: 2013.1-snapshot_2013.01.29_06.01
// Tue Jan 29 06:11:31 GMT 2013
// Date: 01/29/13 11:21:40
//*****

Module my_core_lbist { // {{{
    TCKPort      tck;
    ClockPort    edt_clock;
    ClockPort    shift_clock_src;
    ToClockPort  lbist_clock { Source shiftClkSelected_MUX; }
    ScanInPort   from_edt_scan_out;
    ScanInPort   lbist_scan_in;
    ScanOutPort  lbist_scan_out { Source lbist_scan_out_ff; }
    DataOutPort  lbist_en { Source bist_en; }
    ResetPort    sib_reset;
    SelectPort    sib_en;
    CaptureEnPort sib_capture_en;
    ShiftEnPort   sib_shift_en;
    UpdateEnPort  sib_update_en;
    ToSelectPort  edt_sib_en { Source
my_core_lbist_edt_sib_i.to_enable; }

    ScanInterface host {
        Port lbist_scan_in;
        Port lbist_scan_out;
    }
    ScanInterface client {
        Port from_edt_scan_out;
        Port lbist_scan_out;
        Port edt_sib_en;
    }

    Alias lbist_ctrl_sib      = my_core_lbist_sib_bist_registers_i.sib;
    Alias lbist_ctrl_signals_sib
= my_core_lbist_sib_control_registers_i.sib;
    Alias bist_done          = bist_en { RefEnum YesNo; }

    Instance my_core_lbist_edt_sib_i Of my_core_lbist_sib {
        InputPort reset      = sib_reset;
        InputPort enable     = sib_en;
        InputPort scan_in    = lbist_scan_in;
        InputPort capture_en = sib_capture_en;
        InputPort shift_en   = sib_shift_en;
        InputPort update_en  = sib_update_en;
        InputPort tck        = tck;
        InputPort from_scan_out = from_edt_scan_out;
    }

    //
    // Bist registers
    //
    ScanRegister capture_phase_size[1:0] {
        ScanInSource my_core_lbist_edt_sib_i.scan_out;
    }
}
```

```

ScanRegister shift_clock_select[1:0] {
    ScanInSource    capture_phase_size[0];
    ResetValue      2'b01;
}
ScanRegister warmup_pattern_count[8:0] {
    ScanInSource    shift_clock_select[0];
}
...

```

PDL Example

The typical PDL written out by the tool contains the `lbist_setup_and_clock_verification`, `run_lbist_normal`, `scan_unload_register`, and `run_lbist_hardware_default` iProcs.

```

iProcsForModule my_core
iProc lbist_setup_and_clock_verification { {clock_select lbist_clock}
{mode_name lbist} {one_pattern_per_ncp 0} {teststep_name ""} } {
    ...
}

iProc run_lbist_normal { {clock_select lbist_clock} {begin_pattern 0}
{end_pattern 31} {mode_name lbist} {warmup_pattern_count 0} {misr_compares
1} {sim_monitor off} {teststep_name ""} } {
    if { [string tolower $clock_select] == "tck" } {
        set shift_clock_select 0b10 ; # TCK
    } elseif { [string tolower $clock_select] == "lbist_clock" } {
        set shift_clock_select 0b01 ; # shift_clock_src
    } elseif { [string tolower $clock_select] == "edt_clock" } {
        set shift_clock_select 0b11 ; # ltest_clock
    } else {
        display_message -error "Unrecognized value ($clock_select) for
clock_select. Allowed values are TCK, shift_clock_src and lbist_clock."
        return -code error
    }
}
...

```

```

set patterns_per_compare [expr ($end_pattern - $begin_pattern + 1) / $misr_compares]
set next_begin_pattern $begin_pattern
for {set compare_index 0} {$compare_index < $misr_compares} {incr compare_index} {
    ...
    array set lbist_data [get_logic_bist_pattern_data -design_name my_core -mode_name
$mode_name -begin_pattern $next_begin_pattern -end_pattern $next_end_pattern -
warmup_pattern_cnt $warmup_pattern_count -sim_monitor $sim_monitor]

    ...

    # Define some TCL variables for the values we want to load on the setup chain
    set chain_mask(B1)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.chain_mask,init_value)
    set chain_mask_load_en(B1)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.chain_mask,load_en)
    set prpg_seed(B1)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lfsm_vec,seed)
    set misr_seed(B1_misr)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.misr,seed)
    set mask_shift_reg_value(B1)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_mask_shift_reg,seed)
    set misr_sig(B1_misr)
    $lbist_data(my_core_gate_tessent_edt_lbist_B1_i.misr,expected)

    set chain_mask(B2)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.chain_mask,init_value)
    set chain_mask_load_en(B2)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.chain_mask,load_en)
    set prpg_seed(B2)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lfsm_vec,seed)
    set misr_seed(B2_misr)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.misr,seed)
    set mask_shift_reg_value(B2)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_mask_shift_reg,seed)
    set misr_sig(B2_misr)
    $lbist_data(my_core_gate_tessent_edt_lbist_B2_i.misr,expected)

    set capture_phase_size
    $lbist_data(my_core_gate_tessent_lbist_i.capture_phase_size,init_value)
    set bit_cnt_max [expr [string match -nocase "setup_and_clock_verify"
$sim_monitor] ? 0b10 : $lbist_data(my_core_gate_tessent_lbist_i.bit_cnt_max,init_value)]
    set pattern_length [format %d $bit_cnt_max]
    incr bit_cnt_max -1
    set pattern_cnt [expr $next_end_pattern - $next_begin_pattern +
$warmup_pattern_count + 1 + 1] ; # +1->for pattern range +1-> for the flush pattern

    binary scan [binary format I [expr $pattern_cnt - 1 - $warmup_pattern_count] ] B*
warmup_pattern_count_val
    set warmup_pattern_count_val "0b[string range $warmup_pattern_count_val [expr [string
length $warmup_pattern_count_val] - 9] [expr [string length $warmup_pattern_count_val] -
1]]"

    # When calculating the # of run cycles a capture_phase_size of 0 still results in a
transition through the CAPTURE state but capture_en does not toggle
    set capture_phase_size_eq [scan [regsub {^0x} $capture_phase_size {}] %x]
    if { $capture_phase_size == 0 } {
        set capture_phase_size_eq 1
    }
}

```

```

#                                shift                                Init                                shiftAndCapturePause
capturePhase                    lastPattern                        #                                /
\ /                            \ /                                \ /
set run_cycles [ expr ($pattern_length * $pattern_cnt) + 5 + 12 + 3 + (8 * 2 * $pattern_cnt)
+ ($capture_phase_size_eq * $pattern_cnt) + 8 + 3 ]

set capture_phase_size [format %X $capture_phase_size]
set pattern_cnt_hex    [format %X $pattern_cnt]

set capture_phase_size "0x$capture_phase_size"
set pattern_cnt_hex    "0x$pattern_cnt_hex"

#####
# Setup #
#####

set lbist_full_inst            [get_name_list [get_icl_instances -of_module
my_core_gate_tessent_lbist -below_instances [get_icl_scope]]]
set single_chain_mode_logic_inst [get_name_list [get_icl_instances -of_module
my_core_gate_tessent_single_chain_mode_logic -below_instances [get_icl_scope]]]
set edt_inst0                  [get_name_list [get_icl_instances -of_module
my_core_gate_tessent_edt_lbist_B1 -below_instances [get_icl_scope]]]
set edt_inst1                  [get_name_list [get_icl_instances -of_module
my_core_gate_tessent_edt_lbist_B2 -below_instances [get_icl_scope]]]
set lbist_inst $lbist_full_inst
if {[get_icl_scope] != {}} {
    regsub ^[get_icl_scope]\\.. $lbist_full_inst {} lbist_inst
    regsub ^[get_icl_scope]\\.. $single_chain_mode_logic_inst {}
single_chain_mode_logic_inst
    regsub ^[get_icl_scope]\\.. $edt_inst0 {} edt_inst0
    regsub ^[get_icl_scope]\\.. $edt_inst1 {} edt_inst1
}

# If none of the flops behind a given SIB need an iWrite, because the
# iWrite value is the same as the HW default value, then a sync reset is needed
set inSystemTestInstruemnt [get_icl_instances [regsub {\..host$}
[get_pattern_set_option -active_scan_interfaces] {}] \
-filter
{tessent_instrument_type==mentor::in_system_test} -silent]
set inSystemTestPattern [expr {[sizeof_collection $inSystemTestInstruemnt] > 0}]
set iwrites_needed [dict create]
dict set iwrites_needed edt_inst0_prpg [expr {$prpg_seed(B1) != 0x1614a55d ||

$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_hold_reg,init_value) != 0b1001 ||

$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_toggle_reg,init_value) != 0b0111 ||

$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_switching_reg,init_value) != 0b0010
||

$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_mask_shift_reg,init_value) !=
0x490a4901 ||

!$inSystemTestPattern}] dict set
iwrites_needed edt_inst0_mask [expr {$chain_mask(B1) != 0b11111111 ||
!$inSystemTestPattern}] dict set
iwrites_needed edt_inst0_misr [expr {$misr_seed(B1_misr) != 0x000000 ||
!$inSystemTestPattern}] dict set
iwrites_needed edt_inst1_prpg [expr {$prpg_seed(B2) != 0x282d4aba ||

```



```

$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_hold_reg,init_value) != 0b1001 ||

$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_toggle_reg,init_value) != 0b0111 ||

$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_switching_reg,init_value) != 0b0010
||

$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_mask_shift_reg,init_value) !=
0x12149202 ||

                                !$inSystemTestPattern}]      dict set
iwrites_needed edt_inst1_mask [expr {$chain_mask(B2) != 0b11111111 ||
                                !$inSystemTestPattern}]      dict set
iwrites_needed edt_inst1_misr [expr {$misr_seed(B2_misr) != 0x000000 ||
                                !$inSystemTestPattern}]      dict set
iwrites_needed lbist [expr {$capture_phase_size != 3 ||
                                $warmup_pattern_count_val != 0b001100100 ||
                                $bit_cnt_max != 0b000011111 ||
                                !$inSystemTestPattern}]

$pattern_cnt != 101 ||

                                !$inSystemTestPattern}]

    set msg "Setting up controller $lbist_full_inst"
    if {$misr_compares > 1} {
        append msg " (run [expr $compare_index + 1] of $misr_compares)"
    }
    iNote $msg
    iNote "    Number of patterns      : [expr $pattern_cnt - 1] ([expr $pattern_cnt - 1 -
$warmup_pattern_count] + $warmup_pattern_count warm-up patterns)";
    iNote "    Pattern Length          : $pattern_length";
    iNote "    Shift Clk Select          : $shift_clock_select";
    iNote "    Capture Phase Width       : $capture_phase_size Shift Clock Cycles";
    iNote "    B1 PRPG Seed                : $prpg_seed(B1)";
    iNote "    B1 MISR Seed                : $misr_seed(B1_misr)";
    iNote "    B2 PRPG Seed                : $prpg_seed(B2)";
    iNote "    B2 MISR Seed                : $misr_seed(B2_misr)";

    iWrite $lbist_inst.bist_clock_disable 0b1
    iApply
        # Access LongSetup register
    iTake $lbist_inst.bist_en
    iWrite $lbist_inst.bist_en              0b1
    iApply
    iWrite $lbist_inst.bist_setup            LongSetup
    iWrite $lbist_inst.shift_clock_select $shift_clock_select
    iWrite $lbist_inst.x_bounding_en_reg
$lbist_data(my_core_gate_tessent_lbist_i.x_bounding_en_reg,init_value)
    iWrite $lbist_inst.lbist_low_power_shift_en_reg
$lbist_data(my_core_gate_tessent_lbist_i.lbist_low_power_shift_en_reg,init_value)
    iApply
    iWrite $lbist_inst.bist_clock_disable 0b0
    if {$inSystemTestPattern} {
        iWrite $lbist_inst.bist_sync_reset    0b1
    }
    iApply

    # Load the LongSetup register
    if {$inSystemTestPattern} {
        iWrite $lbist_inst.bist_sync_reset    0b0
    }

```

```
    if {[dict get $iwrites_needed edt_inst0_prpg]} {
        iNote "tessent_pragma iWriteVar ${edt_inst0}.lfsm_vec${tessent_pragma_suffix}
${edt_inst0}.lfsm_vec"
        iWrite $edt_inst0.lfsm_vec          $prpg_seed(B1)
        iWrite $edt_inst0.lbist_lp_hold_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_hold_reg,init_value)
        iWrite $edt_inst0.lbist_lp_toggle_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_toggle_reg,init_value)
        iWrite $edt_inst0.lbist_lp_switching_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B1_i.lbist_lp_switching_reg,init_value)
        iNote "tessent_pragma iWriteVar
${edt_inst0}.lbist_lp_mask_shift_reg${tessent_pragma_suffix}
${edt_inst0}.lbist_lp_mask_shift_reg"
        iWrite $edt_inst0.lbist_lp_mask_shift_reg $mask_shift_reg_value(B1)
    }
    if {[dict get $iwrites_needed edt_inst0_mask]} {
        iWrite $edt_inst0.bist_chain_mask      $chain_mask(B1)
        iWrite $edt_inst0.bist_chain_mask_load_en $chain_mask_load_en(B1)
    }
    if {[dict get $iwrites_needed edt_inst0_misr]} {
        iNote "tessent_pragma iWriteVar ${edt_inst0}.misr_setup${tessent_pragma_suffix}
${edt_inst0}.misr"
        iWrite $edt_inst0.misr          $misr_seed(B1_misr)
    }
    if {[dict get $iwrites_needed edt_inst1_prpg]} {
        iNote "tessent_pragma iWriteVar ${edt_inst1}.lfsm_vec${tessent_pragma_suffix}
${edt_inst1}.lfsm_vec"
        iWrite $edt_inst1.lfsm_vec          $prpg_seed(B2)
        iWrite $edt_inst1.lbist_lp_hold_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_hold_reg,init_value)
        iWrite $edt_inst1.lbist_lp_toggle_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_toggle_reg,init_value)
        iWrite $edt_inst1.lbist_lp_switching_reg
$lbist_data(my_core_gate_tessent_edt_lbist_B2_i.lbist_lp_switching_reg,init_value)
        iNote "tessent_pragma iWriteVar
${edt_inst1}.lbist_lp_mask_shift_reg${tessent_pragma_suffix}
${edt_inst1}.lbist_lp_mask_shift_reg"
        iWrite $edt_inst1.lbist_lp_mask_shift_reg $mask_shift_reg_value(B2)
    }
    if {[dict get $iwrites_needed edt_inst1_mask]} {
        iWrite $edt_inst1.bist_chain_mask      $chain_mask(B2)
        iWrite $edt_inst1.bist_chain_mask_load_en $chain_mask_load_en(B2)
    }
    if {[dict get $iwrites_needed edt_inst1_misr]} {
        iNote "tessent_pragma iWriteVar ${edt_inst1}.misr_setup${tessent_pragma_suffix}
${edt_inst1}.misr"
        iWrite $edt_inst1.misr          $misr_seed(B2_misr)
    }
    if {[dict get $iwrites_needed lbist]} {
        iWrite $lbist_inst.capture_phase_size $capture_phase_size
        iNote "tessent_pragma iWriteVar
${lbist_inst}.warmup_pattern_cnt${tessent_pragma_suffix} ${lbist_inst}.warmup_pattern_cnt"
        iWrite $lbist_inst.warmup_pattern_cnt $warmup_pattern_count_val
        iWrite $lbist_inst.bit_cnt_max      $bit_cnt_max
        iNote "tessent_pragma iWriteVar ${lbist_inst}.vector_cnt${tessent_pragma_suffix}
${lbist_inst}.vector_cnt"
        iWrite $lbist_inst.vector_cnt      $pattern_cnt_hex
    }
}
```

```

if {[length [dict values $iwrites_needed 1]] > 0} {
    if {$last_compare} {
        iWrite $edt_inst0.my_core_gate_tessent_edt_sib_i.sib 0b0
        iWrite $edt_inst0.my_core_gate_tessent_edt_sib_chain_mask_i.sib 0b0
        iWrite $edt_inst1.my_core_gate_tessent_edt_sib_i.sib 0b0
        iWrite $edt_inst1.my_core_gate_tessent_edt_sib_chain_mask_i.sib 0b0
    }
    iApply
}

#####
# Run #
#####

iNote "Starting controller $lbist_full_inst in Normal mode, patterns
$next_begin_pattern to $next_end_pattern";
iWrite $lbist_inst.bist_clock_disable 0b1
iApply

iNote "    Checking that the controller $lbist_full_inst DONE signal is NO at the
beginning of the test";
iNote "tessent_pragma iReadVar $lbist_inst.bist_done_start${tessent_pragma_suffix}
${lbist_inst}.bist_done"
iRead $lbist_inst.bist_done          No
iWrite $lbist_inst.bist_setup         NormalLogicBist
iApply

iWrite $lbist_inst.bist_clock_disable 0b0
iApply

if {[info exists lbist_data(monitorInst)] && $sim_monitor ne "off"} {
    iNote "tessent_pragma verillog_insert ->
`LBIST_SIM_MONITOR_INST.$lbist_data(monitorInst).start_monitor;";
}
if { [string tolower $clock_select] == "tck" } {
    iRunLoop $run_cycles -tck
} else {
    iClock $lbist_inst.lbist_clock
    iRunLoop $run_cycles -sck $lbist_inst.lbist_clock
}
if {[info exists lbist_data(monitorInst)] && $sim_monitor ne "off"} {
    iNote "tessent_pragma verillog_insert ->
`LBIST_SIM_MONITOR_INST.$lbist_data(monitorInst).stop_monitor;";
}

#####
# Check Results #
#####

iRelease $lbist_inst.bist_en
iNote "Test Complete for controller $lbist_full_inst";
iNote "    Checking that signal DONE is YES for controller $lbist_full_inst";
iNote "tessent_pragma iReadVar $lbist_inst.bist_done_end${tessent_pragma_suffix}
${lbist_inst}.bist_done"
iRead $lbist_inst.bist_done          Yes
if {[string match -nocase "setup_and_clock_verify" $sim_monitor]} {
    iWrite $lbist_inst.bist_clock_disable 0b1
    iApply
}

```

```
iNote "Checking results of controller $lbist_full_inst";
iWrite $lbist_inst.bist_setup          LongSetup
iApply

iWrite $lbist_inst.bist_clock_disable 0b0
iApply

iNote "    Expected Signature for B1 of controller $lbist_full_inst :
$misr_sig(B1_misr)";
iNote "tessent_pragma iReadVar ${edt_inst0}.misr${tessent_pragma_suffix}
${edt_inst0}.misr"
iRead $edt_inst0.misr          $misr_sig(B1_misr)
iNote "    Expected Signature for B2 of controller $lbist_full_inst :
$misr_sig(B2_misr)";
iNote "tessent_pragma iReadVar ${edt_inst1}.misr${tessent_pragma_suffix}
${edt_inst1}.misr"
iRead $edt_inst1.misr          $misr_sig(B2_misr)
if {$last_compare} {
    iWrite $edt_inst0.my_core_gate_tessent_edt_sib_misr_i.sib 0b0
    iWrite $edt_inst1.my_core_gate_tessent_edt_sib_misr_i.sib 0b0
}
}
if {$last_compare} {
    iWrite $lbist_inst.bist_en          0b0
    iWrite $lbist_inst.bist_setup       Idle
    iWrite $lbist_inst.x_bounding_en_reg 0b0
    iWrite $lbist_inst.lbist_low_power_shift_en_reg 0b0
    iWrite $lbist_inst.my_core_gate_tessent_lbist_sib_control_registers_i.sib 0b0
    iWrite $lbist_inst.my_core_gate_tessent_lbist_edt_sib_i.sib 0b0
}
iApply
incr next_begin_pattern $patterns_per_compare
}
}

iProc scan_unload_register { {clock_select lbist_clock} {begin_pattern 0} {end_pattern 31}
{mode_name lbist} {warmup_pattern_count 0} {misr_compares 1} {sim_monitor off}
{teststep_name ""} } {
    ...
}

iProc run_lbist_hw_default { {mode_name lbist} {sim_monitor off} {teststep_name ""} } {
    ...
}
```

There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

The Tessent Documentation System	221
Mentor Support Services.....	222

The Tessent Documentation System

At the center of the documentation system is the InfoHub that supports both PDF and HTML content. From the InfoHub, you can access all locally installed product documentation, system administration documentation, videos, and tutorials. For users who want to use PDF, you have a PDF bookcase file that provides access to all the installed PDF files.

For information on defining default HTML browsers, setting up browser options, and setting the default PDF viewer, refer to the “[Documentation Options](#)” in the *Mentor Documentation System* manual.

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the **-manual** invocation switch.
- **File System** — Access the Tessent InfoHub or PDF bookcase directly from your file system, without invoking a Tessent tool. For example:

HTML:

```
firefox $MGC_DFT/docs/infohubs/index.html
```

PDF

```
acroread $MGC_DFT/docs/pdfdocs/_bk_tessent.pdf
```

- **Application Online Help** — ou can get contextual online help within most Tessent tools by using the “**help -manual**” tool command. For example:

> help dofile -manual

This command opens the appropriate reference manual at the “dofile” command description.

Mentor Support Services

Mentor provides a range of industry-leading support services that keep design teams productive and up-to-date with Mentor products.

A Mentor support contract includes the following:

- **Software Updates** — Get the latest releases and product enhancements to keep your environment current.
- **Mentor Graphics Support Center** — Access our online knowledge base, personalized to your Mentor products.
- **Support Forums** — Learn, share, and connect with other Mentor users.
- **Technical Support** — Collaborate with Mentor support engineers to solve complex design challenges.
- **Regular Communications** — Receive the latest knowledge base articles and announcements for your Mentor products.
- **Mentor Ideas** — Share ideas and vote for your favorites to shape future products.

More information is available here:

<https://support.mentor.com>

If your site is under a current support contract, but you do not have a Support Center login, register today:

<https://support.mentor.com/register>

Third-Party Information

Provides information on open source and third-party software that may be included in Tessent products.

For third-party information, refer to the [Third-Party Software for Tessent Products](#) document. Additional open source and third-party software information may be found in <your_Mentor_Graphics_documentation_directory>/legal

End-User License Agreement with EDA Software Supplemental Terms

Use of software (including any updates) and/or hardware is subject to the End-User License Agreement together with the Mentor Graphics EDA Software Supplement Terms. You can view and print a copy of this agreement at:

mentor.com/eula

